

1-savol: Python dasturlash tilining yaratilish tarixi, uning klassifikatsiyasi va boshqa dasturlash tillaridan asosiy farqlari

1. Yaratilish tarixi va falsafasi

Python 1980-yillarning oxirida Gollandiyadagi **Centrum Wiskunde & Informatica (CWI)** markazida **Guido van Rossum** tomonidan ishlab chiqilgan.

- **Kelib chiqishi:** Python **ABC** dasturlash tilining vorisi sifatida yaratilgan. ABC tili juda sodda bo'lsada, u operatsion tizim (Amoeba OS) bilan ishlashda qiyinchiliklarga ega edi. Van Rossum ushbu kamchiliklarni bartaraf etib, xatolarni boshqara oladigan (exception handling) yangi til yaratishni maqsad qilgan.
- **Zen of Python:** Pythonning rivojlanishi "Zen of Python" (PEP 20) tamoyillariga asoslanadi. Uning bosh g'oyasi: **"Go'zallik xunuklikdan afzal, soddalik murakkablikdan afzal"**.

2. Python versiyalari va rivojlanish bosqichlari

Python tarixi uchta yirik bosqichga (versiyaga) bo'linadi:

1. **Python 1.0 (1994):** Birinchi barqaror versiya. Unda funksional dasturlash elementlari (`lambda`, `map`, `filter`) paydo bo'lgan.
2. **Python 2.0 (2000):** Bu versiyada ma'lumotlarni avtomatik tozalash (Garbage Collector) va "List Comprehension" kabi muhim xususiyatlar qo'shildi. Python 2.7 bu tarmoqdagi oxirgi versiya bo'lib, uning qo'llab-quvvatlanishi **2020-yil 1-yanvarda** rasman to'xtatildi.
3. **Python 3.0 (2008):** Bu versiya tilni modernizatsiya qilish va kamchiliklarni tozalash uchun chiqarildi. Muhim jihati: u Python 2 bilan "backward compatibility"ga (orqaga qaytish mosligi) ega emas edi, ya'ni Python 2 kodi 3 da ishlamaydi.

So'nggi versiya: Hozirgi kundagi eng so'nggi barqaror versiya — **Python 3.14** (2025-yil oktabrda chiqarilgan).

- **Asosiy yangiligi:** Python 3.13 da boshlangan "Free-threaded" (GIL-siz Python) loyihasi 3.14 da yanada barqarorlashtirildi. Bu esa ko'p yadroli protsessorlarda dasturlarning parallel ishlash samaradorligini oshiradi.

3. Klassifikatsiyasi

Python quyidagi xususiyatlarga ko'ra klassifikatsiya qilinadi:

- **General-purpose (Universal):** U yordamida veb, ma'lumotlar tahlili, AI, o'yinlar va skriptlar yaratish mumkin.

- **High-level (Yuqori darajali):** Dasturchi xotira manzillari yoki protsessor registrlari bilan ishlamaydi.
- **Strongly Typed:** Ma'lumot turlari bilan ishlashda qat'iy (masalan, sonni matnga qo'shishda avtomatik konvertatsiya qilmaydi, xato beradi).

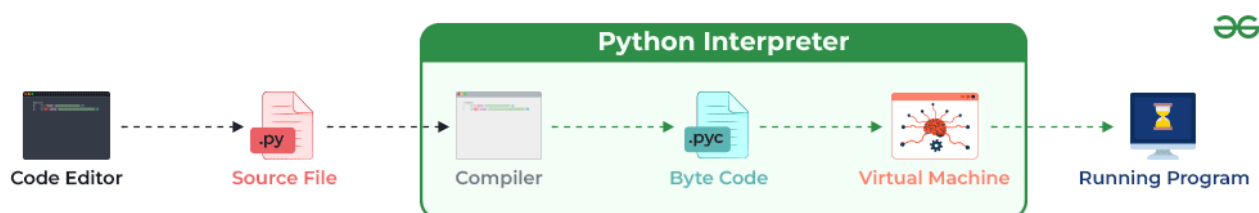
4. Boshqa tillardan asosiy farqlari

1. **Sintaksis va O'qilishi:** C++ yoki Java-da bloklar { } bilan ajratilsa, Pythonda faqat **Indentation** (bo'sh joy) ishlatiladi. Bu kodni "toza" saqlaydi.
2. **Kod uzunligi:** Boshqa tillarda 10 qatorda yoziladigan amalni Pythonda 1-2 qatorda bajarish mumkin.
3. **Dinamik turlash:** O'zgaruvchi turini e'lon qilish shart emas: `x = 5`. C++ da esa `int x = 5;` shart.

Foydalanilgan manbalar:

1. **Wikipedia:** [History of Python](#) — Versiyalar va kelib chiqishi.
2. **Python Foundation:** [General Python FAQ](#) — Til dizayni va falsafasi.
3. **Python.org News:** [Python 3.13.0 Release](#) — So'nggi versiya tafsilotlari.
4. **PEP 20:** [The Zen of Python](#) — Dasturlash tamoyillari.

2-savol: Pythonda interpretator tushunchasi va uning ishlash prinsipi nimalardan iborat?



Python dasturlash tili **interpretatsiya qilinadigan** til hisoblanadi. Bu degani, siz yozgan kod to'g'ridan-to'g'ri protsessor tomonidan tushunilmaydi, balki oraliq dastur — **interpretator** yordamida bajariladi.

1. Interpretator nima?

Interpretator — bu yuqori darajali dasturlash tilida (masalan, Python) yozilgan kodni qatorma-qator o'qib, uni darhol mashina tiliga o'giruvchi va ijro etuvchi dasturdir.

Pythonning standart interpretatori **CPython** deb ataladi (chunki u C tilida yozilgan).

2. Ishlash prinsipi (Bosqichma-bosqich)

Python kodi (.py fayl) ishga tushirilganda jarayon quyidagi 3 ta asosiy bosqichdan o'tadi:

A) Kompilyatsiya (Byte-code yaratish):

Siz .py faylni ishga tushirganingizda, Python interpretatori avval kodni Byte-code (bayt-kod) deb ataladigan oraliq shaklga o'giradi. Bu mashina kodi emas, balki Python uchun optimallashtirilgan past darajali ko'rsatmalardir.

- Bu jarayonda .pyc kengaytmali fayllar (odatda __pycache__ papkasida) hosil bo'ladi.
- Maqsad: Dastur keyingi safar ishga tushganda kodni qayta o'girib o'tirmasdan, tayyor bayt-koddan foydalanish (tezlikni oshirish).

B) Python Virtual Machine (PVM):

Tayyor bayt-kod PVMga yuboriladi. PVM — bu interpretatorning "yuragi" bo'lib, u bayt-kodni qatorma-qator o'qiydi va kompyuterning operatsion tizimi hamda protsessori tushunadigan mashina kodiga aylantirib, natijani chiqaradi.

3. Interpretatorning afzalliklari va kamchiliklari

- **Afzalliklari:**
 - **Portativlik:** Kod har qanday operatsion tizimda (Windows, Linux, Mac) o'zgarishsiz ishlaydi, chunki har bir tizim uchun o'z interpretatori bor.
 - **Debugging (Xatolarni topish):** Kod qatorma-qator bajarilgani uchun xatolik aynan qaysi qatorda ekanligi aniq ko'rsatiladi.
- **Kamchiliklari:**
 - **Tezlik:** Kompilyatsiya qilinadigan tillarga (masalan, C++) qaraganda biroz sekinroq, chunki kod har gal bajarilish jarayonida o'giriladi.

Dasturlash tillari kodning qanday bajarilishiga ko'ra asosan **uchta asosiy turga** bo'linadi. Har bir tur kodni kompyuter protsessori tushunadigan "0" va "1" (mashina kodi) ga aylantirish usuli bilan farqlanadi.

1. Kompilyatsiya qilinadigan tillar (Compiled Languages)

Ushbu tillarda yozilgan kod ishga tushirilishidan oldin maxsus dastur — **kompilyator** yordamida to'liq mashina tiliga o'giriladi va alohida bajariluvchi fayl (masalan, Windowsda .exe fayl) yaratiladi.

- **Xususiyati:** Dastur juda tez ishlaydi, chunki u allaqachon kompyuter tiliga tayyor tayyorlab qo'yilgan.
- **Kamchiligi:** Kodda bitta nuqta o'zgarsa ham, butun dasturni qaytadan kompilyatsiya qilish kerak.
- **Misollar:** C, C++, Rust, Go, Swift.

2. Interpretatsiya qilinadigan tillar (Interpreted Languages)

Bu tillarda kod oldindan o'girilmaydi. Buning o'rniga **interpretator** dasturi kodni o'qiydi va har bir qatorni navbatma-navbat bajaradi.

- **Xususiyati:** Kodni yozish va test qilish juda oson va tez. Har xil operatsion tizimlarda (Windows, Mac, Linux) bir xil ishlaydi.
- **Kamchiligi:** Kompilyatsiya qilinadigan tillarga qaraganda sekinroq ishlaydi (chunki har gal bajarilayotganda "tarjima" qilinadi).
- **Misollar:** Python, JavaScript, PHP, Ruby, Perl.

3. Gibril tillar (Hybrid/JIT Compiled Languages)

Zamonaviy tillarning ko'pchiligi har ikkala usulning afzalliklarini birlashtiradi. Kod avval oraliq shaklga (**Bytecode**) o'giriladi, so'ngra maxsus virtual mashina ichida **JIT (Just-In-Time)** kompilyatori yordamida bajariladi.

- **Xususiyati:** Interpretatsiya qilinadigan tillardan tezroq, lekin sof kompilyatsiya tillaridan biroz sekinroq. Xavfsizlik va platformalararo muvofiqlik darajasi yuqori.
- **Misollar:** Java (JVM yordamida), C# (.NET yordamida).

Qisqacha taqqoslash jadvali

Xususiyat	Kompilyatsiya tillari	Interpretatsiya tillari	Gibril tillar
Bajarilish usuli	To'liq .exe ga o'giriladi	Qatorma-qator o'qiladi	Bayt-kod orqali
Tezlik	Juda yuqori	O'rtacha/Past	Yuqori
Xatolarni topish	Qiyinroq (kompilyatsiyada)	Oson (bajarilishda)	Oson
Misol	C++, Rust	Python, JS	Java, C#

Qiziqarli fakt:

Aslida, Python ham qaysidir ma'noda gibril tilga kiradi. U kodni avval .pyc (Bytecode) ga o'giradi, so'ngra **Python Virtual Machine** uni interpretatsiya qiladi. Lekin biz uni baribir **interpretatsiya qilinadigan til** deb ataymiz, chunki bu jarayon foydalanuvchiga sezilmaydi va kodni darhol ishga tushirish imkonini beradi.

Foydalanilgan manbalar va havolalar:

1. **Python rasmiy hujjatlari:** [What is an Interpreted Language? \(Glossary\)](#)
2. **GeeksforGeeks:** [Internal working of Python](#) — Bayt-kod va PVM haqida batafsil.
3. **Real Python:** [Your Guide to the Python Virtual Machine](#) — Interpretator qanday ishlashi haqida chuqur tahlil.

3-savol: Pythonda o'zgaruvchilar (variables) tushunchasi, ularni nomlash qoidalari va dinamik turlash (dynamic typing) xususiyatini izohlang.

1. O'zgaruvchilar (Variables) tushunchasi

Pythonda o'zgaruvchi — bu ma'lumotlarni (qiymatlarni) kompyuter xotirasida saqlash uchun ishlatiladigan **nomlangan konteynerdir**.

Boshqa ko'plab tillardan (C++, Java) farqli o'laroq, Pythonda o'zgaruvchi deganda "quti" emas, balki xotiradagi obyektga yo'naltirilgan **"ko'rsatkich" (pointer/reference)** tushuniladi. Biz `x = 5` deb yozganimizda, 5 soni obyekt sifatida xotiradan joy oladi, `x` esa shunchaki o'sha obyektga yopishtirilgan **yorliq (label)** bo'ladi.

2. O'zgaruvchilarni nomlash qoidalari (Naming Rules)

Pythonda o'zgaruvchilarga nom berishda quyidagi qoidalarga amal qilish shart:

1. **Belgilar:** Nomlar faqat harflar (A-Z, a-z), raqamlar (0-9) va pastki chiziqdan (`_`) iborat bo'lishi mumkin.
2. **Boshlanishi:** O'zgaruvchi nomi **raqam bilan boshlanishi mumkin emas**. Masalan: `1son` xato, `son1` to'g'ri.
3. **Katta-kichik harf (Case-sensitivity):** Python katta va kichik harflarni farqlaydi. `ism`, `ism` va `ISM` — bular uchta xil o'zgaruvchi.
4. **Kalit so'zlar:** Pythonning band qilingan kalit so'zlarini (`if`, `for`, `while`, `class`, `def` va h.k.) o'zgaruvchi nomi sifatida ishlatib bo'lmaydi.
5. **Bo'sh joy:** Nomlar orasida bo'sh joy (probel) bo'lishi mumkin emas. Buning o'rniga `_` ishlatiladi (Snake Case): `mening_yoshim = 25`.

3. Dinamik turlash (Dynamic Typing)

Python — **dinamik turlanadigan** tildir. Bu uning eng kuchli xususiyatlaridan biri bo'lib, quyidagilarni anglatadi:

- **Turini e'lon qilish shart emas:** O'zgaruvchi yaratayotganda uning turini (masalan, `int` yoki `string`) yozish talab qilinmaydi. Python qiymatga qarab turini o'zi aniqlaydi.
- **Turini o'zgartirish mumkin:** Bitta o'zgaruvchi dastur davomida turli xil ma'lumot turlarini qabul qila oladi.

```
x = 10          # x hozir butun son (int)
print(type(x))
```

```
x = "Salom"    # x endi matnga aylandi (str)
print(type(x))
```

4. Afzalligi va kamchiligi

- **Afzalligi:** Kod qisqa bo'ladi va dasturchi o'zgaruvchi turlari haqida qayg'urmasdan tezroq kod yozadi.
- **Kamchiligi:** Katta loyihalarda ehtiyotsizlik tufayli kutilmagan mantiqiy xatolar (TypeError) kelib chiqishi mumkin (masalan, son kutilgan joyga matn kelib qolishi).

Foydalanilgan manbalar va havolalar:

1. **Python.org Documentation:** [Python Variables and Types](#)
2. **W3Schools:** [Python Variables](#)
3. **Real Python:** [Variables in Python](#) — Xotira bilan ishlash mantiqi haqida batafsil.

4-savol: Pythonda qanday o'rnatilgan (built-in) ma'lumot turlari mavjud?

Pythonda ma'lumotlar turlari o'zgaruvchining qanday qiymat saqlayotganini va u ustida qanday amallar bajarish mumkinligini belgilaydi. Ularni quyidagi asosiy guruhlariga bo'lish mumkin:

1. Sonli turlar (Numeric Types)

- **int (integer):** Butun sonlar. Masalan: 5, -10, 1000.
- **float:** O'nlik kasr sonlar (suzuvchi nuqtali). Masalan: 3.14, -0.001, 2.0.
- **complex:** Kompleks sonlar ($a + bj$ ko'rinishida). Masalan: $3 + 5j$.

2. Matnli tur (Sequence Type)

- **str (string):** Satrlar yoki matnli ma'lumotlar. Qo'shtirnoq yoki bir tirnoq ichida yoziladi. Masalan: "Salom", 'Python'.

3. Ketma-ketlik turlari (Sequence Types)

- **list (ro'yxat):** Tartiblangan va o'zgaruvchan to'plam. Turli xil ma'lumotlarni saqlashi mumkin. Masalan: [1, "olma", 3.5].
- **tuple (kortej):** Tartiblangan, lekin o'zgarmas to'plam. Masalan: (10, 20, 30).
- **range:** Sonlar ketma-ketligini generatsiya qilish uchun ishlatiladi (asosan sikllarda).

4. Lug'at turi (Mapping Type)

- **dict (dictionary):** Ma'lumotlarni "kalit: qiymat" juftligi ko'rinishida saqlaydi. Masalan: {"ism": "Ali", "yosh": 25}.

5. To'plam turlari (Set Types)

- **set:** Tartiblanmagan va unikal (takrorlanmas) elementlar to'plami. Masalan: {1, 2, 3}.
- **frozenset:** set ning o'zgarmas ko'rinishi.

6. Mantiqiy tur (Boolean Type)

- **bool:** Faqat ikkita qiymatni qabul qiladi: `True` (rost) yoki `False` (yolg'on). Shartlarni tekshirishda asosiy rol o'ynaydi.

7. Hech narsa (None Type)

- **NoneType:** O'zgaruvchida hech qanday qiymat yo'qligini bildiradi (`None`).

Ma'lumot turini aniqlash va tekshirish

Pythonda ma'lumot turini bilish uchun `type()` funksiyasidan, ma'lum turga tegishlilikini tekshirish uchun esa `isinstance()` funksiyasidan foydalaniladi:

```
x = 10
print(type(x))          # <class 'int'>
print(isinstance(x, int)) # True
```

Foydalanilgan manbalar va havolalar:

1. **Python Official Docs:** [Built-in Types](#)
2. **W3Schools:** [Python Data Types](#)
3. **Programiz:** [Python Variables, Constants and Literals](#)

5-savol: Pythonda o'zgaruvchining ko'rinish sohasi (Variable Scope) nima?

Variable Scope (o'zgaruvchining ko'rinish sohasi) — bu dasturning qaysi qismida ma'lum bir o'zgaruvchini "ko'rish" (unga murojaat qilish) mumkinligini belgilaydigan qoidalar to'plamidir.

Pythonda o'zgaruvchilarning ko'rinishi **LEGB** qoidasi asosida ishlaydi.

1. LEGB qoidasi nima?

O'zgaruvchini qidirish tartibi quyidagi iyerarxiya bo'yicha amalga oshiriladi:

- **L (Local) — Lokal soha:** Funksiya yoki metod ichida yaratilgan o'zgaruvchilar. Ular faqat o'sha funksiya ichida mavjud bo'ladi.
- **E (Enclosing) — O'rab turuvchi soha:** Ichma-ich joylashgan funksiyalarda (nested functions), ichki funksiya uchun tashqi funksiyaning o'zgaruvchilari ko'rinadigan soha.
- **G (Global) — Global soha:** Dasturning eng yuqori qismida yoki modul darajasida yaratilgan o'zgaruvchilar. Ular butun fayl davomida ko'rinadi.
- **B (Built-in) — O'rnatilgan soha:** Pythonning o'zida mavjud bo'lgan tayyor nomlar (masalan: `print`, `len`, `range`).

2. Global va Local farqi (Misol)

Python

```
x = "Men globalman" # Global o'zgaruvchi

def mening_funksiyam():
    y = "Men lokalman" # Lokal o'zgaruvchi
    print(x) # Globalni funksiya ichida o'qish mumkin
    print(y)

mening_funksiyam()
# print(y) # Xatolik beradi, chunki y tashqarida ko'rinmaydi
```

3. global va nonlocal kalit so'zlari

- **global:** Funksiya ichida turib tashqaridagi (global) o'zgaruvchining qiymatini o'zgartirish uchun ishlatiladi.
- **nonlocal:** Ichma-ich funksiyalarda ichki funksiya tashqi funksiyadagi o'zgaruvchini o'zgartirmoqchi bo'lsa ishlatiladi.

```
count = 0

def increment():
    global count # Global o'zgaruvchiga ruxsat olish
    count += 1

increment()
print(count) # Natija: 1
```

Pythonda o'zgaruvchilar ma'lumotlarni saqlash va boshqarishda asosiy rolni o'ynaydi. Ularning o'zini tutishi va foydalanish imkoniyati (dasturning qaysi qismida ko'rinishi) ularning qayerda aniqlanganiga bog'liq. Ushbu maqolada biz global va lokal o'zgaruvchilar, ularning ishlash prinsipi va keng tarqalgan holatlarni misollar bilan ko'rib chiqamiz.

1. Lokal o'zgaruvchilar (Local Variables)

Lokal o'zgaruvchilar funksiya ichida yaratiladi va faqat funksiya bajarilayotgan vaqtda mavjud bo'ladi. Ularga funksiya tashqarisidan murojaat qilib bo'lmaydi.

1-misol: Funksiya ichida lokal o'zgaruvchi yaratish va unga murojaat qilish.

```
def greet():
    msg = "Funksiya ichidan salom!" # msg - lokal o'zgaruvchi
    print(msg)

greet()
# Natija: Funksiya ichidan salom!
```

Tushuntirish: msg o'zgaruvchisi faqat greet() funksiyasi ichida mavjud. Funksiya chaqirilganda u ekranga chiqariladi.

2-misol: Lokal o'zgaruvchiga funksiya tashqarisidan murojaat qilishga urinish (Xatolik).

Python

```
def greet():
    msg = "Salom!"
    print("Funksiya ichida:", msg)

greet()
# print("Funksiya tashqarisida:", msg)
# Natija: NameError: name 'msg' is not defined (Xatolik: msg aniqlanmagan)
```

Tushuntirish: msg lokal o'zgaruvchi bo'lgani uchun, uni funksiyadan tashqarida chop etishga urinish xatoga olib keladi, chunki u global sohada mavjud emas.

2. Global o'zgaruvchilar (Global Variables)

Global o'zgaruvchilar barcha funksiyalardan tashqarida e'lon qilinadi va ularga dasturning istalgan joyidan, shu jumladan funksiyalar ichidan ham murojaat qilish mumkin.

Misol: Global o'zgaruvchiga funksiya ichida va tashqarisida murojaat qilish.

```
msg = "Python juda ajoyib!" # msg - global o'zgaruvchi

def display():
    print("Funksiya ichida:", msg) # Global o'zgaruvchidan foydalanish

display()
print("Funksiya tashqarisida:", msg)
# Natija:
# Funksiya ichida: Python juda ajoyib!
# Funksiya tashqarisida: Python juda ajoyib!
```

Eslatma: Agar o'zgaruvchi funksiya ichida topilmasa, Python avtomatik ravishda uni global sohadan qidiradi.

3. Lokal va Global o'zgaruvchilardan birgalikda foydalanish

Agar bir xil nomli global va lokal o'zgaruvchi aniqlangan bo'lsa, funksiya ichida lokal o'zgaruvchi globalni **"soyada qoldiradi" (shadowing)**.

```
def fun():
    s = "Men ham." # Lokal o'zgaruvchi
    print(s)

s = "Men GeeksforGeeksni yaxshi ko'raman" # Global o'zgaruvchi
fun()
print(s)
# Natija:
# Men ham.
# Men GeeksforGeeksni yaxshi ko'raman
```

Tushuntirish: `fun()` ichida `s` ning qiymati o'zgaradi, lekin bu global `s` ga ta'sir qilmaydi.

4. Funksiya ichida Global o'zgaruvchini o'zgartirish

Odatda, funksiya ichida global o'zgaruvchini to'g'ridan-to'g'ri o'zgartirib bo'lmaydi. Buning uchun `global` kalit so'zi kerak.

Xato holat (`globalsiz`):

```
def fun():
    # s += ' GFG'  <-- Xatolik beradi
    print(s)

s = "I love GeeksforGeeks"
# fun() # UnboundLocalError: local variable 's' referenced before
assignment
```

To'g'ri holat (`global` bilan):

```
s = "Python juda zo'r!"

def fun():
    global s # s global ekanligini bildiramiz
    s += " GFG" # Global o'zgaruvchini o'zgartiramiz
    print(s)
    s = "GeeksforGeeks Python bo'limini ko'ring" # Qiymatni qayta
    # tayinlaymiz
    print(s)

fun()
print(s)
# Natija:
# Python juda zo'r! GFG
# GeeksforGeeks Python bo'limini ko'ring
# GeeksforGeeks Python bo'limini ko'ring
```

5. Bir xil nomli Global va Lokal o'zgaruvchilar (Qiyosiy misol)

```

a = 1 # Global o'zgaruvchi

def f():
    print("f():", a) # Global 'a' ni ishlatadi

def g():
    a = 2 # Lokal 'a' globalni soyada qoldiradi
    print("g():", a)

def h():
    global a
    a = 3 # Global 'a' ni o'zgartiradi
    print("h():", a)

print("global:", a) # Natija: 1
f() # Natija: 1
print("global:", a) # Natija: 1
g() # Natija: 2
print("global:", a) # Natija: 1
h() # Natija: 3
print("global:", a) # Natija: 3

```

6. Global vs Lokal o'zgaruvchilar (Xulosa jadvali)

Taqqoslash asosi	Global o'zgaruvchi	Lokal o'zgaruvchi
Ta'rif	Funksiyalardan tashqarida e'lon qilinadi.	Funksiya ichida e'lon qilinadi.
Yashash vaqti	Dastur boshlanganda yaratiladi va tugaganda o'chadi.	Funksiya chaqirilganda yaratiladi va qaytganda o'chadi.
Ma'lumot almashish	Barcha funksiyalar o'rtasida umumiy.	Faqat o'zi e'lon qilingan funksiya ichida.
Ko'rinish sohasi	Dasturning istalgan joyida.	Faqat funksiya ichida.
Xotira	Global namespace (global nomlar maydoni).	Funksiya stack frame (stek ramkasi).
Qiymat o'zgarishi	Butun dasturga ta'sir qiladi.	Faqat lokal sohada ta'sir qiladi.

Foydalanilgan manbalar va havolalar:

1. Real Python: [Python Scope & the LEGB Rule](#)
2. GeeksforGeeks: [Global and Local Variables in Python](#)
3. W3Schools: [Python Scope](#)

6-savol: Pythonda "docstring" tushunchasi nima va u nima uchun kerak?

Docstring (Documentation String) — bu Python klassi, moduli yoki funksiyasi nima vazifani bajarishini tushuntirib berish uchun yoziladigan maxsus satrdir. U kodning tushunarlilikini oshirish uchun xizmat qiladi.

1. Docstring qanday yoziladi?

Docstring har doim obyekt (funksiya, klass) aniqlangandan so'ng, uning birinchi qatorida yoziladi va **uchta qo'shtirnoq** ("""...""") yoki **uchta bir tirnoq** ('''...''') ichiga olinadi.

Misol:

Python

```
def daraja(a, b):  
    """  
    Berilgan sonni darajaga ko'tarib qaytaradi.  
  
    Argumentlar:  
    a -- asos (son)  
    b -- daraja ko'rsatkichi (son)  
    """  
    return a ** b
```

2. Nima uchun kerak? (Vazifalari)

- **Kodni hujjatlashtirish:** Dasturchi yozgan funksiyasi nima qilishini eslab qolishi yoki boshqa dasturchilar tushunishi uchun.
- **Avtomatik ma'lumot olish:** IDE'lar (PyCharm, VS Code) funksiya ustiga sichqonchani olib borganda docstringni ko'rsatadi.
- **Hujjat yaratish:** `pydoc` kabi vositalar docstringlardan foydalanib, avtomatik ravishda veb-sahifa ko'rinishidagi qo'llanmalarni yaratadi.

3. Docstringni qanday o'qish mumkin?

Docstringga dastur ichida murojaat qilishning ikkita usuli bor:

1. `__doc__` atributi orqali:

```
print(daraja.__doc__)
```

2. `help()` funksiyasi orqali:

```
help(daraja)
```

4. Docstring va Izoh (Comment) farqi

Ko'pchilik docstringni oddiy izohlar (#) bilan adashtiradi. Ularning asosiy farqlari:

Xususiyat	Izoh (Comment - #)	Docstring ("""...""")
Maqsad	Kodning qanday ishlashini ichkaridan tushuntirish.	Obyekt nima qilishini (interfeysini) tushuntirish.
Dasturda ko'rinishi	Interpretator tomonidan e'tiborsiz qoldiriladi.	<code>__doc__</code> atributi sifatida saqlanadi.
Foydalanish	Kodning o'rtasida istalgan joyda.	Faqat funksiya/klass/modulning boshida.

Foydalanilgan manbalar:

1. **Python.org:** [Docstrings \(PEP 257\)](#) — Docstring yozish standartlari.
2. **W3Schools:** [Python Docstrings](#)
3. **Real Python:** [Documenting Your Python Code](#)

7-savol: Arifmetik operatorlar va ularning ustuvorlik darajalari. // va % operatorlarining farqi.

Pythonda arifmetik operatorlar sonli qiymatlar ustida matematik amallarni bajarish uchun ishlatiladi.

1. Arifmetik operatorlar ro'yxati

Pythonda quyidagi asosiy arifmetik operatorlar mavjud:

Operator	Tavsif	Misol (a=10,b=3)	Natija
+	Qo'shish	a + b	13
-	Ayirish	a - b	7
*	Ko'paytirish	a * b	30
/	Bo'lish (float)	a / b	3.333...
**	Darajaga ko'tarish	a ** b	1000
//	Butun sonli bo'lish	a // b	3
%	Qoldiqli bo'lish	a % b	1

2. // va % operatorlarining farqi

Bu ikki operator bo'lish amali bilan bog'liq, lekin natijasi turlicha:

- **// (Floor division):** Bo'lish natijasidan faqat **butun qismini** oladi (kasr qismini tashlab yuboradi).
 - *Misol:* $17 // 5$ natijasi 3 bo'ladi.
- **% (Modulus):** Bo'lish natijasidan faqat **qoldiqni** qaytaradi.
 - *Misol:* $17 \% 5$ natijasi 2 bo'ladi (chunki $17 = 5 \times 3 + 2$).

3. Operatorlarning ustuvorlik darajasi (Precedence)

Matematikadagi kabi, Pythonda ham amallar ma'lum bir tartibda bajariladi. Bu tartibni **PEMDAS** qoidasi orqali eslab qolish mumkin:

1. **P (Parentheses) — Qavslar:** $()$ ichidagi amallar har doim birinchi bajariladi.
2. **E (Exponents) — Daraja:** $**$ operatori.
3. **MD (Multiplication and Division) — Ko'paytirish va Bo'lish:** $*$, $/$, $//$, $\%$ operatorlari (chapdan o'ngga qarab).
4. **AS (Addition and Subtraction) — Qo'shish va Ayirish:** $+$, $-$ operatorlari.

Misol:

$10 + 5 * 2 ** 2$

1. Daraja: $2^2 = 4$
2. Ko'paytirish: $5 * 4 = 20$
3. Qo'shish: $10 + 20 = 30$

Foydalanilgan manbalar:

1. **W3Schools:** [Python Arithmetic Operators](#)
2. **Programiz:** [Python Operators](#)
3. **Python.org:** [Expressions - Operator precedence](#)

8-savol: Bool (mantiqiy) tipli ma'lumotlar nima va ular dasturlashda qanday holatlarda ishlatiladi? True va False qiymatlari haqida ma'lumot bering.

Bool (Boolean) — bu Python-da eng sodda, ammo eng muhim ma'lumot turlaridan biri bo'lib, u faqat ikkita qiymatdan birini qabul qilishi mumkin: `True` (Rost) yoki `False` (Yolg'on).

1. True va False qiymatlari

- **True:** Mantiqiy "ha", "to'g'ri" yoki "mavjud" degan ma'nolarni anglatadi. Raqamli ifodada u **1** ga teng deb ko'riladi.
- **False:** Mantiqiy "yo'q", "noto'g'ri" yoki "bo'sh" degan ma'nolarni anglatadi. Raqamli ifodada u **0** ga teng.

2. Dasturlashda qo'llanilish holatlari

Boolean qiymatlari dasturning "miyasini" tashkil qiladi va asosan quyidagi joylarda ishlatiladi:

- **Taqqoslash amallarida:** Ikki qiymatni solishtirganda natija har doim `bool` bo'ladi.

Python

```
print(10 > 5)    # Natija: True
print(3 == 4)    # Natija: False
```

- **Shart operatorlarida (`if`, `elif`):** Dastur qaysi yo'ldan ketishini hal qilish uchun.

Python

```
yosh = 20
if yosh >= 18:
    print("Siz ovoz bera olasiz.") # Bu qator True bo'lgandagina
    ishlaydi
```

- **Sikl operatorlarida (while):** Ma'lum bir shart True bo'lib turgunicha amalni qaytarish uchun.

3. "Boolean context" (Haqiqiy va yolg'on ifodalar)

Pythonda har qanday obyektни `bool()` funksiyasi yordamida mantiqiy turga o'tkazish mumkin. Deyarli barcha qiymatlar `True` qaytaradi, faqat quyidagi **"bo'sh"** holatlar `False` qaytaradi:

- `None`
- `0` (nol)
- `""` (bo'sh satr)
- `[]` (bo'sh ro'yxat), `{}` (bo'sh lug'at), `()` (bo'sh kortej)

4. Mantiqiy operatorlar bilan ishlash

Boolean qiymatlarini birlashtirish uchun `and`, `or`, va `not` operatorlari ishlatiladi:

- `True and False -> False`
- `True or False -> True`
- `not True -> False`

Foydalanilgan manbalar:

1. Python Official Docs: [Boolean Values](#)
2. W3Schools: [Python Booleans](#)
3. Real Python: [Operators and Expressions in Python](#)

9-savol: Satrlar (Strings) bilan ishlash: indekslash, qirqib olish (slicing) va satrlarni birlashtirish (concatenation) amallarini tushuntiring.

Python-da satrlar (string) — bu belgilar ketma-ketligi bo'lib, ular bilan ishlash uchun juda qulay imkoniyatlar mavjud.

1. Indekslish (Indexing)

Satrdagi har bir belgi o'z tartib raqamiga ega, buni **indeks** deb ataymiz. Pythonda indekslash **0 dan boshlanadi**.

- **Musbat indekslash:** Chapdan o'ngga qarab (0, 1, 2...).

- **Manfiy indekslash:** O'ngdan chapga qarab (\$-1, -2, -3...\$). Bu satrning oxirgi belgilariga oson murojaat qilish uchun qulay.

```
matn = "Python"
print(matn[0])    # 'P'
print(matn[-1])   # 'n' (oxirgi belgi)
```

2. Qirqib olish (Slicing)

Satrning ma'lum bir qismini ajratib olish uchun slicing ishlatiladi. Uning umumiy ko'rinishi: `satr[boshlanish : tugash : qadam]`

- **Boshlanish:** Qaysi indeksdan boshlash (shu indeks kiradi).
- **Tugash:** Qaysi indeksgacha (bu indeks kirmaydi).
- **Qadam:** Nechtadan sakrab o'tish (ixtiyoriy, standart holatda 1).

```
s = "Dasturlash"
print(s[0:6])    # 'Dastur' (0 dan 5-indeksigacha)
print(s[6:])     # 'lash' (6-indeksdan oxirigacha)
print(s[::-1])   # 'hsalrutsad' (satrni teskari o'girish)
```

3. Satrlarni birlashtirish (Concatenation)

Ikki yoki undan ortiq satrlarni bir-biriga qo'shish uchun `+` operatoridan foydalaniladi. Shuningdek, satrni songa ko'paytirish (`*`) orqali uni takrorlash mumkin.

```
soz1 = "Salom"
soz2 = "Dunyo"
natija = soz1 + " " + soz2    # "Salom Dunyo"
takror = "A" * 5              # "AAAAA"
```

Eslatma: Satrlar Pythonda **immutable** (o'zgarmas) hisoblanadi. Ya'ni, `matn[0] = "J"` deb mavjud satrning biror belgisini o'zgartira olmaysiz. Buning o'rniga yangi satr yaratishingiz kerak.

Satrlar bilan ishlashda faqat indekslash va kesish bilan cheklanib bo'lmaydi, chunki Python satrlar ustida murakkab amallar bajarish uchun juda boy metodlar to'plamiga ega.

Quyida kundalik dasturlashda eng ko'p ishlatiladigan **20 ta muhim satr metodi** keltirilgan:

1. Registr bilan ishlovchi metodlar

1. **upper():** Barcha harflarni katta qiladi. `"salom".upper() -> "SALOM"`
2. **lower():** Barcha harflarni kichik qiladi. `"OLMA".lower() -> "olma"`
3. **capitalize():** Faqat birinchi harfni katta qiladi. `"python".capitalize() -> "Python"`

4. `title()`: Har bir so'zning birinchi harfini katta qiladi. `"salom dunyo".title()`
-> "Salom Dunyo"
5. `swapcase()`: Registri teskarisiga o'zgartiradi. `"PyThOn".swapcase()` ->
"pYtHoN"

2. Qidiruv va tekshirish metodlari

6. `find(sub)`: Berilgan qismni qidiradi va birinchi uchragan indeksini qaytaradi (topilmasa -1). `"apple".find("p")` -> 1
7. `index(sub)`: `find()` kabi ishlaydi, lekin topilmasa xatolik (`ValueError`) beradi.
8. `count(sub)`: Berilgan qism satrda necha marta takrorlanganini sanaydi.
`"banana".count("a")` -> 3
9. `startswith(prefix)`: Satr berilgan belgi/so'z bilan boshlanishini tekshiradi.
`"Python".startswith("Py")` -> True
10. `endswith(suffix)`: Satr berilgan belgi/so'z bilan tugashini tekshiradi.
`"image.png".endswith(".png")` -> True

3. Tozalash va almashtirish metodlari

11. `strip()`: Satrning boshi va oxiridagi bo'shliqlarni olib tashlaydi. `" alo"`
`".strip()` -> "alo"
12. `replace(old, new)`: Satrdagi biror qismni boshqasiga almashtiradi.
`"salom".replace("s", "j")` -> "jalom"
13. `join(iterable)`: Ro'yxat elementlarini satr yordamida birlashtiradi. `"-"`
`".join(["a", "b"])` -> "a-b"
14. `split(sep)`: Satrni berilgan belgi bo'yicha bo'laklarga (ro'yxatga) ajratadi.
`"a,b,c".split(",")` -> ['a', 'b', 'c']

4. Tekshirish (Is...) metodlari

15. `isalpha()`: Faqat harflardan iboratligini tekshiradi. `"Abc".isalpha()` -> True
16. `isdigit()`: Faqat raqamlardan iboratligini tekshiradi. `"123".isdigit()` -> True
17. `isalnum()`: Harf yoki raqamdan iboratligini tekshiradi (belgilar bo'lmasligi kerak). `"A1".isalnum()` -> True
18. `isspace()`: Faqat bo'shliqlardan iboratligini tekshiradi. `" ".isspace()` -> True

5. Formatlash va joylashtirish

19. `zfill(width)`: Satr boshini nollar bilan to'ldiradi. `"7".zfill(3)` -> "007"
20. `center(width)`: Satrni ma'lum kenglik markaziga joylashtiradi.
`"hi".center(6, "-")` -> "--hi--"

Amaliy misollar jamlanmasi:

```
matn = " Python Dasturlash Tili "
```

```
# Metodlarni zanjir kabi ishlatish (Method Chaining)
toza_matn = matn.strip().lower().replace("tili", "asosi")
print(toza_matn) # Natija: "python dasturlash asosi"

# split va join birgalikda
sozlar = "olma anor behi".split() # Bo'shliq bo'yicha ajratadi
yangi_matn = ", ".join(sozlar)
print(yangi_matn) # Natija: "olma, anor, behi"
```

Foydalanilgan manbalar:

1. **Python.org:** [Built-in String Methods](#)
2. **W3Schools:** [Python String Methods Reference](#)
3. **W3Schools:** [Python Strings](#)
4. **Real Python:** [Strings and Character Data in Python](#)
5. **Python Docs:** [Text Sequence Type — str](#)

10-savol: `str.format()` metodi va f-string yordamida satrlarni formatlash usullarini qiyosiy tahlil qiling.

Pythonda o'zgaruvchilarni matn ichiga joylashtirish (formatlash) uchun bir necha usul mavjud. Eng keng tarqalgan va zamonaviy usullar — bu `.format()` metodi va **f-string** (Formatted String Literals) hisoblanadi.

1. `str.format()` metodi (Python 2.7+ va 3.0+)

Ushbu usulda satr ichida maxsus figurali qavslar `{}` (placeholder) ishlatiladi va qiymatlar `.format()` metodiga argument sifatida uzatiladi.

- **Sintaksis:** `"Matn {} {}".format(qiymat1, qiymat2)`
- **Xususiyatlari:**
 - Tartib bo'yicha joylashtirish: `"{} {}".format("Ali", 25)`
 - Indeks bo'yicha joylashtirish: `"{1} {0}".format(25, "Ali")`
 - Nomi bo'yicha joylashtirish: `"{ism} {yosh}".format(ism="Ali", yosh=25)`

2. f-string (Python 3.6+)

Bu eng zamonaviy, tezkor va o'qilishi oson usul hisoblanadi. Satrning boshiga `f` yoki `F` prefiksi qo'yiladi va o'zgaruvchilar to'g'ridan-to'g'ri `{}` ichiga yoziladi.

- **Sintaksis:** `f"Matn {ozgaruvchi}"`
- **Xususiyatlari:**
 - **Tezlik:** f-string `.format()` va `%` operatoridan ancha tezroq ishlaydi, chunki u runtime vaqtida bajariladi.
 - **Ifodalar:** Qavs ichida matematik amallar yoki funksiyalarni bajarish mumkin: `f"Natija: {2 + 3}"`

- **O‘qilishi:** Kod juda qisqa va tushunarli bo‘ladi.

3. Qiyosiy tahlil jadvali

Xususiyat	str.format()	f-string (f'')
Python versiyasi	2.7 va undan yuqori	3.6 va undan yuqori
Tezlik	O'rtacha	Juda yuqori
O'qilishi	Uzun satrlarda biroz qiyin	Juda oson va qulay
Murakkablik	Argumentlar soni ko'paysa chalkashish mumkin	Kod matn bilan birga jipslashgan
Funksiyalar	Cheklangan	Qavs ichida metod/funksiya chaqirish mumkin

Amaliy Misol:

```

ism = "Anvar"
yosh = 20

# .format() usuli
matn1 = "Salom, mening ismim {}. Yoshim {}da.".format(ism, yosh)

# f-string usuli
matn2 = f"Salom, mening ismim {ism}. Yoshim {yosh}da. Kelasi yil {yosh + 1} bo'laman."

print(matn1)
print(matn2)

```

f-string va `.format()` metodlari nafaqat o‘zgaruvchini joylashtirish, balki matnni tekislash, sonlarni formatlash va sanalar bilan ishlash kabi murakkab vazifalarni ham bajaradi.

Quyida ushbu usullarning **10 ta eng muhim vazifasi** va ularning qiyosiy sintaksisi keltirilgan:

1. O‘zgaruvchini joylashtirish (Basic Interpolation)

Eng oddiy vazifa — o‘zgaruvchi qiymatini matnga qo‘shish.

- **f-string:** `f"Salom {ism}"`
- **format:** `"Salom {}".format(ism)`

2. Sonlarni yaxlitlash (Rounding Floats)

Suzuvchi nuqtali sonlarni ma'lum bir xonagacha aniqlikda ko‘rsatish.

- **f-string:** `f"{3.14159:.2f}"`  3.14
- **format:** `"{: .2f}".format(3.14159)`

3. Minglik ajratgichlar (Thousands Separator)

Katta sonlarni o‘qish oson bo‘lishi uchun vergul yoki bo‘shliq bilan ajratish.

- **f-string:** `f"{1000000:,}"`  `1,000,000` (, o‘rniga `_` qo‘yish mumkin)
- **format:** `"{:,}".format(1000000)`

4. Matnni chapga, o‘ngga va markazga tekislash (Alignment)

Ma'lum bir kenglik ichida matn o‘rnini belgilash.

- **Markazga:** `f"{'Assalom':^20}"` (20 ta belgi ichida markazda)
- **O‘ngga:** `f"{'Salom':>10}"`
- **Chapga:** `f"{'Salom':<10}"`
- **format:** `"{:^20}".format("Assalom")`

5. Bo‘shliqlarni belgilar bilan to‘ldirish (Padding)

Tekislash vaqtida bo‘sh qolgan joylarni maxsus belgi bilan to‘ldirish.

- **f-string:** `f"{'Men':*^10}"`  `***Men***`
- **format:** `"{:*^10}".format("Men")`

6. Foiz ko‘rinishida chiqarish (Percentage)

Sonni 100 ga ko‘paytirib, oxiriga % belgisini qo‘shish.

- **f-string:** `f"{0.85:.1%}"`  `85.0%`
- **format:** `"{: .1%}".format(0.85)`

7. Ikkilik, sakkizlik va o‘n oltilik sanoq tizimlari

Sonlarni boshqa sanoq tizimlarida ko‘rsatish.

- **Hex:** `f"{255:x}"`  `ff`
- **Binary:** `f"{10:b}"`  `1010`
- **format:** `"{:b}".format(10)`

8. Sana va vaqtni formatlash (Datetime Formatting)

`datetime` obyekti qismlarini bevosita satr ichida formatlash.

- **f-string:** `f"{hozir:%Y-%m-%d %H:%M}"`
- **format:** `"{: %d %B %Y}".format(hozir)`

```
import datetime
```

```
# Hozirgi vaqtni olish
hozir = datetime.datetime.now()

# f-string yordamida formatlash
formatlangan_vaqt = f"{hozir:%Y-%m-%d %H:%M}"

print(formatlangan_vaqt)
# Natija (taxminan): 2025-12-24 19:45
```

9. Lugʻat (Dictionary) elementlariga murojaat

- **f-string:** `f"{user['ism']}"`
- **format:** `"{ism}".format(**user)` (yoki `"{0[ism]}".format(user)`)

10. Qavslarni oʻzini chop etish (Escaping Braces)

Agar f-string ichida figurali qavs {} ning oʻzi kerak boʻlsa, ularni ikkilantirish lozim.

- **f-string:** `f"Qiymat {{ {x} }} ichida"` ➡ `Qiymat { 10 } ichida`
- **format:** `"Qiymat {{ {} }}".format(x)`

Amaliy kod namunasi (Hamma usullar jamlangan):

```
import datetime

ism = "Ali"
hisob = 1500000.789
foiz = 0.756
bugun = datetime.datetime.now()

# 1. Padding va Alignment (10 ta katak ichida chapga tekislash, qolganini
# bilan to'ldirish)
print(f"Mijoz: {ism:*<10}")
# NATIJA: Mijoz: Ali*****

# 2. Minglik ajratgich va yaxlitlash (vergul bilan ajratish va nuqtadan
# keyin 2 xona)
print(f"Balans: {hisob:,.2f} so'm")
# NATIJA: Balans: 1,500,000.79 so'm

# 3. Foiz (0.756 ni 100 ga ko'paytirib, % belgisini qo'yish va 1 xona
# aniqlik)
print(f"Yutuq ehtimoli: {foiz:.1%}")
# NATIJA: Yutuq ehtimoli: 75.6%

# 4. Vaqt formatlash (Kun/Oy/Yil ko'rinishida)
print(f"Sana: {bugun:%d/%m/%Y}")
# NATIJA: Sana: 24/12/2025

# 5. .format() metodi yordamida (f-string bilan bir xil natija beradi)
print(f"Balans: {hisob:,.2f} so'm".format(hisob))
# NATIJA: Balans: 1,500,000.79 so'm
```

Natijalarning qisqacha tahlili:

1. `Ali*****`: Bu yerda `*<10` buyrug'i shuni anglatadiki: matn jami **10 ta** belgidan iborat joyni egallasin, `Ali` chap tomonda tursin (`<`), bo'sh qolgan 7 ta joy esa `*` bilan to'ldirilsin.
2. `1,500,000.79`: E'tibor bering, asl son `789` bilan tugagan edi, lekin `.2f` (float 2) buyrug'i matematik qoidaga ko'ra sonni **yaxlitlab** (78 dan 79 ga) ko'rsatdi.
3. `75.6%`: Python `0.756` ni avtomatik ravishda foiz ko'rinishiga o'tkazdi.
4. `24/12/2025`: `%d` (kun), `%m` (oy), `%Y` (yil) formatlari sanani biz o'rgangan ko'rinishga keltirdi.

Xulosa: Agar siz Python 3.6+ versiyasida ishlayotgan bo'lsangiz, har doim **f-string** usulidan foydalanish tavsiya etiladi. `.format()` usuli esa asosan eski kodlarda yoki lug'atlar (dict) bilan ishlashda foydali bo'lishi mumkin.

Foydalanilgan manbalar:

1. **Real Python:** [Python 3's f-Strings: An Improved String Formatting Syntax](#)
2. **W3Schools:** [Python String Formatting](#)
3. **Python Docs:** [Formatted string literals](#)
4. **Python Docs:** [Format Specification Mini-Language](#)
5. **PyFormat:** [Using f-strings and .format\(\) comparison](#)

11-savol: Satrlarni kodlash (encode) va dekodlash (decode) metodlarining vazifasi va ular nima uchun baytlar bilan ishlashda kerak?

Kompyuterlar matnli belgilarni (harf, belgi, emoji) tushunmaydi, ular faqat **0 va 1** (bitlar) bilan ishlaydi. Shuning uchun matnni xotirada saqlash yoki tarmoq orqali uzatishdan oldin uni ma'lum bir qoidalar asosida sonli ko'rinishga keltirish kerak.

1. Kodlash (Encoding) nima?

`encode()` — bu inson tushunadigan matnni (string) kompyuter tushunadigan **baytlar (bytes)** ketma-ketligiga o'tkazish jarayonidir.

- **Sintaksis:** `satr.encode(encoding='utf-8')`
- Standart kodlash turi: **UTF-8** (dunyo tillaridagi deyarli barcha belgilarni o'z ichiga oladi).

2. Dekodlash (Decoding) nima?

`decode()` — bu baytlar to'plamini qaytadan inson tushunadigan matn (string) ko'rinishiga qaytarish jarayonidir.

- **Sintaksis:** `baytlar.decode(encoding='utf-8')`

3. Amaliy Misol

```
# 1. Matn yaratamiz
matn = "Python - zo'r! 🐍" # Unicode matn (emoji bilan)

# 2. Kodlash (String -> Bytes)
baytlar = matn.encode('utf-8')
print(f"Baytlar: {baytlar}")
# Natijada: b'Python - zo\r! \xf0\x9f\x90\x8d' (b prefiksi bayt ekanini bildiradi)

# 3. Dekodlash (Bytes -> String)
qayta_matn = baytlar.decode('utf-8')
print(f"Qayta matn: {qayta_matn}")
```

4. Nima uchun baytlar bilan ishlash kerak?

Baytlar darajasida ishlash quyidagi holatlarda zarur:

- **Fayllar bilan ishlash:** Rasmlar, videolar yoki audio fayllar matn emas, baytlar ko'rinishida saqlanadi.
- **Tarmoq (Networking):** Ma'lumotlar internet orqali uzatilganda (masalan, API so'rovlari) ular doimo bayt ko'rinishida yuboriladi.
- **Ma'lumotlar bazasi:** Ba'zi bazalar ma'lumotlarni siqilgan yoki kodlangan bayt formatida saqlashni talab qiladi.

Muhim eslatma: ASCII vs UTF-8

- **ASCII:** Faqat ingliz alifbosi va asosiy belgilarni (128 ta) taniy oladi. O'zbekcha "o'", "g'" yoki emojilarni ASCII orqali kodlab bo'lmaydi (xatolik beradi).
- **UTF-8:** Zamonaviy standart bo'lib, u barcha tillarni va simvollarni qo'llab-quvvatlaydi. Pythonda standart kodlash turi UTF-8 hisoblanadi.

Foydalanilgan manbalar:

- **Python Rasmiy Hujjatlari:** [Unicode HOWTO](#) — Python-da satrlar va baytlar qanday ishlashi haqida eng to'liq qo'llanma.
- **Real Python:** [Strings vs. Bytes: What's the Difference?](#) — Matn va baytlar o'rtasidagi farqni tushuntiruvchi ajoyib maqola.
- **W3Schools:** [Python String encode\(\) Method](#) — `encode()` metodining sodda tushuntirishi va misollari.
- **GeeksforGeeks:** [Python String decode\(\) Method](#) — Baytlarni qayta matnga o'tkazish bo'yicha qo'llanma.

12-savol: Shart operatorlari (if, elif, else) ning ishlash mantig'i va sintaksisini tushuntiring.

Dasturlashda **shart operatorlari** dasturning ma'lum bir shart bajarilishi yoki bajarilmasligiga qarab turli yo'nalishlarda davom etishini ta'minlaydi. Buni "yo'l ayrig'i" deb tasavvur qilish mumkin.

1. Ishlash mantig'i

Shart operatorlari har doim mantiqiy ifodaning natijasiga tayanadi (`True` yoki `False`).

- Agar shart `True` (rost) bo'lsa, o'sha blokda kod bajariladi.
- Agar shart `False` (yolg'on) bo'lsa, Python u blokni tashlab o'tadi.

2. Sintaksis va tarkibiy qismlar

Pythonda shart operatorlari quyidagi kalit so'zlar orqali yoziladi:

- **if (agar):** Tekshiriladigan birinchi va asosiy shart.
- **elif (aks holda, agar):** Birinchi shart xato bo'lsa, keyingi qo'shimcha shartni tekshiradi. (Istalgacha miqdorda bo'lishi mumkin).
- **else (aks holda):** Yuqoridagi barcha shartlar xato (`False`) bo'lganda ishlaydigan yakuniy blok.

Muhim qoida: Har bir shartdan keyin **ikki nuqta (:)** qo'yiladi va shartga tegishli kodlar **indentatsiya** (o'ngga surilgan bo'sh joy) bilan yoziladi.

3. Amaliy Misol

Python

```
ball = 85

if ball >= 90:
    print("Sizning bahongiz: A")
elif ball >= 80:
    print("Sizning bahongiz: B")
elif ball >= 70:
    print("Sizning bahongiz: C")
else:
    print("Siz imtihondan o'ta olmadingiz.")
```

Natija: Sizning bahongiz: B (Chunki 85 soni 90 dan katta emas, lekin 80 dan katta).

4. Shart operatorlarining turlari

1. **Oddiy if:** Faqat bitta shartni tekshiradi.

2. **if-else:** Shart bajarilsa bir ishni, bajarilmasa boshqa ishni qiladi.
3. **if-elif-else zanjiri:** Ko'p variantli tanlovlar uchun.
4. **Bir qatorli (Ternary) if:** Qisqa shartlarni yozish uchun.
 - o *Misol:* `natija = "O'tdi" if ball >= 60 else "Yiqildi"`

Foydalanilgan manbalar:

1. **W3Schools:** [Python If...Else](#)
2. **Real Python:** [Conditional Statements in Python](#)
3. **Python Docs:** [More Control Flow Tools](#)

13-savol: Pythonda match-case operatori va uning an'anaviy if-elif-else tizimidan farqi.

Python 3.10 da kiritilgan bu operator shunchaki "switch-case" emas, balki ma'lumotlar strukturasini tekshirishning juda kuchli usulidir.

1. if-elif-else bilan solishtirish (Sintaksis)

An'anaviy usul (if):

```
command = "login admin"
parts = command.split()

if parts[0] == "login" and len(parts) == 2:
    user = parts[1]
    print(f"Xush kelibsiz, {user}")
elif parts[0] == "logout":
    print("Xayr!")
```

Zamonaviy usul (match-case):

```
command = "login admin"

match command.split():
    case ["login", user]: # Andoza (pattern): 2 ta elementli list bo'lsa
        print(f"Xush kelibsiz, {user}")
    case ["logout"]:
        print("Xayr!")
    case _: # Wildcard (else o'rnida)
        print("Noma'lum buyruq")
```

2. match-case ning asosiy afzalliklari:

1. **Destructuring (Strukturani buzish):** `match` bir vaqtning o'zida ham ma'lumotning turini (masalan, list ekanligini), ham uning tarkibini tekshirib, ichidagi qiymatlarni o'zgaruvchilarga biriktirib yuboradi.
2. **Guards (Himoyalovchi shartlar):** `case` blokiga qo'shimcha `if` shartini qo'shish mumkin:

```
case ["age", x] if int(x) >= 18:
    print("Sizga ruxsat berildi")
```

3. Bir nechta qiymatlarni bitta `case`da tekshirish: | (OR) operatori orqali:

```
case 400 | 401 | 404:
    print("Mijoz xatoligi")
```

4. Wildcard (`_`): Har qanday qiymatga mos keladi. Bu `if-else` dagi oxirgi `else` bilan bir xil vazifani bajaradi.

3. Asosiy farqlar jadvali

Xususiyat	if-elif-else	match-case
Tekshirish turi	Faqat mantiqiy (True/False)	Ma'lumot andozasi (shakli) bo'yicha
O'zgaruvchilarni ajratish	Qo'lda (<code>parts[1]</code>)	Avtomatik (<code>case [x, y]</code>)
O'qilishi	Murakkab shartlarda kod chigallashadi	Kod juda toza va sxematik bo'ladi
Versiya	Barcha versiyalarda bor	Faqat Python 3.10 va undan yuqorida

Foydalanilgan manbalar:

1. **Python.org:** [PEP 636 – Structural Pattern Matching: Tutorial](#) — Rasmiy darslik.
2. **Real Python:** [Python 3.10: Structural Pattern Matching](#) — Batafsil qo'llanma.
3. **GeeksforGeeks:** [Python match-case Statement](#) — Amaliy misollar bilan.

14-savol: Mantiqiy operatorlar (`and`, `or`, `not`) ning shart operatorlari ichida qo'llanilishi va natijani aniqlash tartibi qanday?

Mantiqiy operatorlar bir nechta shartlarni birlashtirish yoki mavjud shart natijasini teskarisiga o'zgartirish uchun ishlatiladi. Ular dasturning qaror qabul qilish jarayonini yanada moslashuvchan qiladi.

1. Mantiqiy operatorlar turlari va ishlash mantiqiy

Pythonda uchta asosiy mantiqiy operator mavjud:

- **and (va):** Barcha shartlar `True` bo'lgandagina umumiy natija `True` bo'ladi. Agar bittagina shart `False` bo'lsa ham, natija `False` chiqadi.
- **or (yoki):** Kamida bitta shart `True` bo'lsa, umumiy natija `True` bo'ladi. Faqat barcha shartlar `False` bo'lganda natija `False` chiqadi.
- **not (emas):** Mantiqiy qiymatni teskarisiga o'zgartiradi (`True` ni `False` ga, `False` ni `True` ga).

2. Shart operatorlari ichida qo'llanilishi (Amaliy misol)

Python

```
foydalanuvchi_yoshi = 20
haydovchilik_guvohnomasi = True
mast_holatda = False

# and va not operatorlari birgalikda
if foydalanuvchi_yoshi >= 18 and haydovchilik_guvohnomasi and not mast_holatda:
    print("Siz mashina hayday olasiz.")
else:
    print("Sizga mashina haydash taqiqlanadi.")
```

3. Natijani aniqlash tartibi (Operator Precedence)

Agar bitta shartli ifoda ichida bir nechta turli operatorlar kelsa, Python ularni quyidagi qat'iy tartibda bajaradi:

1. **Qavslar ()**: Har doim birinchi bajariladi.
2. **Taqqoslash operatorlari**: <, >, ==, != va h.k.
3. **not**: Birinchi darajali mantiqiy operator.
4. **and**: Ikkinchi darajali mantiqiy operator.
5. **or**: Uchinchi darajali mantiqiy operator.

4. "Short-Circuit Evaluation" (Qisqa tutashuv)

Python mantiqiy ifodalarni hisoblashda aqlli tizimdan foydalanadi:

- **and** amali bajarilayotganda, agar birinchi shart `False` bo'lsa, qolgan shartlar tekshirilmaydi (chunki natija baribir `False` chiqishi aniq).
- **or** amali bajarilayotganda, agar birinchi shart `True` bo'lsa, qolganlari tekshirilmaydi (chunki natija baribir `True` chiqishi aniq).

Foydalanilgan manbalar:

1. **W3Schools**: [Python Logical Operators](#)
2. **Real Python**: [Operators and Expressions in Python](#)
3. **GeeksforGeeks**: [Python Logical Operators with Examples](#)

15-savol: Ichma-ich (nested) joylashgan shart operatorlarining afzalliklari va kamchiliklari nimalardan iborat?

Ichma-ich (nested) shart operatorlari — bu bir `if`, `elif` yoki `else` bloking ichida boshqa bir shart operatorining kelishidir. Bu usul iyerarxik yoki bir-biriga bog'liq bo'lgan murakkab shartlarni tekshirish uchun ishlatiladi.

1. Sintaksis va ishlatilishi

Ichma-ich shartlar odatda asosiy shart bajarilgandan keyingina qo'shimcha "kichik" shartlarni tekshirish kerak bo'lganda qo'llaniladi.

```
yosh = 20
hujjat = True

if yosh >= 18:
    print("Siz voyaga yetgansiz.")
    if hujjat:
        print("Kirishingiz mumkin.")
    else:
        print("Hujjatingiz yo'q, kirish taqiqlanadi.")
else:
    print("Siz hali yoshsiz.")
```

2. Afzalliklari

- **Iyerarxik mantiq:** Ma'lumotlarni bosqichma-bosqich saralash imkonini beradi. Masalan, avval foydalanuvchi tizimda bormi, keyin esa uning paroli to'g'rimi deb tekshirish.
- **Aniq ajratish:** Har bir shart o'zidan oldingi shartga bog'liq bo'lgani uchun, mantiqiy xatolarni kamaytirishga yordam beradi.
- **Resurslarni tejash:** Agar asosiy (tashqi) shart `False` bo'lsa, Python ichki shartlarni umuman o'qib o'tirmaydi, bu esa vaqtni tejaydi.

3. Kamchiliklari

- **Kodning murakkablashishi (Pyramid of Doom):** Agar ichma-ichlik darajasi 3-4 tadan oshib ketsa, kodni o'qish va tushunish juda qiyin bo'lib ketadi.
- **Indentatsiya (bo'sh joy) muammosi:** Pythonda indentatsiya muhim bo'lgani uchun, kod juda o'ngga surilib ketadi, bu esa xato yozish ehtimolini oshiradi.
- **Qiyin testlash:** Har bir ichki blok uchun alohida test holatlarini yaratish qiyinlashadi.

4. Alternativa (Yaxshiroq yechimlar)

Ko'p hollarda ichma-ich `if`lardan qochish uchun quyidagilar tavsiya etiladi:

1. **Mantiqiy operatorlar:** `if shart1 and shart2:` ko'rinishida yozish.
2. **Guard Clauses:** Shart bajarilmasa, funksiyani darhol `return` bilan to'xtatish.
3. **match-case:** Yuqorida ko'rganimizdek, murakkab tuzilmalar uchun qulayroq.

Foydalanilgan manbalar:

1. **W3Schools:** [Python Nested If](#)
2. **GeeksforGeeks:** [Nested IF Statement in Python](#)
3. **Programiz:** [Python if...else Statement \(Nested if\)](#)

16-savol: Takrorlanuvchi jarayonlarni tashkil etishda `for` va `while` sikl operatorlarining asosiy farqlari nimada? `range()` va `enumerate()` funksiyalarining sikllar bilan birga qo'llanilishi va afzalliklarini tushuntiring.

Dasturlashda bir xil amalni bir necha marta bajarish kerak bo'lganda **sikllar** (loops) ishlatiladi. Pythonda ikkita asosiy sikl operatori mavjud.

1. `for` va `while` sikllari o'rtasidagi farq

Xususiyat	<code>for</code> sikli	<code>while</code> sikli
Maqsadi	To'plam elementlari (list, str, range) bo'ylab aylanish.	Ma'lum bir shart True bo'lib turguncha aylanish.
Takrorlanish soni	Oldindan ma'lum bo'ladi.	Oldindan noma'lum bo'lishi mumkin (shartga bog'liq).
Sintaksis	for element in to'plam:	while shart:
Xavfsizlik	Cheksiz sikl xavfi deyarli yo'q.	Shartni noto'g'ri yozsa, cheksiz sikl hosil bo'ladi.

2. `range()` funksiyasi

`range()` funksiyasi sonlar ketma-ketligini hosil qilish uchun ishlatiladi. Bu ayniqsa `for` siklida ma'lum marta takrorlanishni amalga oshirishda qulay.

- **Sintaksis:** `range(start, stop, step)`
- **Afzalligi:** Xotirani tejaydi, chunki u barcha sonlarni birdaniga yaratmaydi, balki sikl davomida bittadan taqdim etadi.

```
for i in range(1, 10, 2):  
    print(i) # 1, 3, 5, 7, 9 sonlarini chiqaradi
```

3. `enumerate()` funksiyasi

`enumerate()` — bu ro'yxat (yoki boshqa ketma-ketlik) elementlari bilan ishlaganda, bir vaqtning o'zida ham **elementning indeksini**, ham **qiymatini** olish imkonini beradi.

- **Afzalligi:** Qo'lda sanoq o'zgaruvchisi (masalan, `i = 0`) yaratish va uni har safar oshirib borish (`i += 1`) majburiyatidan xalos qiladi. Kod ancha toza va "Pythonic" ko'rinadi.

```
mevalar = ["olma", "banan", "olcha"]  
for indeks, qiymat in enumerate(mevalar, start=1):  
    print(f"{indeks}-chi meva: {qiymat}")
```

```
# Natija:  
# 1-chi meva: olma  
# 2-chi meva: banan  
# 3-chi meva: olcha
```

4. Qachon qaysi birini tanlash kerak?

- Agar sizda tayyor ro'yxat bo'lsa yoki necha marta aylanishni bilsangiz — `for` ishlatilg.
- Agar sikl foydalanuvchi ma'lum bir tugmani bosguncha yoki biror holat o'zgarguncha davom etishi kerak bo'lsa — `while` ishlatilg.

Foydalanilgan manbalar:

1. **W3Schools:** [Python For Loops](#) / [While Loops](#)
2. **Real Python:** [Python's enumerate\(\): Simplify Looping With Counters](#)
3. **GeeksforGeeks:** [Python range\(\) function](#)

17-savol: Sikl jarayonini boshqarishda `break`, `continue` va `pass` operatorlarining vazifalari nimalardan iborat?

Sikllar ichida jarayonni to'xtatish, ma'lum bir qismni tashlab o'tish yoki vaqtinchalik joy qoldirish uchun ushbu uchta maxsus operator ishlatiladi.

1. `break` operatori

`break` operatori siklni muddatidan oldin **butunlay to'xtatish** uchun ishlatiladi. Dastur sikldan chiqadi va undan keyingi qatorga o'tadi.

- **Qachon kerak?** Masalan, qidirilayotgan ma'lumot topilganda siklni davom ettirishdan ma'no qolmaganda.

```
for son in range(1, 10):
    if son == 5:
        break # 5 ga yetganda sikl butunlay to'xtaydi
    print(son)
# Natija: 1, 2, 3, 4
```

2. `continue` operatori

`continue` operatori siklning joriy iteratsiyasini (takrorlanishini) to'xtatadi va darhol **keyingi takrorlanishga** o'tadi.

- **Qachon kerak?** Ma'lum bir shart bajarilganda o'sha elementni "sakrab o'tib" (tashlab ketib), qolganlari bilan ishlashni davom ettirishda.

```
for son in range(1, 6):
    if son == 3:
        continue # 3 ni chop etmaydi, darhol 4 ga o'tadi
    print(son)
# Natija: 1, 2, 4, 5
```

3. `pass` operatori

`pass` operatori — bu **bo'sh operatsiyadir**. U hech narsa qilmaydi.

- **Qachon kerak?** Python sintaksisiga ko'ra shart yoki sikl bloki bo'sh bo'lishi mumkin emas. Agar siz kodning biror qismini hali yozmagan bo'lsangiz, lekin dastur xato bermasdan ishlashini xohlasangiz, `pass` dan foydalanasiz (to'ldiruvchi vazifasini o'taydi).

```
for son in range(5):  
    if son == 2:  
        pass # Bu yerda keyinchalik biror kod yoziladi  
    print(son)
```

Farqlarni eslab qolish uchun:

- **`break`:** "Sikldan qochish" (exit).
- **`continue`:** "Keyingisiga o'tish" (skip).
- **`pass`:** "Shunchaki turish" (placeholder).

Foydalanilgan manbalar:

1. W3Schools: [Python Break/Continue](#)
2. GeeksforGeeks: [Python break, continue and pass statements](#)
3. Real Python: [Python "while" Loops \(Controlling Loops\)](#)

18-savol: Ro'yxatlar (List) nima va ular massivlardan qanday farq qiladi? Ro'yxat yaratishning turli usullarini sanab o'ting.

Ro'yxat (List) — bu Python dasturlash tilidagi eng moslashuvchan va ko'p ishlatiladigan ma'lumot turlaridan biri bo'lib, u bir nechta elementlarni bitta tartiblangan ketma-ketlikda saqlash imkonini beradi.

1. Ro'yxatlarning asosiy xususiyatlari

- **Tartiblangan (Ordered):** Har bir element o'z indeksiga (0 dan boshlanadi) ega va ular kiritilgan tartibda saqlanadi.
- **O'zgaruvchan (Mutable):** Ro'yxat yaratilgandan keyin uning elementlarini o'zgartirish, o'chirish yoki yangi element qo'shish mumkin.
- **Turli xillik (Heterogeneous):** Bir vaqtning o'zida har xil turdagi ma'lumotlarni (masalan, `int`, `str`, `float` va hatto boshqa `list`) birga saqlay oladi.

2. Ro'yxat (List) va Massiv (Array) o'rtasidagi farqlar

Ko'p dasturchilar bu ikkisini bir xil deb o'ylashadi, lekin Python-da ular o'rtasida jiddiy farqlar mavjud:

Xususiyat	Ro'yxat (List)	Massiv (Array)
Elementlar turi	Har xil turdagi elementlarni saqlay oladi.	Faqat bir xil turdagi (masalan, faqat son) elementlarni saqlaydi.
Hajmi	Dinamik — xohlagancha kengayishi mumkin.	Ko'pincha statik yoki hajmi chegaralangan bo'ladi.
Xotira (Memory)	Ko'proq joy egallaydi (har bir element ob'ekt sifatida saqlanadi).	Juda kam joy egallaydi va tezroq ishlaydi.
Amallar	Umumiy maqsadlar uchun boy metodlarga ega.	Matematik va ilmiy hisob-kitoblar uchun (masalan, NumPy kutubxonasi).

3. Ro'yxat yaratishning turli usullari

Python-da ro'yxat yaratishning bir nechta yo'li bor:

a) Kvadrat qavslar yordamida (Eng ko'p ishlatiladigan usul):

```
mevalar = ["olma", "banan", "anor"] # Matnli ro'yxat
sonlar = [1, 2, 3, 4.5]             # Sonli ro'yxat
aralash = ["Ali", 25, True]         # Aralash ro'yxat
bosh_list = []                      # Bo'sh ro'yxat
```

b) list() konstruktori yordamida:

Boshqa iterativ ob'ektlarni (string, tuple, set) ro'yxatga aylantirish uchun ishlatiladi.

```
belgilar = list("Python") # Natija: ['P', 'y', 't', 'h', 'o', 'n']
```

c) Ro'yxat generatori (List Comprehension) orqali:

Bir qator kod yordamida mantiqiy asosda ro'yxat yaratish.

```
kvadratlar = [x**2 for x in range(5)] # Natija: [0, 1, 4, 9, 16]
```

d) Ko'paytirish operatori yordamida:

Bir xil elementdan iborat ro'yxatni tezkor yaratish.

```
nollar = [0] * 10 # Natija: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Foydalanilgan manbalar:

1. **Python.org:** [Data Structures - Lists](#)
2. **GeeksforGeeks:** [Python List vs Array](#)
3. **Real Python:** [Python Lists and Tuples](#)

19-savol: Ro'yxatlar bilan ishlovchi asosiy metodlarning vazifalarini tushuntirib bering.

Python ro'yxatlari (List) o'zgaruvchan (**mutable**) bo'lgani uchun, ularning elementlarini tahrirlash, qo'shish va o'chirish uchun juda boy metodlar to'plamiga ega. Quyida eng muhim metodlarni vazifasi bo'yicha guruhlab ko'rib chiqamiz.

1. Element qo'shish metodlari

- **append(qiymat)**: Ro'yxatning **oxiriga** bitta element qo'shadi.
- **insert(indeks, qiymat)**: Elementni **belgilangan indeksga** joylashtiradi. O'sha o'rindagi va undan keyingi elementlar o'ngga suriladi.
- **extend(to'plam)**: Ro'yxat oxiriga boshqa bir to'plamni (list, tuple) ulab yuboradi.

```
mevalar = ["olma", "banan"]
mevalar.append("uzum")          # ["olma", "banan", "uzum"]
mevalar.insert(1, "anor")       # ["olma", "anor", "banan", "uzum"]
mevalar.extend(["behi", "kivi"]) # ["olma", "anor", "banan", "uzum",
                                   "behi", "kivi"]
```

2. Elementlarni o'chirish metodlari

- **remove(qiymat)**: Ro'yxatdagi ko'rsatilgan qiymatga teng bo'lgan **birinchi** elementni o'chiradi. Agar bunday qiymat bo'lmasa, xatolik (**ValueError**) beradi.
- **pop(indeks)**: Berilgan indeksdagi elementni o'chiradi va **o'sha qiymatni qaytaradi**. Agar indeks berilmasa, eng oxirgi elementni o'chiradi.
- **clear()**: Ro'yxatni butunlay bo'shatadi (elementlarni o'chiradi, lekin ro'yxatning o'zi qoladi).

```
sonlar = [10, 20, 30, 20, 40]
sonlar.remove(20) # [10, 30, 20, 40] (faqat birinchi 20 o'chdi)
ochirilgan = sonlar.pop(1) # 30 o'chadi va ochirilgan = 30 bo'ladi
# natija: [10, 20, 40]
```

3. Tartiblash va qidirish metodlari

- **sort(key=None, reverse=False)**: Ro'yxatni asl holatida (**in-place**) tartiblaydi. **reverse=True** qilinsa, teskari tartibda saqlaydi.
- **reverse()**: Ro'yxat tartibini shunchaki teskarisiga aylantiradi (saralamaydi).
- **index(qiymat)**: Berilgan qiymat birinchi marta uchragan **indeksni** qaytaradi.
- **count(qiymat)**: Ro'yxat ichida ushbu qiymat **necha marta** qatnashganini sanaydi.

```
harflar = ["B", "A", "C", "A"]
print(harflar.count("A")) # 2
print(harflar.index("C")) # 2
harflar.sort()            # ["A", "A", "B", "C"]
```

4. Nusxalash va boshqa yordamchi metodlar

- `copy()`: Ro'yxatning "yuzaki" nusxasini (shallow copy) yaratadi. Bu juda muhim, chunki `list2 = list1` deb yozsangiz, bitta ro'yxatga ikki xil nom bergan bo'lasiz xolos.
- `len()` (**funksiya**): Ro'yxatdagi jami elementlar sonini qaytaradi.

Metodlarning qisqacha xulosasi:

Metod	Vazifasi	Asl ro'yxatni o'zgartiradimi?
<code>append()</code>	Oxiriga qo'shish	Ha
<code>insert()</code>	Orasiga qo'shish	Ha
<code>pop()</code>	Indeks bo'yicha o'chirish	Ha
<code>remove()</code>	Qiymat bo'yicha o'chirish	Ha
<code>sort()</code>	Tartiblash	Ha
<code>count()</code>	Sanash	Yo'q

Foydalanilgan manbalar:

1. **Python.org**: [More on Lists](#)
2. **W3Schools**: [Python List Methods](#)
3. **GeeksforGeeks**: [Common List Methods in Python](#)

20-savol: List, Set, Dictionary Comprehension nima? Ularga amaliy misollar keltiring.

Comprehension (shakllantiruvchi ifodalar) — bu mavjud bo'lgan iterativ obyektlar (list, tuple, range, set) asosida yangi ma'lumotlar to'plamini yaratishning ixcham, bir qatorli va juda tezkor usulidir. Bu usul an'anaviy `for` sikllari o'rniga ishlatiladi va kodni ancha "Pythonic" (toza va tushunarli) holatga keltiradi.

1. List Comprehension (Ro'yxat shakllantiruvchi)

Eng ko'p ishlatiladigan turi bo'lib, natija kvadrat qavslar `[]` ichida qaytariladi.

- **Sintaksis**: `[ifoda for element in to'plam if shart]`
- **An'anaviy usul**:

```
sonlar = [1, 2, 3, 4, 5]
kvadratlar = []
for x in sonlar:
    if x % 2 == 0:
        kvadratlar.append(x**2)
```

- **Comprehension usuli**:

```
kvadratlar = [x**2 for x in sonlar if x % 2 == 0]
# Natija: [4, 16]
```

2. Set Comprehension (To'plam shakllantiruvchi)

Natija figurali qavslar {} ichida qaytariladi. Uning `list`dan asosiy farqi — natijada faqat **unikal (takrorlanmas)** elementlar saqlanadi.

- **Misol:** Matndagi har bir harfni katta qilib, takrorlanmas to‘plam yaratish.

```
matn = "abracadabra"
unikal_harflar = {h.upper() for h in matn}
# Natija: {'A', 'B', 'R', 'C', 'D'}
```

Ushbu kodning unikal (takrorlanmas) qiymatlarni qaytarishi Python-dagi **Set (To‘plam)** ma’lumot turining tabiatiga bog‘liq.

Keling, buni bosqichma-bosqich tahlil qilamiz:

1. Figurali qavslar {} roli

Kodda siz kvadrat qavs [] emas, balki figurali qavs {} ishlatdingiz. Pythonda figurali qavs ichida bajariladigan comprehension (shakllantiruvchi ifoda) **Set Comprehension** deb ataladi.

Set (To‘plam) ma’lumot turining asosiy qoidasi: **Unda bir xil element bir necha marta qatnasha olmaydi.**

2. Jarayon qanday kechadi?

Python "abracadabra" so'zidagi har bir harfni olib, uni katta harfga o'tkazadi va to'plamga (Set) solishni boshlaydi:

1. Birinchi 'a' harfi keladi -> To'plamga 'A' qo'shiladi.
2. Keyin 'b' keladi -> To'plamga 'B' qo'shiladi.
3. Keyin yana 'a' keladi -> Python qaraydi: **"Menda allaqachon 'A' bor!"** va uni tashlab yuboradi.
4. Bu jarayon barcha harflar uchun takrorlanadi.

3. Natijaning tartibi

Siz chop etganingizda natija har safar turlicha tartibda chiqishi mumkin (masalan: {'R', 'D', 'A', 'C', 'B'}). Buning sababi:

- **Set** tartiblanmagan (**unordered**) to'plamdir.
- Unda elementlar indeks bo'yicha emas, maxsus "hash-jadval" bo'yicha saqlanadi.

Unikal qiymatlar **Set** ma'lumot turining **dublikatlarni (takroriy elementlarni) avtomatik ravishda o'chirib yuborish** xususiyati hisobiga hosil bo'lyapti. Agar siz {}

o'rniga [] (List) ishlatganingizda, natijada barcha 11 ta harf (takrorlanganlari bilan birga) chiqib kelar edi.

3. Dictionary Comprehension (Lugʻat shakllantiruvchi)

Kalit va qiymat juftligini yaratish uchun ishlatiladi. Bunda ham figurali qavslar {} ishlatiladi, lekin `kalit: qiymat` koʻrinishida yoziladi.

- **Misol:** Sonlarni kalit, ularning kubini esa qiymat sifatida saqlash.

```
kublar = {x: x**3 for x in range(1, 5)}  
# Natija: {1: 1, 2: 8, 3: 27, 4: 64}
```

4. Afzalliklari va Kamchiliklari

Afzalliklari:

1. **Ixchamlik:** 4-5 qatorli kodni 1 qatorga tushiradi.
2. **Tezlik:** Python ichki mexanizmlari tufayli comprehension `append()` metodiga qaraganda tezroq ishlaydi.
3. **Oʻqishga qulaylik:** Mantiqni bir qarashda tushunish mumkin.

Kamchiliklari:

- **Murakkablik:** Agar shartlar va sikllar haddan tashqari koʻp boʻlib ketsa (masalan, ichma-ich sikllar), kodni tushunish qiyinlashadi. Bunday hollarda oddiy `for` siklidan foydalanish afzal.

Foydalanilgan manbalar:

1. **Real Python:** [When to Use a List Comprehension in Python](#)
2. **W3Schools:** [Python List Comprehension](#)
3. **GeeksforGeeks:** [Python Comprehensions](#)

21-savol: Set (Toʻplam) va Dictionary (Lugʻat) oʻrtasidagi farqlarni sanab bering.

Set va **Dictionary** Python'da ma'lumotlarni saqlash uchun ishlatiladigan eng muhim kolleksiyalardan hisoblanadi. Ularning ikkalasi ham figurali qavslar {} yordamida yaratiladi va ikkalasi ham **tartiblanmagan (unordered)** to'plamlardir. Biroq, ularning vazifasi va tuzilishi butunlay farq qiladi.

1. Asosiy farqlar jadvali

Xususiyat	Set (Toʻplam)	Dictionary (Lugʻat)
-----------	---------------	---------------------

Tarkibi	Faqat alohida qiymatlar (elementlar) saqlanadi.	Kalit: Qiymat (Key: Value) juftliklari saqlanadi.
Elementga murojaat	Indeks yoki kalit yo'q, faqat sikl yoki in operatori orqali.	Kalit orqali qiymatga murojaat qilinadi.
Unikallik	Barcha elementlar unikal (takrorlanmas) bo'lishi shart.	Kalitlar unikal bo'lishi shart, lekin qiymatlar takrorlanishi mumkin.
Maqsadi	Matematik to'plam amallari (birlashma, kesishma) va dublikatlarni tozalash uchun.	Ma'lumotlarni bog'langan holda (xaritaga o'xshash) saqlash uchun.

2. Set (To'plam) xususiyatlari

Set — bu faqatgina **kalitlarsiz** qiymatlardan iborat to'plamdir.

- **Dublikat yo'q:** Bir xil elementni ikki marta qo'shib bo'lmaydi.
- **Matematik amallar:** To'plamlar ustida birlashtirish (`union`), kesishish (`intersection`) kabi amallarni bajarish juda oson.

```
toplam = {1, 2, 3, 3}
print(toplam) # Natija: {1, 2, 3} (3 faqat bir marta chiqadi)
```

3. Dictionary (Lug'at) xususiyatlari

Dictionary — bu har bir elementga nom (kalit) berilgan ma'lumotlar to'plamidir.

- **Kalit (Key):** O'zgarmas (immutable) turda bo'lishi kerak (son, satr, kortej).
- **Qiymat (Value):** Istalgan ma'lumot turi bo'lishi mumkin.

```
lugat = {"ism": "Ali", "yosh": 25}
print(lugat["ism"]) # Natija: Ali
```

```
lugat = {"ism": "Ali", "yosh": 25, 'ism': 'Vali'}
print(lugat["ism"]) # Natija: Vali
```

4. Qachon qaysi biri ishlatiladi?

- Agar sizga faqatgina elementlar ro'yxati kerak bo'lsa va ularning **takrorlanmasligi** muhim bo'lsa — **Set** ishlatiladi.
- Agar sizga elementlarni biror **nom bilan bog'lab** saqlash kerak bo'lsa (masalan, telefon kitobi: "ism" -> "raqam") — **Dictionary** ishlatiladi.

5. Bo'sh to'plam yaratishda ehtiyot bo'ling!

Pythonda bo'sh figurali qavs `{}` doim **bo'sh lug'at (dict)** yaratadi.

- Bo'sh lug'at: `d = {}`
- Bo'sh to'plam: `s = set()`

Foydalanilgan manbalar:

1. W3Schools: [Python Sets](#) / [Python Dictionaries](#)
2. Real Python: [Dictionaries in Python](#)
3. GeeksforGeeks: [Difference between Set and Dictionary](#)

22-savol: Kortej (Tuple) nima, ularning ro'yxatlardan (list) farqi nimada va qaysi holatlarda kortejlardan foydalanish afzalroq?

Kortej (Tuple) — bu bir nechta elementlarni bitta o'zgaruvchida saqlash imkonini beruvchi ma'lumot turi. U tashqi ko'rinishidan ro'yxatga (list) juda o'xshaydi, lekin fundamental bir farqga ega: kortejlar **o'zgarmas (immutable)** hisoblanadi.

1. Kortej va Ro'yxat o'rtasidagi farqlar

Xususiyat	Ro'yxat (List)	Kortej (Tuple)
Qavslar	Kvadrat qavs []	Oddiy qavs ()
O'zgaruvchanlik	Mutable (o'zgartirish mumkin)	Immutable (o'zgartirib bo'lmaydi)
Tezlik	Sekinroq	Tezroq (protsessor uchun qulay)
Xotira	Ko'proq joy egallaydi	Kamroq joy egallaydi
Metodlar	Metodlari juda ko'p (append, pop...)	Faqat 2 ta metod: count va index

2. Kortej yaratish usullari

Kortejlar oddiy qavslar ichida yaratiladi. Agar kortej bitta elementdan iborat bo'lsa, uning oxiriga **vergul** qo'yish shart (aks holda Python uni oddiy qavsga olingan o'zgaruvchi deb o'ylaydi):

```
ranglar = ("qizil", "yashil", "ko'k")
bitta_element = (5,) # Vergul shart!
bosh_tuple = ()
```

Bu juda nozik va muhim masala. Tashqaridan qaraganda vergulsiz ham ishlayotgandek ko'rinsada, aslida Python uni butunlay boshqa narsa deb tushunadi.

Keling, buni amalda tekshirib ko'ramiz:

Farq nimada?

Agar siz vergul qo'ymasangiz, Python qavslarni **matematik ifoda qavslari** deb hisoblaydi (masalan, $(2 + 3) * 5$ dagi kabi) va ichidagi ma'lumot turini o'z holicha qoldiradi.

```
# 1. Vergul bilan (Kortej)
son1 = (5,)
print(type(son1)) # Natija: <class 'tuple'>

# 2. Vergulsiz (Oddiy butun son)
```

```
son2 = (5)
print(type(son2)) # Natija: <class 'int'>
```

Nega bu xavfli?

Agar dasturingiz kortej (tuple) kutayotgan bo'lsa va siz unga oddiy son (int) yoki matn (str) yuborsangiz, kortejga xos metodlarni ishlatganingizda dastur xato berib to'xtaydi.

Misol:

Kortejlarni birlashtirish mumkin, lekin kortejni son bilan qo'shib bo'lmaydi.

```
a = (5) # Bu shunchaki 5 soni
b = (1, 2)
# print(a + b) --> XATOLIK! (int va tuple ni qo'shib bo'lmaydi)

a_true = (5,) # Bu kortej
print(a_true + b) # Natija: (5, 1, 2) --> ISHLADI!
```

Matnlarda yana ham qiziqroq

Agar bitta matnni kortej qilmoqchi bo'lib vergulni unutib qoldirsangiz, Python uni shunchaki oddiy string deb tushunadi:

```
ismlar = ("Ali") # Bu string (str)
ismlar_tuple = ("Ali",) # Bu kortej (tuple)
```

Vergul Pythonga "Bu shunchaki qavs ichidagi qiymat emas, bu kortej strukturasi" deb xabar beruvchi asosiy belgidir. Qavslar kortej uchun majburiy emas (aslida x = 5, ham kortej yaratadi), lekin vergul majburiydir.

3. O'zgarmaslik (Immutability) nimani anglatadi?

Kortej yaratilgandan so'ng uning elementini o'zgartirib bo'lmaydi:

```
mevalar = ("olma", "banan")
# mevalar[0] = "anor" --> Xatolik (TypeError) beradi!
```

4. Qachon kortejdan foydalanish afzalroq?

Dasturlashda kortejlar quyidagi holatlarda ishlatiladi:

1. **Ma'lumotlar xavfsizligi:** Agar siz yaratgan ro'yxat dastur davomida tasodifan o'zgarib ketmasligi kerak bo'lsa (masalan, haftaning kunlari, oylar nomlari, koordinatalar).
2. **Tezkorlik (Performance):** Agar sizda juda katta hajmdagi o'zgarmas ma'lumotlar to'plami bo'lsa, kortej bilan ishlash ro'yxatga qaraganda tezroq kechadi.

3. **Lugʻat (Dictionary) kaliti sifatida:** Roʻyxatlarni lugʻatga kalit qilib boʻlmaydi (chunki ular oʻzgaruvchan), lekin kortejlarni kalit sifatida ishlatish mumkin.
4. **Funksiyadan bir nechta qiymat qaytarish:** Funksiyalar odatda bir nechta natijani bitta kortejga oʻrab qaytaradi.

***Lugʻat (Dictionary) kaliti sifatida:** Roʻyxatlarni lugʻatga kalit qilib boʻlmaydi (chunki ular oʻzgaruvchan), lekin kortejlarni kalit sifatida ishlatish mumkin.*

Bu Python-dagi juda muhim tushuncha — **Hashable** (hashlanuvchi) va **Unhashable** (hashlanmaydigan) ob'ektlar bilan bog'liq. Lugʻat (Dictionary) kaliti bo'lishi uchun ob'ekt o'zgarmas bo'lishi shart.

1. Nega Ro'yxatni (List) kalit qilib bo'lmaydi?

Lugʻat (Dictionary) juda tez ishlashining siri uning **Hash-jadval** (Hash Table) texnologiyasida. Siz lugʻatga kalit berganingizda, Python o'sha kalitni "hashlab" (maxsus raqamli kodga aylantirib), xotiraning aynan qaysi manziliga borishni aniqlaydi.

Agar kalit **o'zgaruvchan (mutable)** bo'lsa (masalan, List), siz uning ichidagi bitta elementni o'zgartirsangiz, uning "hash-kodi" ham o'zgarishi kerak bo'ladi. Bu esa lugʻatning tartibini buzib yuboradi — xuddi uyingizning manzili har kuni o'zgarib turgandek, sizni topish imkonsiz bo'lib qoladi.

```
# XATO HOLAT:  
# meva = ["olma"]  
# lugat = {meva: "bu shirin meva"} # TypeError: unhashable type: 'list'
```

2. Nega Kortejni (Tuple) kalit qilib bo'ladi?

Kortejlar **o'zgarmas (immutable)**. Ularni yaratgandan keyin o'zgartirib bo'lmaydi. Bu degani, ularning hash-kodi doim bir xil qoladi. Python kortejga ishonadi: u hech qachon o'zgarmaydi, demak uni lugʻat kaliti sifatida xotiraga xavfsiz bog'lab qo'yish mumkin.

Amaliy misol:

Tasavvur qiling, sizda xarita koordinatalari bor. Koordinatalar (kenglik va uzunlik) o'zgarmas juftlikdir.

```
# TO'G'RI HOLAT:  
koordinata = (41.2995, 69.2401) # Toshkent koordinatasi  
manzil = {koordinata: "Tashkent City"}  
  
print(manzil[(41.2995, 69.2401)])  
# Natija: Tashkent City
```

3. "O'zgaruvchan Kortej" tuzog'i

Kichik bir istisno bor: Agar kortej ichida o'zgaruvchan element (masalan, list) bo'lsa, uni ham kalit sifatida ishlatib bo'lmaydi.

- (1, 2, 3) — Kalit bo'la oladi ✓
- (1, 2, [3, 4]) — Kalit bo'la olmaydi ✗ (chunki ichidagi list o'zgarishi mumkin).

Xullas:

- **Lug'at kaliti** doimiy, barqaror bo'lishi kerak.
- **List** — beqaror (o'zgaruvchan), shuning uchun kalit bo'lolmaydi.
- **Tuple** — barqaror (o'zgarmas), shuning uchun kalit bo'la oladi.

Foydalanilgan manbalar:

1. W3Schools: [Python Tuples](#)
2. Real Python: [Lists and Tuples in Python](#)
3. GeeksforGeeks: [Tuples in Python](#)

23-savol: Pythonda funksiya tushunchasi, uning tuzilishi va kodni optimallashtirishdagi o'rni.

Funksiya — bu ma'lum bir amalni bajarishga mo'ljallangan va qayta-qayta ishlatilishi mumkin bo'lgan kod blokidir. Dasturlashda "G'ildirakni qayta kashf qilma" tamoyili bor, funksiyalar aynan shu maqsadga — kodni takrorlamaslikka xizmat qiladi.

1. Funksiyaning tuzilishi

Pythonda funksiya `def` (inglizcha *define* — aniqlash) kalit so'zi orqali e'lon qilinadi.

Asosiy tarkibiy qismlari:

1. **`def` kalit so'zi:** Funksiya boshlanayotganini bildiradi.
2. **Funksiya nomi:** Funksiyani chaqirish uchun ishlatiladigan nom (kichik harflarda, so'zlar orasi `_` bilan).
3. **Parametrlar (qavs ichida):** Funksiya ishlashi uchun tashqaridan qabul qilinadigan ma'lumotlar (ixtiyoriy).
4. **Ikki nuqta (:):** Funksiya sarlavhasi tugaganini bildiradi.
5. **Funksiya tanasi:** Indentatsiya (surilish) bilan yozilgan kod qatori.
6. **`return` (qaytarish):** Natijani tashqariga chiqarish uchun (ixtiyoriy).

```
def salom_ber(ism):          # Funksiya e'lon qilinishi
    """Salom beruvchi funksiya""" # Docstring (izoh)
    return f"Salom, {ism}!"    # Natijani qaytarish

# Funksiyani chaqirish
```

```
print(salom_ber("Ali"))
```

2. Kodni optimallashtirishdagi o‘rni

Funksiyalar dasturiy ta'minotni ishlab chiqishda bir qancha strategik afzalliklarni beradi:

- **DRY (Don't Repeat Yourself) tamoyili:** Bir xil kodni bir necha joyda yozish o'rniga, uni funksiyaga o'rab, kerakli joyda chaqirish kifoya. Bu kod hajmini qisqartiradi.
- **Modullik:** Katta va murakkab muammoni kichik, boshqarish oson bo'lgan funksiyalarga bo'lish imkonini beradi.
- **Xatolarni tuzatish osonligi:** Agar hisob-kitobda xato bo'lsa, uni 10 ta joyda emas, faqat bitta funksiya ichida tuzatish yetarli bo'ladi.
- **O‘qishga qulaylik:** Yaxshi nomlangan funksiyalar (masalan, `hisob_kitob_qil()`) kod nima ish qilayotganini tushunishni osonlashtiradi.

Funksiyalardan foydalanishning juda ko‘p afzalliklari bo‘lishiga qaramay, ma’lum vaziyatlarda ularning ham o‘ziga xos **kamchiliklari** yoki **nojo‘ya ta’sirlari** mavjud.

Dasturlashda har bir qulaylikning o‘z "narxi" bor. Funksiyalarni ishlatishdagi asosiy kamchiliklarni quyidagicha guruhlash mumkin:

Ishlash tezligining pasayishi (Overhead)

Har safar funksiya chaqirilganda, kompyuter protsessori va xotirasi qo‘shimcha ish bajaradi:

- Hozirgi bajarilayotgan kod manzili xotirada saqlanadi.
- Funksiya parametrlari uchun yangi xotira ajratiladi.
- Funksiya tugagach, yana eski manzilga qaytiladi.

Agar funksiya juda kichik bo‘lsa va u millionlab marta takrorlanadigan sikl ichida chaqirilsa, bu dastur tezligini sezilarli darajada pasaytirishi mumkin. Bunday holatlarda kodni funksiyaga bo‘lmasdan, to‘g‘ridan-to‘g‘ri yozish (inline code) tezroq ishlaydi.

Xotira iste’moli (Stack Memory)

Funksiyalar "Stack" (stek) deb ataladigan maxsus xotira qismidan foydalanadi. Agar funksiya o‘z-o‘zini juda ko‘p marta chaqirsa (rekursiya) yoki haddan tashqari ko‘p ichma-ich funksiyalar bo‘lsa, **Stack Overflow** (xotira to‘lib qolishi) xatosi yuz berishi mumkin.

Kodning tarqalib ketishi (Fragmentation)

Dasturni juda ko'p mayda funksiyalarga bo'lib tashlash ba'zan teskari natija beradi:

- **O'qish qiyinlashadi:** Dasturchi kod mantig'ini tushunish uchun doimiy ravishda bir funksiyadan ikkinchisiga "sakrashi" kerak bo'ladi.
- **Navigatsiya muammosi:** Katta loyihalarda funksiya qayerda e'lon qilinganini topish vaqt oladi.

Nojo'ya ta'sirlar (Side Effects)

Agar funksiya uning tashqarisidagi global o'zgaruvchilarni o'zgartirsa, bu dasturda kutilmagan xatolarni keltirib chiqaradi. Bunday funksiyalarni testdan o'tkazish va xatosini topish (debugging) juda qiyin bo'ladi. Buni dasturlashda "**Side Effect**" deb atashadi.

Murakkablikning ortishi (Abstraction Overhead)

Ba'zida funksiyalar kodni shunchalik murakkablashtirib yuboradiki, ularni boshqarish uchun qo'shimcha hujjatlashtirish (docstring) talab etiladi. Noto'g'ri nomlangan yoki parametrlari juda ko'p bo'lgan funksiyalar kodni tushunarsiz qilib qo'yadi.

Qachon funksiya ishlatmaslik kerak?

- Agar kod faqat bir marta ishlatilsa va u juda qisqa bo'lsa.
- Agar funksiya chaqiruvi sarflaydigan vaqt funksiya ichidagi kodning bajarilish vaqtidan ko'p bo'lsa (juda yuqori tezlik talab qilinadigan qismlarda).
- Agar funksiya haddan tashqari ko'p vazifani bittada bajarayotgan bo'lsa (buni bir nechta funksiyaga bo'lish yoki soddalashtirish kerak).

3. Funksiyaning ishlash tartibi

Funksiya e'lon qilinganda u shunchaki xotiraga yuklanadi, lekin ishga tushmaydi. U faqat nomi bilan chaqirilgandagina (invokatsiya) ishlaydi.

Foydalanilgan manbalar:

1. W3Schools: [Python Functions](#)
2. Real Python: [Defining Your Own Python Function](#)
3. GeeksforGeeks: [Python Functions](#)

24-savol: Funksiyada parametr va argument tushunchalari, shuningdek, qiymat qaytaruvchi (return) funksiyalarning ishlash prinsipi.

1. Terminologiya: Parametr vs Argument

- **Parametrlar (Parameters):** Funksiya e'lon qilinayotgan (`def`) vaqtda qavs ichida yoziladigan "o'zgaruvchilar". Ular funksiya uchun "kirish eshigi" vazifasini o'taydi.
- **Argumentlar (Arguments):** Funksiya chaqirilayotgan vaqtda o'sha eshikdan kirib keladigan "haqiqiy qiymatlar".

2. Parametrlarning 5 ta asosiy turi

Pythonda parametrlarni funksiyaga quyidagi tartibda uzatish mumkin:

A. Pozitsion parametrlar (*Required Positional*)

Bular eng oddiy parametrlar bo'lib, argumentlar ularga navbati bilan uzatiladi.

```
def info(ism, yosh):  
    print(f"{ism} - {yosh} yoshda")  
  
info("Ali", 25) # "Ali" -> ism, 25 -> yosh
```

B. Standart qiymatli parametrlar (*Default Parameters*)

Agar foydalanuvchi argument bermasa, ishlatiladigan tayyor qiymat.

```
def salom(ism, xabar="Salom"):  
    print(f"{xabar}, {ism}")
```

C. Ixtiyoriy miqdordagi pozitsion parametrlar (**args*)

Funksiya nechta argument kelishini bilmasa, barcha argumentlarni **Tuple** (kortej) sifatida yig'ib oladi.

```
def summa(*sonlar):  
    return sum(sonlar)
```

D. Ixtiyoriy miqdordagi kalit so'zli parametrlar (***kwargs*)

Nomli argumentlarni **Dictionary** (lug'at) sifatida qabul qiladi.

```
def user_data(**malumot):  
    print(malumot) # {'ism': 'Ali', 'yosh': 25}
```

```
user_data(ism="Ali", yosh=25)
```

E. Keyword-Only va Positional-Only parametrlar (*Python 3.8+*)

- / belgisi — undan chapdagi hamma narsa faqat pozitsion bo'lishi shart.
- * belgisi — undan o'ngdagi hamma narsa faqat kalit so'z bilan (`key=value`) berilishi shart.

3. Parametrlarning qat'iy ketma-ketligi

Pythonda funksiya e'lon qilayotganda parametrlarni xohlagan tartibda yozib bo'lmaydi. Tartib quyidagicha bo'lishi shart:

1. **Standard Positional** (Oddiy parametrlar)
2. **Default** (Standart qiymatli)
3. ***args** (Ixtiyoriy pozitsion)
4. **Keyword-Only** (Faqat nom bilan beriladigan)
5. ****kwargs** (Ixtiyoriy lug'at)

Misol:

```
def murakkab_funksiya(a, b, c=10, *args, d, **kwargs):  
    # a, b — pozitsion  
    # c — standart  
    # args — qo'shimcha pozitsionlar  
    # d — keyword-only (nom bilan berish shart)  
    # kwargs — qo'shimcha lug'at  
    pass
```

4. Qiymat qaytarish: `return` ning ishlash mexanizmi

`return` — funksiyaning natijasini tashqariga (uni chaqirgan qismga) olib chiquvchi operator.

`return` ning 3 ta oltin qoidasi:

1. **Funksiyani o'sha zahoti to'xtatadi.** `return` dan keyin yozilgan birorta kod qatori (hatto u funksiya ichida bo'lsa ham) ishlamaydi.
2. **Bitta funksiya, ko'p qiymat.** Python `return a, b, c` ko'rinishida bir vaqtda bir nechta qiymat qaytara oladi. Texnik jihatdan bu **bitta Tuple** qaytarish deganidir.
3. **None qaytarish.** Agar funksiyada `return` yozilmasa, Python avtomatik ravishda `None` qaytaradi.

5. `print()` va `return` farqi:

- `print()` — ma'lumotni shunchaki foydalanuvchi ko'rishi uchun ekranga (konsolga) yuboradi. Undan keyingi hisob-kitobda foydalanib bo'lmaydi.
- `return` — natijani dasturning xotirasiga qaytaradi. Bu natijani o'zgaruvchiga saqlab, keyinroq boshqa funksiyalarda ishlatish mumkin.

Foydalanilgan manbalar:

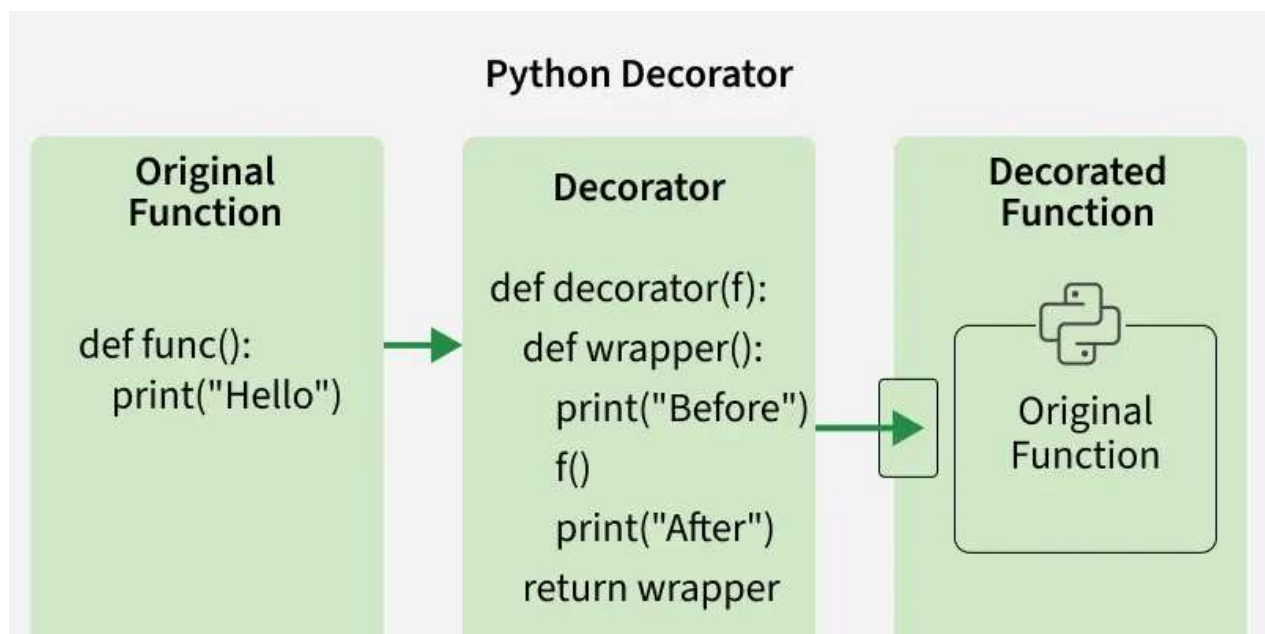
1. **Python Docs:** [Defining Functions](#)
2. **Real Python:** [Python Function Arguments: A Deep Dive](#)
3. **GeeksforGeeks:** [Internal mechanism of function calls in Python](#)

25-savol: Dekoratorlar (@decorator) nima va ulardan foydalanish bo'yicha ma'lumot bering.

Pythonda dekoratorlar — bu funksiya yoki metodlarning asl kodini o'zgartirmagan holda, ularning xatti-harakatini o'zgartirish yoki kengaytirishning moslashuvchan usulidir.

Dekorator, mohiyatan, boshqa bir funksiyaning argument sifatida qabul qiluvchi va kengaytirilgan funktsionallikka ega yangi funksiya qaytaruvchi funksiyadir.

Dekoratorlar ko'pincha log yuritish (logging), autentifikatsiya va memorizatsiya (keshlash) kabi holatlarda qo'llaniladi; ular mavjud funksiya yoki metodlarga qo'shimcha vazifalarni toza va qayta ishlatish mumkin bo'lgan shaklda qo'shish imkonini beradi.



Misol: Keling, oddiy Python dekoratori funksiya chaqirilishidan oldin va keyin qanday qilib qo'shimcha xatti-harakat qo'shishini ko'rib chiqamiz.

```
def decorator(func):  
    def wrapper():  
        print("Funksiya chaqirilishidan oldin.")  
        func()  
        print("Funksiya chaqirilishidan keyin.")  
    return wrapper  
  
@decorator # Dekoratorni funksiyaga qo'llash  
def greet():  
    print("Salom, Dunyo!")  
  
greet()
```

Natija:

```
Funksiya chaqirilishidan oldin.  
Salom, Dunyo!  
Funksiya chaqirilishidan keyin.
```

Tushuntirish:

- `decorator` funksiyasi `greet` funksiyasini argument sifatida qabul qiladi.
- U birinchi bo‘lib xabar chiqaradigan, so‘ngra `greet()` funksiyasini chaqiradigan va oxirida yana bir xabar chiqaradigan yangi funksiyani (`wrapper`) qaytaradi.
- `@decorator` sintaksisi — bu `greet = decorator(greet)` yozuvining qisqartirilgan ko‘rinishidir.

Parametrli dekoratorlar

Dekoratorlar ko‘pincha argumentlarga ega bo‘lgan funksiyalar bilan ishlashiga to‘g‘ri keladi. Bizning `wrapper` (o‘rab turuvchi) funksiyamiz istalgan miqdordagi argumentlarni qabul qila olishi uchun `*args` va `**kwargs` dan foydalanamiz.

Misol: Keling, funksiya bajarilishidan oldin va keyin funksionallik qo‘shadigan dekorator misolini ko‘ramiz:

```
def decorator_name(func):  
    def wrapper(*args, **kwargs):  
        print("Bajarilishdan oldin")  
        result = func(*args, **kwargs)  
        print("Bajarilishdan keyin")  
        return result  
    return wrapper  
  
@decorator_name  
def add(a, b):  
    return a + b  
  
print(add(5, 3))
```

Natija:

```
Bajarilishdan oldin  
Bajarilishdan keyin  
8
```

Parametrlarning tushuntirilishi:

- `decorator_name(func)`: Bu dekorator funksiyasi bo‘lib, u kirish sifatida boshqa bir funksiyani (`func`) qabul qiladi.
- `wrapper(*args, **kwargs)`: Bu `func` funksiyasini o‘rab oluvchi ichki funksiya. `*args` pozitsion argumentlarni, `**kwargs` esa nomlangan (kalit so‘zli) argumentlarni yig‘ib oladi, natijada `wrapper` har qanday funksiya bilan ishlay oladi.
- `@decorator_name`: Bu `add = decorator_name(add)` yozuvining qisqartirilgan shakli (syntax sugar).

Funksiyalar — "Birinchi darajali ob'ektlar" (First-Class Objects) sifatida

Pythonda funksiyalar **birinchi darajali ob'ektlar** hisoblanadi, ya'ni ularga har qanday boshqa ob'ekt (masalan, butun sonlar, satrlar yoki ro'yxatlar) kabi munosabatda bo'lish mumkin. Bu funksiyalarni **o'zgaruvchilarga yuklash**, **argument sifatida uzatish**, **boshqa funksiyalardan natija sifatida qaytarish** va **ma'lumotlar tuzilmalarida saqlash** imkonini beradi. Bu xususiyatlar dekoratorlarni ham o'z ichiga olgan moslashuvchan dasturlash uslublarini shakllantiradi.

Misol: Ushbu kod funksiyalarning birinchi darajali ob'ekt ekanligini ko'rsatuvchi barcha to'rtta xususiyatni namoyish etadi.

```
# 1. Funksiyani o'zgaruvchiga yuklash
def greet(n):
    return f"Salom, {n}!"

say_hi = greet # greet funksiyasini say_hi o'zgaruvchisiga yuklash
print(say_hi("Ali"))

# 2. Funksiyani argument sifatida uzatish
def apply(f, v):
    return f(v)

res = apply(say_hi, "Vali")
print(res)

# 3. Funksiyani boshqa funksiyadan qaytarish
def make_mult(f):
    def mult(x):
        return x * f
    return mult

dbl = make_mult(2)
print(dbl(5))
```

Natija:

```
Salom, Ali!
Salom, Vali!
10
```

Tushuntirish:

- `greet()` funksiyasi `say_hi` o'zgaruvchisiga yuklangan va u orqali "Ali"ga salom yo'llangan.
- `apply()` funksiyasi argument sifatida funksiya va qiymat qabul qiladi, funksiyani qiymatga nisbatan qo'llaydi va natijani qaytaradi.
- `apply` funksiyasining ishlashi `say_hi` va "Vali"ni argument qilib yuborish orqali ko'rsatilgan.

- `make_mult()` funksiyasi berilgan koeffitsient asosida yangi ko'paytiruvchi funksiya yaratadi.

Dekoratorlarda birinchi darajali funksiyalarning roli:

- Dekoratorlar o'zgarish kiritish yoki xatti-harakatni boyitish uchun funksiya kirish argumenti sifatida qabul qiladi.
- Ular asl funksiya o'rab oluvchi va unga ijrodan oldin yoki keyin yangi vazifalar qo'shuvchi yangi funksiya qaytaradi.
- Asl funksiya bir xil nom bilan qayta tayinlanganda, u dekoratsiya qilingan (yangi) funksiya bilan almashadi.

Yuqori tartibli funksiyalar (Higher-Order Functions)

Yuqori tartibli funksiyalar — bu bir yoki bir nechta funksiyalarni argument sifatida qabul qiladigan, natija sifatida funksiya qaytaradigan yoki har ikkala amalni bajaradigan funksiyalardir. Mohiyatan, yuqori tartibli funksiya boshqa funksiyalar ustida amal bajaruvchi funksiyadir. Bu funksional dasturlashdagi kuchli tushuncha bo'lib, dekoratorlar qanday ishlashini tushunishda asosiy komponent hisoblanadi.

Yuqori tartibli funksiyalarning asosiy xususiyatlari

- **Funksiyalarni argument sifatida qabul qilish:** Yuqori tartibli funksiya boshqa funksiyalarni parametr sifatida qabul qila oladi.
- **Funksiyalarni qaytarish:** Yuqori tartibli funksiya keyinchalik chaqirilishi mumkin bo'lgan yangi funksiya natija sifatida qaytara oladi.

Misol: Ushbu kod boshqa funksiya argument sifatida qabul qiladigan va uni berilgan qiymatga qo'llaydigan yuqori tartibli funksiya ko'rsatadi.

```
# Boshqa funksiya argument sifatida qabul qiluvchi yuqori tartibli
funksiya
def fun(f, x):
    return f(x)

# Uzatish uchun oddiy funksiya
def square(x):
    return x * x

res = fun(square, 5) # square funksiyasini qo'llash uchun fun'dan
foydalanish
print(res)
```

Natija:

Tushuntirish: `fun` — yuqori tartibli funksiya, chunki u `f` funksiyasini argument sifatida qabul qiladi va uni `x` qiymatiga qo'llaydi.

Dekoratorlarda yuqori tartibli funksiyalarning roli

Dekoratorlar yuqori tartibli funksiyalar bo'lib, ular funksiyani qabul qiladi, uni modifikatsiya qiladi va kengaytirilgan yoki o'zgartirilgan xatti-harakatga ega yangi funksiyani qaytaradi.

Dekoratorlarning turlari

1. Funksiya dekoratorlari (Function Decorators)

Pythondagi eng keng tarqalgan dekoratorlar turi bo'lib, ular asl funksiya ishga tushishidan oldin yoki keyin qo'shimcha xatti-harakatlar qo'shish orqali funksiyani o'rab olish (wrap) va uni boyitish uchun ishlatiladi.

Misol: Ushbu misolda dekorator o'ralgan funksiya bajarilishidan oldin va keyin xabar chiqaradi.

```
def simple_decorator(func):
    def wrapper():
        print(">>> Funksiya boshlanmoqda")
        func()
        print(">>> Funksiya yakunlandi")
    return wrapper
```

```
@simple_decorator
def greet():
    print("Salom, Dunyo!")
```

```
greet()
```

Natija:

```
>>> Funksiya boshlanmoqda
Salom, Dunyo!
>>> Funksiya yakunlandi
```

Tushuntirish:

- **`simple_decorator(func)`:** Ushbu dekorator `greet` funksiyasini argument (`func`) sifatida qabul qiladi va asl funksiyani chaqirishdan oldin va keyin ba'zi funksionalliklarni qo'shadigan yangi funksiya (`wrapper`) qaytaradi.
- **`@simple_decorator`:** Bu dekorator sintaksisi bo'lib, u `simple_decorator`ni `greet` funksiyasiga qo'llaydi.
- **`greet()` ni chaqirish:** `greet()` chaqirilganda, u shunchaki asl funksiyani bajarmaydi, balki birinchi bo'lib `wrapper` funksiyasi tomonidan qo'shilgan xatti-harakatlarni ishga tushiradi.

2. Metod dekoratorlari (Method Decorators)

Klass ichidagi metodlar uchun ishlatiladigan maxsus dekoratorlar. Ular funksiya dekoratorlariga o'xshab ishlaydi, biroq instansiya metodlari uchun `self` parametrini to'g'ri boshqaradi.

Misol: Bu yerda dekorator metod bajarilishidan oldin va keyin xabar chiqaradi hamda `self` argumentini to'g'ri qabul qiladi.

```
def method_decorator(func):
    def wrapper(self, *args, **kwargs):
        print("Metod bajarilishidan oldin")
        res = func(self, *args, **kwargs)
        print("Metod bajarilishidan keyin")
        return res
    return wrapper

class MyClass:
    @method_decorator
    def say_hello(self):
        print("Salom!")

obj = MyClass()
obj.say_hello()
```

Natija:

```
Metod bajarilishidan oldin
Salom!
Metod bajarilishidan keyin
```

Tushuntirish:

- `method_decorator(func)`: Dekorator metodni (`say_hello`) argument (`func`) sifatida qabul qiladi. U asl metodni chaqirishdan oldin va keyin xatti-harakat qo'shuvchi `wrapper` funksiyasini qaytaradi.
- `wrapper(self, *args, **kwargs)`: `wrapper` albatta `self`ni qabul qilishi shart, chunki bu klass instansiyasining metodidir. `self` — bu klassning ob'ekti, `*args` va `**kwargs` esa kerak bo'lganda boshqa argumentlarni uzatish imkonini beradi.
- `@method_decorator`: Bu `method_decorator`ni `MyClass`ning `say_hello` metodiga qo'llaydi.
- `obj.say_hello()` **ni chaqirish**: Endi `say_hello` metodi qo'shimcha xatti-harakatlar bilan o'rab olingan holda ishga tushadi.

3. Klass dekoratorlari (Class Decorators)

Klass dekoratorlari klassning xatti-harakatini o'zgartirish yoki uni boyitish uchun ishlatiladi. Funksiya dekoratorlari kabi, klass dekoratorlari ham klass ta'rifiga (definition) nisbatan qo'llaniladi. Ular klassni argument sifatida qabul qilish va klassning modifikatsiya qilingan (o'zgartirilgan) versiyasini qaytarish orqali ishlaydi.

Misol: Ushbu kod klassga uning nomini saqlovchi `class_name` atributini qo'shuvchi klass dekoratorini namoyish etadi.

```
def fun(cls):
    cls.class_name = cls.__name__
    return cls

@fun
class Person:
    pass

print(Person.class_name)
```

Natija:

Person

Tushuntirish:

- **fun(cls):** Bu dekorator `cls` klassiga yangi `class_name` atributini qo'shadi. `class_name` qiymati klassning asl nomiga (`cls.__name__`) teng qilib belgilanadi.
- **@fun:** Bu `Person` klassiga dekoratorni qo'llaydi. Natijada klass yaratilishi bilanoq unda qo'shimcha atribut paydo bo'ladi.

O'rnatilgan (Built-in) dekoratorlar

Python klass ta'riflarida ishlatiladigan bir nechta **o'rnatilgan dekoratorlarni** taqdim etadi. Ushbu dekoratorlar klassdagi metodlar va atributlarning xatti-harakatini o'zgartiradi, bu esa ularni boshqarish va samarali foydalanishni osonlashtiradi. Eng ko'p ishlatiladigan o'rnatilgan dekoratorlar: `@staticmethod`, `@classmethod` va `@property`.

@staticmethod

Ushbu dekorator klass instansiyasi bilan ishlamaydigan (ya'ni, `self` parametrini ishlatmaydigan) metodni aniqlash uchun qo'llaniladi. Statik metodlar klass instansiyasi (ob'ekti) orqali emas, balki to'g'ridan-to'g'ri klassning o'zi orqali chaqiriladi.

Misol: Ushbu misol klass ichida `@staticmethod`ni qanday aniqlash va undan foydalanishni ko'rsatadi.

```
class MathOperations:
    @staticmethod
    def add(x, y):
        return x + y

# Statik metoddan foydalanish
res = MathOperations.add(5, 3)
print(res)
```

Natija:

8

Tushuntirish:

- `add` — bu `@staticmethod` dekoratori bilan aniqlangan statik metod.
- Uni ob'ekt yaratmasdan (instanciya olmasdan), to'g'ridan-to'g'ri `MathOperations` klassi orqali chaqirish mumkin.

@classmethod

Ushbu dekorator klassning o'zi bilan ishlaydigan (ya'ni, `cls` parametrini ishlatuvchi) metodni aniqlash uchun qo'llaniladi. Klass metodlari klassning holatiga (state) kirish huquqiga ega va uni o'zgartira oladi, bu o'zgarish esa klassning barcha instansiyalari (ob'ektlari) uchun amal qiladi.

Misol: Ushbu kodda `Employee` (Xodim) klassi, uning `raise_amount` (maosh oshirilishi koeffitsienti) o'zgaruvchisi va butun klass uchun ushbu o'zgaruvchini yangilovchi `set_raise_amount` klass metodi aniqlangan.

```
class Employee:
    raise_amount = 1.05
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    @classmethod
    def set_raise_amount(cls, amount):
        cls.raise_amount = amount

# Klass metodidan foydalanish
Employee.set_raise_amount(1.10)
print(Employee.raise_amount)
```

Natija:

1.1

Tushuntirish:

- `set_raise_amount` — bu `@classmethod` dekoratori bilan aniqlangan klass metodidir.
- U `Employee` klassi va uning barcha instansiyalari uchun umumiy bo'lgan `raise_amount` klass o'zgaruvchisini o'zgartira oladi.

@property

Ushbu dekorator metodni xuddi atribut (o'zgaruvchi) kabi chaqirish imkonini beruvchi xususiyat (property) sifatida aniqlash uchun ishlatiladi. Bu metodning amalga

oshirilish qismini yashirish (enkapsulyatsiya), lekin foydalanuvchiga sodda interfeys taqdim etish uchun juda foydali.

Misol: Ushbu kod radiusga xavfsiz o‘zgartirishlar kiritishni va atributlarga nazorat ostida kirishni ta’minlovchi `Circle` (Aylana) klassini namoyish etadi.

```
class Circle:
    def __init__(self, radius):
        self._radius = radius

    @property
    def radius(self):
        return self._radius

    @radius.setter
    def radius(self, value):
        if value >= 0:
            self._radius = value
        else:
            raise ValueError("Radius manfiy bo'lishi mumkin emas")

    @property
    def area(self):
        return 3.14159 * (self._radius ** 2)

# Propertydan foydalanish
c = Circle(5)
print(c.radius)
print(c.area)
c.radius = 10
print(c.area)
```

Natija:

```
5
78.53975
314.159
```

Tushuntirish:

- `radius` va `area` — `@property` dekoratori yordamida aniqlangan xususiyatlardir.
- `radius` xususiyati, shuningdek, qiymatni tekshirish (validatsiya) bilan o‘zgartirishga imkon beruvchi `setter` metodiga ega.
- Ushbu xususiyatlar enkapsulyatsiyani saqlagan holda shaxsiy (private) atributlarni o‘qish va o‘zgartirish usulini taqdim etadi.

Bir nechta dekoratorlarni ketma-ket ishlatish (Chaining)

Dekoratorlarni ketma-ket ishlatish — bu bitta funksiyaga bir nechta dekoratorni qo‘llashni anglatadi. Har bir dekorator funksiyani ketma-ket o‘rab oladi va qatlamli xatti-harakatlarni qo‘shadi.

Misol: Ushbu misolda ikki xil dekoratorni turli tartibda qo'llash orqali natija qanday o'zgarishini ko'rish mumkin.

```
def decor1(func):
    def inner():
        x = func()
        return x * x
    return inner

def decor(func):
    def inner():
        x = func()
        return 2 * x
    return inner

@decor1
@decor
def num():
    return 10

@decor
@decor1
def num2():
    return 10

print(num())
print(num2())
```

Natija:

400
200

Tushuntirish:

- `num()` funksiyasida avval `decor` ishlaydi (10 soni 20 ga aylanadi), so'ngra `decor1` uni kvadratga oshiradi ($20 * 20 = 400$).
- `num2()` funksiyasida avval `decor1` ishlaydi (10 soni 100 ga aylanadi), so'ngra `decor` uni ikki barobarga oshiradi ($100 * 2 = 200$).

Dekoratorlarning amaliy hayotdagi qo'llanilishi

- **Log yuritish (Logging):** Funksiya chaqiruvlarini kuzatib borish (masalan, `@logger`).
- **Autentifikatsiya:** Veb-ilovalarda kirish huquqini cheklash (masalan, Flask/Django freymvorklarida).
- **Tezlikni cheklash (Rate Limiting):** Har bir foydalanuvchi uchun API'dan foydalanish miqdorini nazorat qilish.
- **Keshlash (Caching):** Natijalarni saqlab qolish uchun `functools.lru_cachedan` foydalanish.
- **Qayta urinish mantiqi (Retry Logic):** Muvaffaqiyatsiz chiqqan tarmoq so'rovlarini avtomatik ravishda qayta ishga tushirish.

Professional dasturlashda `@logger` (yoki logging) dekoratori funksiya qachon chaqirilgani, qanday argumentlar uzatilgani va qanday natija qaytarganini kuzatib borish uchun ishlatiladi.

Pythonda buni amalga oshirishning eng sodda va tushunarli usuli — funksiya chaqiruvlarini konsolga yoki faylga yozib boruvchi shaxsiy dekorator yaratishdir.

Amaliy Misol: Funksiya chaqiruvlarini kuzatish

```
from functools import wraps
import datetime

def logger(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        # 1. Chaqirilish vaqtini olish
        vaqt = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")

        # 2. Kirish parametrlarini yozish
        print(f"[{vaqt}] ' {func.__name__} ' funksiyasi chaqirildi.")
        print(f"    Argumentlar: positional={args}, keyword={kwargs}")

        # 3. Asl funksiyani bajarish
        natija = func(*args, **kwargs)

        # 4. Natijani ko'rsatish
        print(f"    Natija: {natija}")
        print("-" * 30)

        return natija
    return wrapper

# Dekoratorni qo'llaymiz
@logger
def hisobla(a, b, amal="qo'shish"):
    if amal == "qo'shish":
        return a + b
    elif amal == "ko'paytirish":
        return a * b

# Funksiyalarni chaqiramiz
hisobla(5, 10)
hisobla(4, 3, amal="ko'paytirish")
```

Natija (Konsolda):

```
[2025-12-25 21:05:30] ' hisobla ' funksiyasi chaqirildi.
    Argumentlar: positional=(5, 10), keyword={}
    Natija: 15
-----
[2025-12-25 21:05:31] ' hisobla ' funksiyasi chaqirildi.
    Argumentlar: positional=(4, 3), keyword={'amal': 'ko'paytirish'}
    Natija: 12
-----
```


Nima uchun bu juda foydali?

1. **Xatolarni topish (Debugging):** Katta loyihalarda qaysi funksiya qanday noto'g'ri ma'lumot qabul qilganini darhol ko'rish imkonini beradi.
2. **Kodni tozalash:** Har bir funksiya ichiga `print("Funksiya ishladi")` deb yozib chiqish shart emas. `@logger` buni avtomatik qiladi.
3. **Haqiqiy loyihalarda:** Bu ma'lumotlar konsolga emas, maxsus `.log` fayllariga yoziladi. Bu tizimda nosozlik yuz berganda "qora quti" (black box) vazifasini o'taydi.

Foydalanilgan manbalar:

1. **Real Python:** [Primer on Python Decorators](#)
2. **GeeksforGeeks:** [Decorators in Python](#)
3. **Python.org:** [PEP 318 – Decorators for Functions and Methods](#)

26-savol: `*args` va `**kwargs` nima va ular qachon ishlatiladi?

Python'da funksiyalarga o'zgaruvchan miqdordagi (soni oldindan noma'lum bo'lgan) argumentlarni uzatish uchun `*args` va `**kwargs` parametrlaridan foydalaniladi.

1. `*args` (Non-Keyword Arguments)

`*args` — funksiyaga istalgancha **pozitsion** (nomsiz) argumentlarni yuborish imkonini beradi. Python bu argumentlarni bitta **Tuple (kortej)** ichiga yig'ib qo'yadi.

- **Qachon ishlatiladi:** Funksiyaga nechta qiymat kelishi aniq bo'lmaganda (masalan, cheksiz miqdordagi sonlarni qo'shuvchi funksiya).
- **Asosiy belgisi:** Bitta yulduzcha (*).

Misol:

```
def hamma_sonlarni_qosh(*sonlar):  
    # 'sonlar' bu yerda tuple bo'ladi: (1, 2, 3, 4)  
    return sum(sonlar)  
  
print(hamma_sonlarni_qosh(1, 2))           # Natija: 3  
print(hamma_sonlarni_qosh(10, 20, 30, 5)) # Natija: 65
```

2. `**kwargs` (Keyword Arguments)

`**kwargs` — funksiyaga istalgancha **nomlangan** (kalit so'zli) argumentlarni `key=value` ko'rinishida yuborish imkonini beradi. Python ularni bitta **Dictionary (lug'at)** ichiga joylaydi.

- **Qachon ishlatiladi:** Funksiyaga qo'shimcha sozlamalar yoki turli xil xususiyatlarni nomlari bilan uzatish kerak bo'lganda.
- **Asosiy belgisi:** Ikkita yulduzcha (**).

Misol:

```
def avtomobil_malumotlari(**kwargs):
    # 'kwargs' bu yerda lug'at: {'brend': 'BMW', 'model': 'X5'}
    for kalit, qiymat in kwargs.items():
        print(f"{kalit}: {qiymat}")

avtomobil_malumotlari(brend="BMW", model="X5", yil=2023, rang="Qora")
```

3. Argumentlarning kelish tartibi

Agar bitta funksiyada oddiy argumentlar, `*args` va `**kwargs` birga kelsa, ularni quyidagi qat'iy tartibda yozish shart:

1. **Standard arguments** (Oddiy argumentlar)
2. ***args**
3. ****kwargs**

Misol:

Python

```
def buyurtma(ovqat, *qoshimchalar, **manzil):
    print(f"Asosiy ovqat: {ovqat}")
    print(f"Qo'shimchalar: {qoshimchalar}") # Tuple
    print(f"Manzil ma'lumotlari: {manzil}") # Dict

buyurtma("Pizza", "Pishloq", "Zaytun", shahar="Toshkent", kocha="Navoiy")
```

4. "Unpacking" (Yoyib yuborish) operatori

Yulduzchalar nafaqat funksiya yaratishda, balki tayyor List yoki Dict'ni funksiyaga alohida argument qilib uzatishda ham qo'llaniladi:

- *** (List/Tuple uchun):** uchta_sonni_qosh(*[10, 20, 30]) → (10, 20, 30)
- **** (Dictionary uchun):** funksiyaga(**{'a': 1, 'b': 2}) → (a=1, b=2)

Xulosa Jadvali

Sintaksis	Turi	Tuzilmasi	Vazifasi
<code>*args</code>	Positional	Tuple	Nomsiz, ketma-ket argumentlarni yig'adi
<code>**kwargs</code>	Keyword	Dict	Nomlangan (key=value) argumentlarni yig'adi

Foydalanilgan manbalar:

1. **W3Schools:** [Python *args and **kwargs](#)
2. **Real Python:** [Python args and kwargs: Demystified](#)
3. **GeeksforGeeks:** [args and kwargs in Python](#)

27-savol: `is` va `==` operatorlarining farqini tushuntiring.

Bu Python intervyularida eng ko‘p tushadigan savollardan biri. Ularning farqi juda oddiy, lekin muhim:

- **== (Equality): Qiymatlarni** tekshiradi (Ichidagi narsa bir xilmi?).
- **is (Identity): Obyektning o‘zini** tekshiradi (Xotirada bitta joyda turibdimi?).

Buni oddiy hayotiy misol va kod bilan ko‘rib chiqamiz.

1. Hayotiy Misol (Egizaklar)

Tasavvur qiling, **Hasan** va **Husan** ismli egizaklar bor. Ular bir xil kiyingan, bo‘yi ham, ko‘rinishi ham bir xil.

- **Hasan == Husan (True):** Ular **teng**. Chunki ko‘rinishi va kiyimi (qiymati) bir xil.
- **Hasan is Husan (False):** Ular **bir odam emas**. Ular ikki xil tana, ikki xil shaxs (xotirada ikki xil joy egallagan).

Agar Hasan o‘zini ko‘zguda ko‘rsa:

- **Hasan is Hasan (True):** Bu uning o‘zi.

2. Dasturiy tushuntirish

Python xotirasida har bir obyektning o‘z **ID raqami** (manzili) bo‘ladi.

- **==** operatori ikki o‘zgaruvchining **qiymati** bir xilligini tekshiradi.
- **is** operatori ikki o‘zgaruvchining **ID raqami (id())** bir xilligini tekshiradi.

3. Kod orqali isbot

Keling, ro‘yxatlar (list) misolida ko‘ramiz.

```
# Ikkita bir xil ro'yxat yaratamiz
a = [1, 2, 3]
b = [1, 2, 3]

# 1. Qiymatni tekshiramiz (==)
print(a == b)
# Natija: True
# (Chunki ikkalasining ichida ham 1, 2, 3 bor)

# 2. ID ni tekshiramiz (is)
print(a is b)
# Natija: False
# (Chunki bular kompyuter xotirasining ikki xil joyida yaratilgan.
# Ular xuddi egizaklarga o'xshaydi)
```

```
# 3. Keling, manzillarini ko'ramiz
print(id(a)) # Masalan: 2345678
print(id(b)) # Masalan: 9876543 (Boshqa raqam!)
```

Qachon is True bo'ladi?

Agar biz ikkinchi o'zgaruvchini birinchisiga tenglashtirib qo'ysak (Alias):

```
a = [1, 2, 3]
c = a # c endi a turgan joyni ko'rsatyapti

print(a is c)
# Natija: True
# (Chunki ikkalasi ham xotiradagi aynan bitta obyektga ishora qilyapti)
```

4. Muhim istisno (Kichik sonlar va Satrlar)

Pythonda bir "qopqon" bor. Python xotirani tejash uchun kichik sonlar (-5 dan 256 gacha) va qisqa so'zlar uchun xotiradan yangi joy ochmaydi, borini ishlatadi.

```
x = 10
y = 10

print(x is y)
# Natija: True (Aslida False bo'lishi kerak edi-ku?)
```

Sababi: Python 10 sonini oldindan xotiraga yozib qo'ygan (Interning). `x` va `y` o'sha bitta 10 ga bog'lanadi. Lekin ro'yxatlar (list) uchun bu qoida ishlamaydi.

Xulosa

- Agar sizga **qiymat** muhim bo'lsa (masalan, parol to'g'riligini yoki hisob-kitobni tekshirishda) `→ ==` ishlatang.
- Agar sizga obyekt **aynan o'sha narsaning o'zi** ekanligi muhim bo'lsa (ko'pincha `None` bilan tekshirishda: `if x is None:`) `→ is` ishlatang.

Foydalanilgan manbalar:

1. Real Python: [Python 'is' vs '==': Comparison Operators](#)
2. GeeksforGeeks: [Difference between == and is operator in Python](#)
3. W3Schools: [Python Comparison Operators](#)

28-savol: Lambda funksiyasi nima va uning oddiy funksiyadan qanday farqlari mavjud?

Lambda funksiyasi — bu Pythonda bir qatorli, nomsiz (anonim) funksiyadir. U `def` kalit so'zi o'rniga `lambda` kalit so'zi yordamida yaratiladi.

1. Sintaksisi

Lambda funksiyasining tuzilishi juda sodda:

Lambda parametrlar : ifoda

Misol:

```
# Oddiy funksiya
def kvadrat(x):
    return x * x

# Lambda funksiya
kvadrat_lambda = lambda x: x * x

print(kvadrat_lambda(5)) # Natija: 25
```

2. Lambda va Oddiy funksiya o'rtasidagi farqlar

Xususiyat	Oddiy funksiya (def)	Lambda funksiyasi (lambda)
Nomi	Nomga ega bo'lishi shart	Nomsiz (anonim)
Kalit so'z	def ishlatiladi	lambda ishlatiladi
Hajmi	Bir necha qatorli bo'lishi mumkin	Faqat bir qatorli (bitta ifoda)
Return	return so'zi yoziladi	return yozilmaydi (avtomatik qaytaradi)
Murakkablik	Murakkab mantiq va sikllar uchun	Sodda, vaqtinchalik amallar uchun

3. Qachon ishlatiladi?

Lambda funksiyalari asosan boshqa funksiyalar (yuqori tartibli funksiyalar) ichida qisqa muddatli amalni bajarish uchun ishlatiladi. Eng ko'p qo'llaniladigan joylari: `map()`, `filter()` va `sort()` funksiyalari.

Misollar:

- **Saralashda (Sorting):** Lug'at elementlarini ma'lum bir kalit bo'yicha saralash.

```
talabalar = [('Ali', 20), ('Vali', 18), ('Sami', 22)]
# Yoshi bo'yicha saralash
talabalar.sort(key=lambda x: x[1])
```

- **Filtrlashda (Filter):** Ro'yxatdan faqat juft sonlarni ajratib olish.

```
sonlar = [1, 2, 3, 4, 5, 6]
juft_sonlar = list(filter(lambda x: x % 2 == 0, sonlar))
```

4. Lambda funksiyasining cheklavlari

1. Faqat **bitta ifoda** (expression) qabul qiladi. Unda `if-else` (bir qatorli ko'rinishda) ishlatish mumkin, lekin `for` yoki `while` sikllarini ishlatib bo'lmaydi.
2. Katta va murakkab kodlarni lambda bilan yozish kodning **o'qilishini (readability) qiyinlashtiradi**.

Xulosa:

Lambda funksiyalari — bu "ishlat va tashla" (throw-away) tipidagi funksiyalar bo'lib, ular kodni qisqartirish va vaqtinchalik amallarni bajarish uchun juda qulaydir.

Foydalanilgan manbalar:

1. **Python Software Foundation.** "Functional Programming Modules — Lambda expressions". docs.python.org
2. **Real Python.** "How to Use Python Lambda Functions". realpython.com
3. **W3Schools.** "Python Lambda Functions". w3schools.com
4. **GeeksforGeeks.** "Python Lambda Fillers". geeksforgeeks.org
5. **Mark Lutz.** "Learning Python", 5th Edition. O'Reilly Media (Function Basics: Lambdas bo'limi).

29-savol: Fayllar bilan ishlash: faylni ochish rejimlari (r, w, a, x) va fayl yopilishi (close) nima uchun muhim?

Pythonda fayllar bilan ishlash uchun asosiy **open()** funksiyasidan foydalaniladi. Faylni ochishda uning nomi va qanday maqsadda ochilayotganini ko'rsatuvchi "rejim" (mode) ko'rsatilishi shart.

1. Faylni ochish rejimlari (Modes)

Pythonda fayllar bilan ishlashning quyidagi asosiy rejimlari mavjud:

- **'r' (Read - O'qish):** Standart rejim. Faylni faqat o'qish uchun ochadi. Agar fayl diskda mavjud bo'lmasa, **FileNotFoundError** xatoligini beradi.
- **'w' (Write - Yozish):** Faylga ma'lumot yozish uchun ochadi. Agar fayl mavjud bo'lsa, uning ichidagi barcha eski ma'lumotlarni **o'chirib yuboradi** (faylni yangidan yozadi). Agar fayl yo'q bo'lsa, yangisini yaratadi.
- **'a' (Append - Qo'shish):** Faylning oxiriga ma'lumot qo'shish uchun ochadi. Bunda eski ma'lumotlar saqlanib qoladi va yangi yozilgan matn fayl oxiriga tushadi. Fayl mavjud bo'lmasa, yangi fayl yaratadi.
- **'x' (Create - Yaratish):** Faqat yangi fayl yaratish uchun ishlatiladi. Agar ko'rsatilgan nomli fayl allaqachon mavjud bo'lsa, dastur xatolik beradi.

2. Qo'shimcha rejim belgilari

Fayl turiga qarab rejimlarga quyidagi harflar qo'shilishi mumkin:

- **'t' (Text):** Matnli fayllar bilan ishlash (standart).
- **'b' (Binary):** Binar fayllar (rasm, video, audio) bilan ishlash. Masalan: `rb` yoki `wb`.

Python'da 't' belgisi "**text mode**" (matn rejimi) degan ma'noni anglatadi. Bu rejim fayldagi ma'lumotlarni oddiy satr (`string`) ko'rinishida o'qiydi va yozadi.

Aslida, Python'da faylni ochganda agar hech narsa yozmasangiz, u avtomatik ravishda 't' rejimida ochiladi (masalan, 'r' rejimi aslida 'rt' ga teng). Lekin kodni o'qiyotgan boshqa dasturchiga aniq bo'lishi uchun uni yozish ham mumkin.

1. 't' (Matn) rejimining ishlatilishi

Ushbu misolda faylni ham yozish, ham o'qishda matn rejimidan foydalanamiz:

```
# Faylga matn rejimida yozish ('wt' - write text)
with open("test.txt", "wt", encoding="utf-8") as fayl:
    fayl.write("Salom! Bu matnli fayl.")
    # Bu yerda biz string (matn) yozyapmiz

# Faylni matn rejimida o'qish ('rt' - read text)
with open("test.txt", "rt", encoding="utf-8") as fayl:
    malumot = fayl.read()
    print(type(malumot)) # Natija: <class 'string'>
    print(malumot)
```

2. 't' (Text) va 'b' (Binary) farqi

Nega 't' kerakligini tushunish uchun uni 'b' (binar) bilan solishtirish kifoya:

- **Matn rejimi ('rt'):** Faylni o'qiganda natija **String** (matn) bo'ladi.
- **Binar rejim ('rb'):** Faylni o'qiganda natija **Bytes** (baytlar) bo'ladi.

Solishtirish misoli:

```
# Matn rejimida o'qish
with open("test.txt", "rt") as f:
    print(f"Matn rejimi: {f.read()}")
    # Natija: Salom!

# Binar rejimda o'qish
with open("test.txt", "rb") as f:
    print(f"Binar rejim: {f.read()}")
    # Natija: b'Salom!' (baytlar to'plami)
```

't' rejimi bizga fayldagi ma'lumotlarni inson tushunadigan harf va belgilar ko'rinishida olishga yordam beradi. Agar rasm yoki video emas, oddiy `.txt`, `.csv` kabi fayllar bilan ishlayotgan bo'lsangiz, har doim matn rejimidan foydalanasiz.

3. Faylni yopish (close) nima uchun muhim?

Har qanday ochilgan fayl bilan ish tugagach, **fayl.close()** metodi orqali uni yopish shart. Buning asosiy sabablari:

1. **Resurslarni tejash:** Har bir ochilgan fayl kompyuterning operativ xotirasidan joy egallaydi. Fayllar yopilmasa, tizim resurslari yetishmasligi mumkin.
2. **Ma'lumotlar butunligi:** Faylga yozilgan ma'lumotlar ko'pincha darhol diskka yozilmaydi, ular vaqtinchalik xotirada (buferda) turadi. `close()` metodi chaqirilganda barcha ma'lumotlar to'liq diskka tushadi.
3. **Xavfsizlik:** Yopilmagan fayl boshqa dasturlar tomonidan o'chirilmasligi yoki tahrirlanmasligi mumkin (fayl qulflanib qoladi).

4. Kod misoli:

```
# Faylga yozish rejimi
fayl = open("hisobot.txt", "w")
fayl.write("Bugun Python darsida fayllar bilan ishlashni o'rgandik.")
fayl.close() # Faylni yopish

# Faylni o'qish rejimi
fayl = open("hisobot.txt", "r")
matn = fayl.read()
print(matn)
fayl.close() # Faylni yopish
```

Foydalanilgan manbalar:

1. **Python Software Foundation.** "Input and Output — Python 3.12 documentation". docs.python.org
2. **W3Schools.** "Python File Handling". [w3schools.com](https://www.w3schools.com)
3. **Real Python.** "Reading and Writing Files in Python (Guide)". realpython.com
4. **Mark Lutz.** "Learning Python", 5th Edition. O'Reilly Media.

30-savol: `os` moduli yordamida fayl tizimi va kataloglar (papka) bilan ishlash imkoniyatlari.

Pythonda `os` moduli (Operating System — Operatsion Tizim) standart kutubxonaning bir qismi bo'lib, u dasturchiga operatsion tizim bilan muloqot qilish, fayl tizimi ustida amallar bajarish imkonini beradi. Ushbu modul yordamida biz papkalar yaratishimiz, fayllarni o'chirishimiz yoki joriy manzilni aniqlashimiz mumkin.

1. Kataloglar (Papkalar) bilan ishlash

Kataloglar bilan ishlashda eng ko'p ishlatiladigan funksiyalar:

- `os.getcwd()` (Get Current Working Directory): Dastur ayni vaqtda qaysi papka ichida ishlayotgan bo'lsa, o'sha papkaning to'liq manzilini qaytaradi.
- `os.listdir(path='.')`: Berilgan manzildagi barcha fayl va papkalar nomini ro'yxat (list) ko'rinishida qaytaradi. Agar manzil berilmasa, joriy papkani tekshiradi.

- **`os.mkdir("papka_nomi")`**: Yangi bitta papka yaratadi. Agar papka allaqachon mavjud bo'lsa, xatolik beradi.
- **`os.makedirs("ota/bola/nabira")`**: Ichma-ich joylashgan bir nechta papkalarni ketma-ket yaratish uchun ishlatiladi. Agar papkalar allaqachon mavjud bo'lsa, xatolik beradi.

2. Fayl va papkalarni boshqarish (O'chirish va O'zgartirish)

- **`os.rename("eski_nom.txt", "yangi_nom.txt")`**: Fayl yoki papka nomini o'zgartiradi. Shuningdek, faylni bir papkadan ikkinchisiga ko'chirishda ham ishlatish mumkin.
- **`os.remove("fayl.txt")`**: Faylni butunlay o'chirib tashlaydi. (Diqqat: papkalarni o'chira olmaydi).
- **`os.rmdir("papka_nomi")`**: Faqat bo'sh bo'lgan papkani o'chiradi.
- **`os.removedirs("yo'l")`**: Ichma-ich joylashgan bo'sh papkalar zanjirini o'chiradi.

3. `os.path` yordamchi moduli

Fayl tizimi bilan ishlashda yo'llarni (path) to'g'ri boshqarish juda muhim. Buning uchun `os.path` dan foydalaniladi:

- **`os.path.exists("manzil")`**: Berilgan manzil (fayl yoki papka) diskda mavjud bo'lsa `True`, bo'lmasa `False` qaytaradi.
- **`os.path.join("papka1", "fayl.txt")`**: Operatsion tizimdan kelib chiqib yo'llarni birlashtiradi (Windowsda `\`, Linuxda `/` belgisini o'zi qo'yadi).
- **`os.path.isfile("manzil")`**: Ko'rsatilgan manzil fayl ekanligini tekshiradi.
- **`os.path.isdir("manzil")`**: Ko'rsatilgan manzil papka ekanligini tekshiradi.

4. `os.system()` funksiyasi

Ushbu funksiya yordamida to'g'ridan-to'g'ri tizim terminaliga (CMD yoki Terminal) buyruq yuborish mumkin:

```
import os
os.system("cls") # Windowsda konsol ekranini tozalash
os.system("dir") # Joriy papkadagi fayllarni terminalda ko'rish
```

5. Amaliy misol:

```
import os

# 1. Joriy manzilni ko'rish
print("Hozirgi manzil:", os.getcwd())
```

```
# 2. Yangi papka yaratish (agar yo'q bo'lsa)
if not os.path.exists("test_papka"):
    os.mkdir("test_papka")
    print("Papka yaratildi.")

# 3. Papka ichidagi fayllarni ko'rish
print("Fayllar ro'yxati:", os.listdir())
```

OS moduli funksiyalari

Python `os` modulining ba'zi muhim funksiyalarini ko'rib chiqamiz:

- Joriy ishchi katalogni boshqarish
- Katalog yaratish
- Python yordamida fayllar va kataloglar ro'yxatini shakllantirish
- Python yordamida katalog yoki fayllarni o'chirish
- Fayl ruxsatnomalari va metadata (metama'lumotlar)

1. Joriy ishchi katalogni boshqarish (Handling Current Working Directory)

Joriy ishchi katalog (CWD — Current Working Directory) — bu Python ayni vaqtda faoliyat yuritayotgan papkadir. Faylni to'liq manzilini (full path) ko'rsatmasdan ochganingizda, Python ularni aynan shu katalog ichidan qidiradi.

Eslatma: Python skripti ishlayotgan papka "Joriy katalog" deb ataladi. Bu har doim ham Python skripti joylashgan manzil bilan bir xil bo'lmasligi mumkin.

1.1 Joriy ishchi katalogni aniqlash

Joriy ishchi katalog manzili qayerdaligini bilish uchun **`os.getcwd()`** funksiyasidan foydalaniladi.

Misol: Ushbu kod Python skriptining joriy ishchi katalogini (CWD) aniqlash va chop etish uchun 'os' modulidan foydalanadi. Manzil '`os.getcwd()`' orqali olinadi va konsolga chiqariladi.

```
import os

cwd = os.getcwd()
print("Current working directory:", cwd)
```

Natija (Output):

Current working directory: /home/nikhil/Desktop/gfg

1.2 Joriy ishchi katalogni o'zgartirish (Changing the Current working directory)

Joriy ishchi katalogni **os.chdir(path)** funksiyasi yordamida o'zgartirishingiz mumkin. U yagona argument sifatida nishon katalogining manzili (path)ni qabul qiladi va ishchi muhitni o'sha papkaga o'tkazadi.

Eslatma: Joriy ishchi katalog — bu Python skripti ayni vaqtda faoliyat yuritayotgan papkadir.

Misol: Ushbu kod joriy ishchi katalogni (CWD) ikki marta: **os.chdir('..')** yordamida bir daraja yuqoridagi papkaga o'zgartirishdan oldin va keyin tekshiradi va ko'rsatadi. Bu joriy ishchi katalog bilan qanday ishlash bo'yicha oddiy misoldir.

```
import os

def current_path():
    print("Current working directory")
    print(os.getcwd())
    print()

# Birinchi marta tekshirish
current_path()

# Bir daraja yuqoriga o'tish (../)
os.chdir('../')

# O'zgarishdan keyin tekshirish
current_path()
```

Natija (Output):

```
Current working directory before
C:\Users\Nikhil Aggarwal\Desktop\gfg
```

```
Current working directory after
C:\Users\Nikhil Aggarwal\Desktop
```

2. Katalog yaratish (Creating a Directory)

OS modulida katalog yaratish uchun turli usullar mavjud. Bular:

- **os.mkdir()**
- **os.makedirs()**

2.1 os.mkdir() dan foydalanish

os.mkdir() metodi ko'rsatilgan manzil (path) bo'yicha yangi katalog yaratish uchun ishlatiladi. Agar yaratilishi kerak bo'lgan katalog allaqachon mavjud bo'lsa, bu metod **FileExistsError** xatoligini keltirib chiqaradi.

Misol: Ushbu kod "D:/Pycharm projects/" manzili ichida "GeeksforGeeks" va "Geeks" nomli ikkita katalogni yaratadi.

- Birinchi katalog rejim (mode) ko'rsatmasdan yaratiladi.
- Ikkinchi katalog uchun maxsus rejim (0o666) beriladi, bu o'qish va yozish ruxsatlarini taqdim etadi.

```
import os

# Birinchi katalog
directory = "GeeksforGeeks"
parent_dir = "D:/Test/"
path = os.path.join(parent_dir, directory)
os.mkdir(path)
print("Directory '% s' created" % directory)

# Ikkinchi katalog (maxsus ruxsat bilan)
directory = "Geeks"
parent_dir = "D:/Test"
mode = 0o666
path = os.path.join(parent_dir, directory)
os.mkdir(path, mode)
print("Directory '% s' created" % directory)
```

Natija (Output):

```
Directory 'GeeksforGeeks' created
Directory 'Geeks' created
2.2 os.makedirs() dan foydalanish
```

os.makedirs() metodi katalogni **rekursiv** tarzda yaratish uchun ishlatiladi. Bu shuni anglatadiki, agar oxirgi (leaf) katalogni yaratish jarayonida oraliqdagi birorta katalog mavjud bo'lmasa, `os.makedirs()` ularning barchasini avtomatik ravishda yaratib chiqadi.

Misol: Ushbu kod turli ota kataloglar ichida "Nikhil" va "c" papkalarini yaratadi. U `os.makedirs` funksiyasidan foydalangan holda, agar yo'lda ko'rsatilgan ota kataloglar mavjud bo'lmasa, ularni ham yaratilishini ta'minlaydi.

```
import os

# Rekursiv yaratish 1
directory = "Nikhil"
parent_dir = "D:/Pycharm projects/GeeksForGeeks/Authors"
path = os.path.join(parent_dir, directory)
os.makedirs(path)
print("Directory '% s' created" % directory)
```

```
# Rekursiv yaratish 2 (ruxsatlar bilan)
directory = "c"
parent_dir = "D:/Pycharm projects/GeeksforGeeks/a/b"
mode = 0o666
path = os.path.join(parent_dir, directory)
os.makedirs(path, mode)
print("Directory '% s' created" % directory)
```

Natija (Output):

```
Directory 'Nikhil' created
Directory 'c' created
```

3. Python yordamida fayllar va kataloglar ro'yxatini shakllantirish (Listing out Files and Directories)

os.listdir() metodi ko'rsatilgan katalogdagi barcha fayl va papkalar ro'yxatini olish uchun ishlatiladi. Agar biz katalogni ko'rsatmasak, joriy ishchi katalogdagi fayllar va kataloglar ro'yxati qaytariladi.

Misol: Ushbu kod ildiz katalogidagi ("/") barcha fayl va kataloglarni sanab o'tadi. U ko'rsatilgan manzildagi ob'ektlar ro'yxatini olish uchun `os.listdir` funksiyasidan foydalanadi va natijalarni chop etadi.

```
import os

path = "/"
dir_list = os.listdir(path)

print("Files and directories in '", path, "' :")
print(dir_list)
```

Natija (Output):

```
Files and directories in ' / ' :
['sys', 'run', 'tmp', 'boot', 'mnt', 'dev', 'proc', 'var', 'bin', 'lib64',
'usr',
'lib', 'srv', 'home', 'etc', 'opt', 'sbin', 'media'] (Linuxda)
```

4. Python yordamida katalog yoki fayllarni o'chirish (Deleting Directory or Files)

OS moduli Pythonda katalog va fayllarni olib tashlash uchun turli metodlarni taqdim etadi. Bular:

- **os.remove()** dan foydalanish
- **os.rmdir()** dan foydalanish

4.1 os.remove() metodidan foydalanish

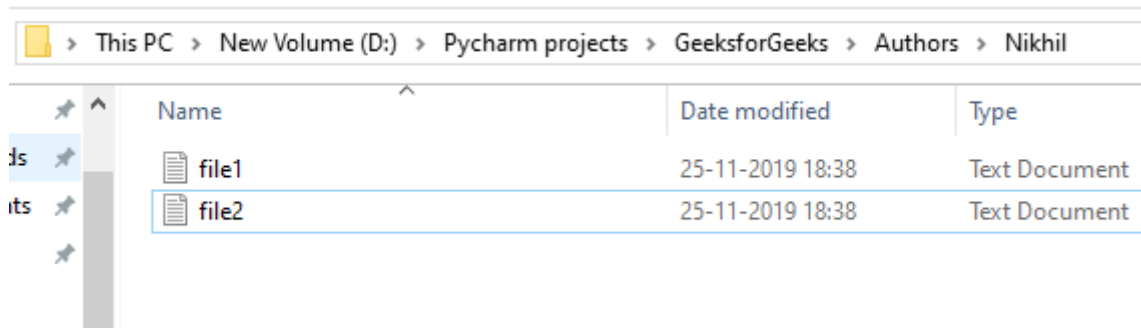
os.remove() metodi fayl yo'lini o'chirish yoki olib tashlash uchun ishlatiladi. Ushbu metod **katalogni (papkani) o'chira olmaydi**. Agar ko'rsatilgan yo'l katalog bo'lsa, metod tomonidan **OSError** xatoligi yuzaga keladi.

Misol: Faraz qilaylik, papka ichida quyidagi fayl mavjud:

```
import os

# Fayl nomi
file = 'test.txt'

# Faylni o'chirish
os.remove(file)
print(f"{file} muvaffaqiyatli o'chirildi")
```



The screenshot shows a Windows File Explorer window with the address bar displaying the path: This PC > New Volume (D:) > Pycharm projects > GeeksforGeeks > Authors > Nikhil. The main area contains a table with three columns: Name, Date modified, and Type. There are two rows of files listed.

Name	Date modified	Type
file1	25-11-2019 18:38	Text Document
file2	25-11-2019 18:38	Text Document

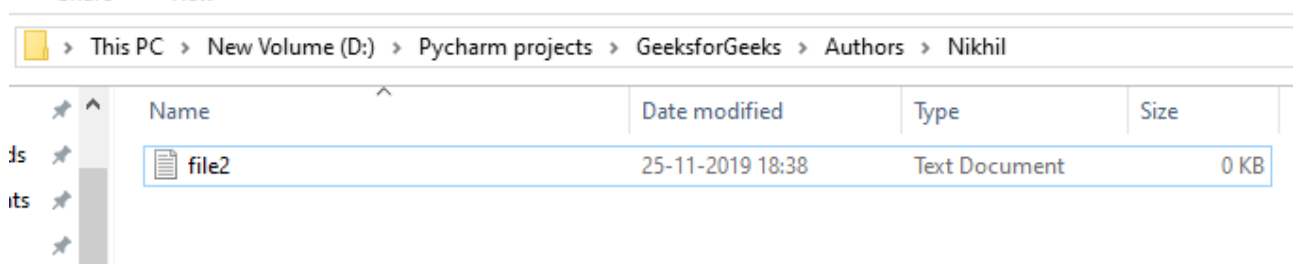
Ushbu kod ko'rsatilgan "D:/Pycharm/projects/GeeksforGeeks/Authors/Nikhil/" manzilidagi "file1.txt" nomli faylni o'chirib tashlaydi. Ko'rsatilgan yo'ldagi faylni o'chirish uchun `os.remove` funksiyasidan foydalaniladi.

```
import os

file = 'file1.txt'
location = "D:/Pycharm projects/GeeksforGeeks/Authors/Nikhil/"
path = os.path.join(location, file)

os.remove(path)
```

Natija (Output):



The screenshot shows a Windows File Explorer window with the same address bar as before. The main area contains a table with four columns: Name, Date modified, Type, and Size. Only one file, file2, is listed.

Name	Date modified	Type	Size
file2	25-11-2019 18:38	Text Document	0 KB

4.2 os.rmdir() metodidan foydalanish

os.rmdir() metodi bo'sh katalogni o'chirish yoki olib tashlash uchun ishlatiladi. Agar ko'rsatilgan yo'l bo'sh bo'lmasa, **OSError** xatoligi yuzaga keladi.

Misol: Faraz qilaylik, kataloglar quyidagicha:

```
import os

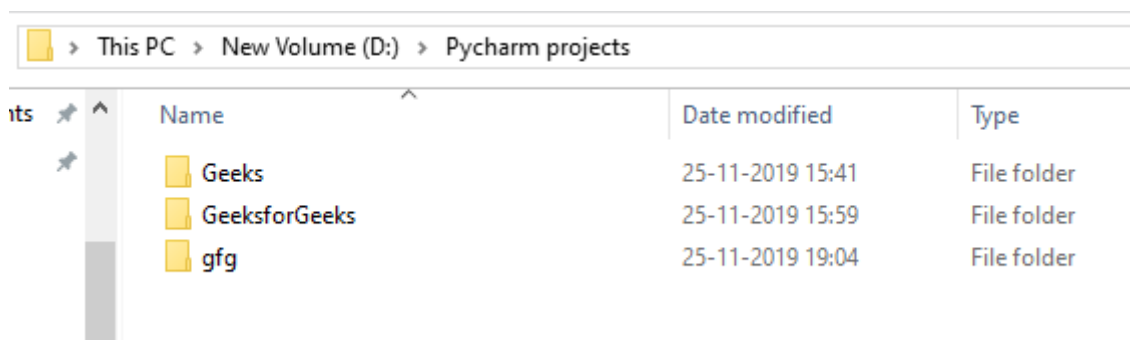
# O'chirilishi kerak bo'lgan papka nomi
directory = "Geeks"

# Ota katalog manzili
parent_dir = "D:/Pycharm projects/"

# To'liq yo'lni hosil qilish
path = os.path.join(parent_dir, directory)

# Katalogni o'chirish
os.rmdir(path)
```

Natija (Output):



This PC > New Volume (D:) > Pycharm projects			
Name	Date modified	Type	
Geeks	25-11-2019 15:41	File folder	
GeeksforGeeks	25-11-2019 15:59	File folder	
gfg	25-11-2019 19:04	File folder	

Ushbu kod "D:/Pycharm projects/" manzilida joylashgan "Geeks" nomli katalogni o'chirishga harakat qiladi. Katalogni o'chirish uchun `os.rmdir` funksiyasidan foydalaniladi. Agar katalog bo'sh bo'lsa, u o'chiriladi. Agar uning ichida fayllar yoki quyi kataloglar bo'lsa, xatolik yuzaga kelishi mumkin.

```
import os

directory = "Geeks"
parent = "D:/Pycharm projects/"
path = os.path.join(parent, directory)

os.rmdir(path)
```

Natija (Output):

(Agar katalog bo'sh bo'lsa, u o'chiriladi. Aks holda **OSError** xatoligi chiqadi.)

This PC > New Volume (D:) > Pycharm projects				
	Name	Date modified	Type	Size
	GeeksforGeeks	25-11-2019 15:59	File folder	
	gfg	25-11-2019 19:26	File folder	

5. Fayl ruxsatnomalari va metadata (File Permissions and Metadata)

Fayl va kataloglar bilan bog'liq asosiy amallardan tashqari, Pythonning **os** moduli quyi darajadagi fayl tizimi metama'lumotlari va ruxsatnomalarini boshqarish imkonini beradi. Bu skriptlar yozish, ma'murlash va tizim darajasidagi vazifalar uchun foydalidir. Ushbu turkumdagi uchta muhim metod:

- **os.chmod():** Fayl yoki katalog ruxsatnomalarini o'zgartirish.
- **os.chown():** Fayl egasi va guruhini o'zgartirish (faqat Unix tizimlari uchun).
- **os.stat():** Fayl hajmi, o'zgartirilgan vaqti, ruxsatnomalari kabi metama'lumotlarni olish.

Ushbu funksiyalar Unix/Linux tizimlaridagi `chmod`, `chown` va `ls -l` buyruqlari kabi fayl tizimi xususiyatlari bilan ishlash imkonini beradi.

5.1 os.chmod()

os.chmod() metodi fayl yoki katalogning ruxsatnomalarini (o'qish, yozish, bajarish) o'zgartirish uchun ishlatiladi. Ruxsatnomalar **sakkizlik (octal) formatda** (masalan, 0o600) uzatilishi kerak.

```
import os

# Fayl egasi uchun o'qish va yozish ruxsatini o'rnatish (0o600)
os.chmod("sample.txt", 0o600)
```

Izoh: 0o600 quyidagini anglatadi:

- **Ega (Owner):** O'qish va yozish (Read & write)
- **Guruh (Group):** Ruxsat yo'q
- **Boshqalar (Others):** Ruxsat yo'q

5.2 os.chown()

os.chown() metodi faylning egasi (owner) va guruh ID-sini (group ID) o'zgartirish imkonini beradi. Bu odatda tizim skriptlarida qo'llaniladi va tegishli ruxsatnomalarni talab qiladi. Ushbu metod faqat Unix/Linux tizimlarida ishlaydi.

```
import os

# Fayl egasi va guruhini 1000 ID-ga o'zgartirish
```



```
os.chown("example.txt", 1000, 1000)
```

Izoh:

- **uid** (user ID) va **gid** (group ID) butun son (integer) bo'lishi shart.
- Fayl egaligini o'zgartirish uchun **root** (superfoydalanuvchi) huquqi talab qilinadi.
- Qo'llab-quvvatlanmaydigan platformalarda (masalan, Windows) bu metod **AttributeError** xatoligini keltirib chiqaradi.

5.3 `os.stat()`

os.stat() metodi fayl haqidagi metama'lumotlarni, masalan, uning hajmi, ruxsatnomalari va vaqt belgilarini (timestamps) olish uchun ishlatiladi.

```
import os

# Fayl haqida ma'lumotlarni olish
stats = os.stat("example.txt")

print("Hajmi:", stats.st_size, "bayt")
print("Oxirgi o'zgartirilgan vaqti:", stats.st_mtime)
print("Ruxsatnomalar:", oct(stats.st_mode)[-3:])
```

Natija (Output):

```
Size: 48 bytes
Last modified: 1720948800.0
Permissions: 600
```

Izoh:

- **st_size**: Baytlardagi o'lcham.
- **st_mtime**: Oxirgi o'zgartirilgan vaqt belgisi (UNIX vaqti).
- **st_mode**: Fayl rejimi bitlari; ruxsatnomalar haqidagi ma'lumot uchun biz oxirgi 3 ta raqamni ajratib olamiz.
- Unix/Linux'dagi `ls -l` buyrug'i natijasiga o'xshash.

Ko'p qo'llaniladigan funksiyalar

1. `os.name` funksiyasidan foydalanish

Ushbu funksiya import qilingan operatsion tizimga bog'liq modulning nomini qaytaradi. Hozirgi vaqtda quyidagi nomlar ro'yxatga olingan: 'posix', 'nt', 'os2', 'ce', 'java' va 'riscos'.

```
import os
print(os.name)
```

Natija:

```
nt
```

Eslatma: Turli interpretatorlarda natija turlicha bo‘lishi mumkin, masalan, kodni shu yerda ishlatganingizda 'nt' natijasini berishi mumkin.

2. `os.error` funksiyasidan foydalanish

Ushbu moduldagi barcha funksiyalar noto‘g‘ri yoki kirish imkoni bo‘lmagan fayl nomlari va manzillari, yoki turi to‘g‘ri bo‘lsa-da operatsion tizim tomonidan qabul qilinmaydigan boshqa argumentlar holatida `OSError` xatoligini keltirib chiqaradi. `os.error` — bu o‘rnatilgan `OSError` istisnosining muqobili (alias) hisoblanadi.

Ushbu kod 'GFG.txt' nomli fayl tarkibini o‘qiydi. U yuzaga kelishi mumkin bo‘lgan xatolarni, xususan, faylni o‘qishda muammo bo‘lsa kelib chiqadigan **`IOError`** xatoligini boshqarish uchun `try...except` blokidan foydalanadi. Agar xatolik yuz bersa, "Problem reading: GFG.txt" degan xabar chop etiladi.

```
import os

try:
    filename = 'GFG.txt'
    # 'rU' - universal yangi qator rejimi (eskirgan)
    f = open(filename, 'r')
    text = f.read()
    f.close()
except IOError:
    print('Problem reading: ' + filename)
```

Natija:

Problem reading: GFG.txt

Eslatma: `os.error` shunchaki Python’ning o‘rnatilgan `OSError` xatoligining boshqa nomidir. Zamonaviy Python’da bevosita `OSError` (yoki yanada aniqroq holatlar uchun `FileNotFoundError`, `PermissionError` kabi) xatoliklaridan foydalanish tavsiya etiladi.

3. `os.popen()` funksiyasidan foydalanish

Ushbu metod tizim buyrug'iga yoki buyrug'idan kanal (pipe) ochadi. Qaytarilgan qiymat 'r' (o'qish) yoki 'w' (yozish) rejimiga qarab o'qilishi yoki yozilishi mumkin.

Sintaksisi:

```
os.popen(command[, mode[, bufsize]])
```

mode va bufsize parametrlari majburiy emas; agar ko'rsatilmasa, rejim uchun standart 'r' olinadi.

Ushbu kod 'GFG.txt' nomli faylni yozish rejimida ochadi, unga "Hello" so'zini yozadi, so'ngra tarkibini o'qiydi va chop etadi. `os.popen` dan foydalanish tavsiya etilmaydi va bu vazifalar uchun standart fayl amallari qo'llaniladi.

```
import os

fd = "GFG.txt"

# Standart fayl amallari
file = open(fd, 'w')
file.write("Hello")
file.close()

file = open(fd, 'r')
text = file.read()
print(text)

# popen orqali yozish (tavsiya etilmaydi)
file = os.popen(fd, 'w')
file.write("Hello")
```

Natija:

Hello

Eslatma: `popen()` uchun natija (output) ko'rinmaydi, o'zgarishlar to'g'ridan-to'g'ri faylda sodir bo'ladi. Zamonaviy Python'da bu kabi vazifalar uchun `subprocess` moduli tavsiya etiladi.

4. `os.close()` funksiyasidan foydalanish

`open()` yordamida ochilgan faylni faqat `close()` metodi orqali yopish mumkin. Ammo `os.popen()` orqali ochilgan faylni ham `close()`, ham `os.close()` orqali yopish imkoniyati mavjud. Agar biz `open()` orqali ochilgan faylni `os.close()` yordamida yopishga harakat qilsak, Python `TypeError` xatoligini qaytaradi.

```
import os

fd = "GFG.txt"
file = open(fd, 'r')
text = file.read()
print(text)

# Bu yerda xatolik yuz beradi
os.close(file)
```

Natija:

```
Traceback (most recent call last):
  File "C:\Users\GFG\Desktop\GeeksForGeeksOSFile.py", line 6, in <module>
    os.close(file)
TypeError: an integer is required (got type _io.TextIOWrapper)
```

Eslatma: `os.close()` faqat quyi darajadagi fayl deskriptorlari (ya'ni `os.open()` tomonidan qaytariladigan butun sonlar) bilan ishlaydi, `open()` funksiyasi yaratgan fayl ob'ektlari bilan emas. Yuqoridagi misolda `TypeError` kelib chiqishining sababi ham shundadir.

5. `os.rename()` funksiyasidan foydalanish

"old.txt" fayli `os.rename()` funksiyasi yordamida "new.txt" ga o'zgartirilishi mumkin. Fayl nomi faqatgina fayl mavjud bo'lsa va foydalanuvchida faylni o'zgartirish uchun yetarli ruxsatnomalar (privilege) bo'lsagina o'zgaradi.

```
import os

fd = "GFG.txt"
os.rename(fd, 'New.txt')
os.rename(fd, 'New.txt')
```

Natija (Output):

```
Traceback (most recent call last):
  File "C:\Users\GFG\Desktop\ModuleOS\GeeksForGeeksOSFile.py", line 3, in <module>
    os.rename(fd, 'New.txt')
FileNotFoundError: [WinError 2] The system cannot find the
file specified: 'GFG.txt' -> 'New.txt'
```

Izoh: "GFG.txt" nomli fayl mavjud, shuning uchun `os.rename()` birinchi marta ishlatilganda fayl nomi o'zgaradi. Funksiya ikkinchi marta chaqirilganda esa "New.txt" mavjud, lekin "GFG.txt" endi yo'q, shu sababli Python **FileNotFoundError** xatoligini qaytaradi.

6. `os.path.exists()` funksiyasidan foydalanish

Ushbu metod parametr sifatida fayl nomini qabul qilib, uning mavjud yoki mavjud emasligini tekshiradi. OS modulining ichida `PATH` deb nomlangan quyi moduli mavjud bo'lib, u orqali yana ko'plab funksiyalarni bajarish mumkin.

```
import os
# os modulini import qilish
```

```
result = os.path.exists("fayl_nomi") # fayl nomini parametr sifatida berish
print(result)
```

Natija:

False

Yuqoridagi kodda fayl mavjud bo'lmagani uchun `False` natijasi chiqadi. Agar fayl mavjud bo'lsa, natija `True` bo'ladi.

7. `os.path.getsize()` funksiyasidan foydalanish

`os.path.getsize()` funksiyasi faylning hajmini baytlarda qaytaradi. Ushbu metoddan foydalanish uchun parametr sifatida fayl nomini ko'rsatish lozim.

```
import os # os modulini import qilish

size = os.path.getsize("fayl_nomi")
print("Faylning hajmi:", size, " bayt.")
```

Natija:

Size of the file is 192 bytes.

Foydalanilgan manbalar:

1. **Python Software Foundation.** "os — Miscellaneous operating system interfaces". docs.python.org
2. **GeeksforGeeks.** "Python OS Module". [geeksforgeeks.org](https://www.geeksforgeeks.org)
3. **Real Python.** "Working With Files in Python". realpython.com
4. **Mark Lutz.** "Learning Python", 5th Edition. O'Reilly Media.

31-savol: Fayldan ma'lumot o'qish va yozish metodlari qanday ishlaydi va `with open()` usulida faylni ochishning oddiy `open()` ga qaraganda qanday qulayligi bor?

Python dasturlash tilida ma'lumotlarni saqlash va ulardan foydalanish uchun fayllar bilan ishlash tizimi juda mukammal ishlab chiqilgan. Bu jarayon asosan faylni ochish, undagi ma'lumotni manipulyatsiya qilish va faylni yopish bosqichlaridan iborat.

1. Fayldan ma'lumot o'qish metodlari

Faylni o'qish uchun u `open()` funksiyasi orqali `'r'` (read) rejimida ochiladi. Asosiy metodlar:

- **`read(size)`:** Faylning barcha tarkibini bitta yaxlit satr (`string`) sifatida o'qiydi.

- *Xususiyati:* Agar `size` ko'rsatilmasa, butun faylni o'qiydi. Katta fayllarda operativ xotirani to'ldirib yuborish xavfi bor.
- `readline()`: Fayldan faqat bitta (joriy) qatorni o'qiydi.
- *Xususiyati:* Har safar chaqirilganda kursor keyingi qatorga o'tadi. Katta hajmli ma'lumotlarni qatorma-qator qayta ishlashda eng samarali usuldir.
- `readlines()`: Fayldagi barcha qatorlarni o'qib, ularni satrlardan iborat **ro'yxat (list)** ko'rinishida qaytaradi.
- *Xususiyati:* Har bir element qator oxiridagi `\n` belgisi bilan birga saqlanadi.

2. Faylga ma'lumot yozish metodlari

Yozish uchun fayl `'w'` (eskisini o'chirib yozish) yoki `'a'` (oxiriga qo'shish) rejimlarida ochilishi shart:

- `write(string)`: Berilgan satrni faylga yozadi.
- *Muhim:* Bu metod avtomatik ravishda yangi qatorga (`\n`) o'tmaydi. Yangi qator kerak bo'lsa, satr oxiriga `\n` qo'shish dasturchi zimmasida bo'ladi.
- `writelines(list)`: Satrlardan iborat ro'yxatni (ketma-ketlikni) faylga yozish uchun ishlatiladi.

3. `with open()` (Kontekst menejeri) ning afzalliklari

Klassik usulda fayl `f = open()` bilan ochilgach, ish yakunida dasturchi `f.close()` metodini qo'lda yozishi shart. `with` operatori esa quyidagi inqilobiy afzalliklarni beradi:

1. **Avtomatik resurs boshqaruvi:** Blok ichidagi ish tugashi bilan Python faylni avtomatik ravishda yopadi.
2. **Xatoliklarga chidamlilik (Exception Safety):** Agar kod ichida kutilmagan xatolik yuz berib, dastur to'xtab qolsa ham, `with` operatori faylni baribir xavfsiz yopilishini ta'minlaydi. Oddiy usulda esa fayl "ochiq" (locked) holatda qolib, ma'lumotlar yo'qolishi mumkin.
3. **Toza va tushunarli kod:** Kod qisqaradi, o'qilishi osonlashadi va "faylni yopish esdan chiqibdi" degan xatolarga o'rin qolmaydi.

Amaliy Misollar

A) Ma'lumotni xavfsiz o'qish:

```
# with ishlatilganda close() shart emas
with open("lugat.txt", "r", encoding="utf-8") as f:
    matn = f.read()
    print(matn)
```

B) Ma'lumotni qatorma-qator yozish:

```
ismlar = ["Ali", "Vali", "Gani"]
with open("ismlar.txt", "w", encoding="utf-8") as f:
    for ism in ismlar:
        f.write(ism + "\n") # Har bir ism yangi qatordan tushadi
```

Foydalanilgan manbalar:

1. **Python.org Documentation:** "Input and Output — Reading and Writing Files" — Rasmiy texnik qo'llanma.
2. **GeeksforGeeks:** "Reading and Writing to files in Python". [geeksforgeeks.org](https://www.geeksforgeeks.org/)
3. **Real Python:** "Python Context Managers and the 'with' Statement". realpython.com
4. **W3Schools:** "Python File Handling" — Boshlang'ich darajadagi amaliy darslik.

32-savol: Obyektga Yo'naltirilgan Dasturlash (OOP) ning asosiy tamoyillari va Python dasturlash tilida uning o'rni

Obyektga Yo'naltirilgan Dasturlash (OOP) — bu dasturni "obyektlar" to'plami sifatida ko'radigan metodologiya. Bu uslub kodni qayta ishlatish (reusability), tartibga solish va murakkab tizimlarni oson boshqarish imkonini beradi.

1. Python tizimida OOPning o'rni

Python — bu **toza obyektga yo'naltirilgan til**. Bu shuni anglatadiki, Pythonda deyarli hamma narsa (sonlar, satrlar, funksiyalar va hattoki modullar ham) obyekt hisoblanadi.

- **Falsafasi:** Python dasturchiga ham funksional, ham obyektga yo'naltirilgan uslubda kod yozish imkonini beradi, lekin yirik loyihalarda OOP ustuvorlik qiladi.

2. OOPning 4 ta asosiy tamoyili

OOP quyidagi to'rtta "ustun" ustiga qurilgan:

1. **Inkapsulyatsiya (Encapsulation):** Ma'lumotlar (atributlar) va ular ustida ishlaydigan metodlarni bitta "qobiq" (klass) ichiga jamlash hamda ularni tashqi aralashuvdan himoya qilish.
2. **Vorislilik (Inheritance):** Tayyor klass (ota) asosida yangi klass (bola) yaratish. Bunda bola klass otasining barcha xususiyatlarini meros qilib oladi va o'ziga xos yangilarini qo'shadi.
3. **Polimorfizm (Polymorphism):** "Ko'p shakllilik" degani. Bir xil nomli metodning turli klasslarda turlicha ishlashi (masalan, `Ovoz_chiqar()` metodi itda "Vov", mushukda "Miyov" natijasini berishi).
4. **Abstraksiya (Abstraction):** Murakkab ichki mexanizmlarni yashirib, foydalanuvchiga faqat kerakli interfeysni ko'rsatish. (Masalan, mashina haydashda motor qanday ishlashini bilish shart emas, faqat rul va pedallar yetarli).

3. Klass va Obyekt tushunchasi

- **Klass:** Bu obyektning chizmasi yoki shabloni.
- **Obyekt:** Shu chizma asosida yaratilgan aniq bir nusxa.

Amaliy Misol

```
# Klass yaratish (Chizma)
```

```

class Hayvon:
    def __init__(self, nomi):
        self.nomi = nomi # Inkapsulyatsiya boshlanishi

    def ovoz(self):
        pass # Abstrakt tushuncha

# Vorislik va Polimorfizm
class It(Hayvon):
    def ovoz(self):
        return f"{self.nomi}: Vov-vov!"

class Mushuk(Hayvon):
    def ovoz(self):
        return f"{self.nomi}: Miyov!"

# Obyektlar yaratish
itvoy = It("Olapar")
mushukvoy = Mushuk("Simba")

print(itvoy.ovozi())
print(mushukvoy.ovozi())

```

Foydalanilgan manbalar:

1. **Python.org Documentation:** "Classes — Object-Oriented Programming". docs.python.org
2. **GeeksforGeeks:** "Python OOPs Concepts". [geeksforgeeks.org](https://www.geeksforgeeks.org)
3. **Real Python:** "Object-Oriented Programming (OOP) in Python 3". realpython.com
4. **Programiz:** "Python Object Oriented Programming". [programiz.com](https://www.programiz.com)

33-savol: Klass (Class) va Obyekt (Object) tushunchalari, ularning bog'liqligi hamda atributlar o'rtasidagi farqlar

Obyektga yo'naltirilgan dasturlashning (OOP) asosi klass va obyekt tushunchalari o'rtasidagi munosabatga qurilgan.

1. Klass va Obyekt: Ta'rif va Mantiqiy Farqlar

- **Klass (Class):** Bu obyekt yaratish uchun ishlatiladigan **andoza, shablon** yoki **qolipdir**. Klass obyektida qanday ma'lumotlar (atributlar) va qanday amallar (metodlar) bo'lishini belgilaydi. U nazariy tushuncha bo'lib, xotirada ma'lumot saqlash uchun joy egallamaydi.
- **Obyekt (Object):** Bu klassning xotirada joy egallagan **aniq nusxasi** yoki **instanciyasidir**. Klass "nima bo'lishi kerakligini" aytsa, obyekt "shu narsaning o'zi" hisoblanadi.

Xususiyati	Klass (Class)	Obyekt (Object)
Mohiyati	Nazariy qolip yoki chizma.	Amaliy, real mavjud bo'lgan nusxa.
Xotira	Xotirada ma'lumot uchun joy egallamaydi.	Yaratilganda operativ xotiradan (RAM) joy oladi.
Soni	Dasturda bir marta ta'riflanadi.	Bitta klassdan istalgancha obyekt olish mumkin.

2. Klass atributlari va Obyekt (Instance) atributlari o'rtasidagi farq

Pythonda atributlar (o'zgaruvchilar) ikki turga bo'linadi va ularning farqi juda muhim:

1. **Klass atributlari (Class Attributes):** Klass ichida, lekin metodlardan tashqarida e'lon qilinadi. Ular ushbu klassdan olingan **barcha obyektlar uchun umumiy** hisoblanadi. Agar klass atributi o'zgartirilsa, bu barcha obyektlarda aks etadi.
2. **Obyekt atributlari (Instance Attributes):** Odatda `__init__` metodi ichida `self` kalit so'zi yordamida yaratiladi. Ular **har bir obyekt uchun xususiy (unikal)** bo'ladi. Bir obyektning atributi o'zgartirilishi boshqasiga ta'sir qilmaydi.

3. O'zaro bog'liqlik (Instantiation)

Klass va obyekt o'rtasidagi bog'liqlik **instansiya yaratish** (instantiation) deb ataladi. Klass ta'riflangandan so'ng, uni funksiya kabi chaqirish orqali yangi obyekt olinadi. Obyekt yaratilganda, klassdagi barcha metodlar va klass atributlariga kirish huquqiga ega bo'ladi.

Amaliy Misol

Quyidagi misolda klass va obyekt atributlari farqini ko'rishimiz mumkin:

```
class Dasturchi:
    # Klass atributi (Hamma dasturchilar uchun umumiy)
    soha = "IT"

    def __init__(self, ism, yosh):
        # Obyekt (Instance) atributlari (Har bir dasturchi uchun xususiy)
        self.ism = ism
        self.yosh = yosh

# Obyektlar yaratish
d1 = Dasturchi("Ali", 25)
d2 = Dasturchi("Vali", 30)

# 1. Obyekt atributlarini ko'rish
print(f"{d1.ism} - {d1.soha} sohasida") # Ali - IT sohasida
print(f"{d2.ism} - {d2.soha} sohasida") # Vali - IT sohasida

# 2. Klass atributini o'zgartirish
Dasturchi.soha = "Axborot Texnologiyalari"

print(d1.soha) # Natija: Axborot Texnologiyalari
print(d2.soha) # Natija: Axborot Texnologiyalari (Hamma obyektga o'zgardi!)

# 3. Obyekt atributini o'zgartirish
d1.ism = "Alisher"
print(d1.ism) # Alisher
print(d2.ism) # Vali (Faqat d1 o'zgardi, d2 ga ta'sir qilmadi)
```

Foydalanilgan manbalar:

1. **Python Software Foundation.** "Classes - A First Look". docs.python.org
2. **Real Python.** "Instance, Class, and Static Methods Demystified". realpython.com

3. **GeeksforGeeks.** "Class and Instance Attributes in Python". [geeksforgeeks.org](https://www.geeksforgeeks.org/)
4. **W3Schools.** "Python Classes and Objects". [w3schools.com](https://www.w3schools.com/)

34-savol: Klass konstruktori (`__init__` metodi) nima va u obyekt yaratilishida qanday vazifani bajaradi?

Python dasturlash tilida klasslar bilan ishlashda obyektlarni to'g'ri shakllantirish va ularga boshlang'ich qiymatlarni berish uchun **konstruktor** tushunchasi qo'llaniladi.

1. *Klass konstruktori nima?*

Konstruktor — bu obyekt yaratilgan zahoti **avtomatik ravishda** ishga tushadigan maxsus metoddir. Pythonda konstruktor vazifasini `__init__` metodi bajaradi.

- **"Dunder" metod:** Uning nomi ikkita pastki chiziq bilan boshlanib, ikkita pastki chiziq bilan tugaydi (`__init__`), bu uning maxsus (sehrli) metod ekanligini bildiradi.

2. *`__init__` metodining asosiy vazifalari*

Obyekt yaratilish jarayonida konstruktor quyidagi muhim ishlarni amalga oshiradi:

1. **Atributlarni initsializatsiya qilish:** Obyektga xos bo'lgan xususiyatlarga (ism, yosh, rang va h.k.) dastlabki qiymatlarni biriktiradi.
2. **Xotirani tayyorlash:** Obyekt uchun ajratilgan xotira maydonida kerakli o'zgaruvchilarni joylashtiradi.
3. **Avtomatik ijro:** Dasturchi ushbu metodni qo'lda chaqirishi shart emas; u `Obyekt_nomi = Klass_nomi()` qatori yozilishi bilan o'zi ishga tushadi.

3. *`self` kalit so'zi va konstruktor*

`__init__` metodining birinchi parametri har doim `self` bo'lishi shart.

- **`self`** — bu yaratilayotgan **aynan shu obyektning o'ziga** ishora qiluvchi ko'rsatkichdir.
- U orqali biz klass ichidagi metodlardan foydalangan holda obyekt atributlariga murojaat qilamiz (masalan, `self.ism = ism`).

Amaliy Misol

Tasavvur qiling, bizda "Smartfon" klassi bor. Har bir smartfon yaratilganda uning modeli va narxi bo'lishi shart:

```
class Smartfon:
    # Klass konstruktori
    def __init__(self, model, narx):
        self.model = model # Obyekt atributiga qiymat berish
        self.narx = narx   # Obyekt atributiga qiymat berish
        print(f"Yangi obyekt yaratildi: {self.model}")
```

```
# Obyekt yaratish (shu zahoti __init__ ishga tushadi)
phone1 = Smartfon("iPhone 15", 1000)
phone2 = Smartfon("Samsung S24", 900)

# Atributlarga murojaat
print(f"Model: {phone1.model}, Narxi: {phone1.narx}$")
```

Foydalanilgan manbalar:

1. **Python Software Foundation.** "Classes - Instance Objects". docs.python.org
2. **GeeksforGeeks.** "Constructors in Python". [geeksforgeeks.org](https://www.geeksforgeeks.org)
3. **Real Python.** "The Python `__init__` Method: Initialize Your Objects". realpython.com
4. **W3Schools.** "Python `__init__()` Function". [w3schools.com](https://www.w3schools.com)

35-savol: Pythonda `self` parametri va uning klass metodlaridagi ahamiyati

1. Ta'rifi va mohiyati

self — bu Python-da klass ichidagi metodlarning birinchi parametri bo'lib, u klassdan yaratilgan muayyan **ob'ektning o'ziga (instance)** ishora qiladi.

Python — dinamik til bo'lgani uchun, ob'ekt metodini chaqirganda, o'sha ob'ektning manzili metodga avtomatik ravishda birinchi parametr sifatida uzatiladi. Dasturchi metodni e'lon qilayotganda ushbu manzilni qabul qilib olish uchun `self` o'zgaruvchisidan foydalanadi.

2. Klass metodlaridagi ahamiyati

`self` parametrining klasslar ichidagi o'rni beqiyosdir:

- **Ob'ekt atributlarini yaratish va boshqarish:** `self` orqali metod ichida e'lon qilingan o'zgaruvchilar (masalan: `self.ism = ism`) ob'ektning xususiyatiga (atributiga) aylanadi va klassning istalgan joyida mavjud bo'ladi.
- **Inkapsulyatsiyani ta'minlash:** U bitta klassdan olingan bir nechta ob'ektlar (masalan, `avto1` va `avto2`) bir-birining ma'lumotlarini aralashtirib yubormasligini kafolatlaydi. Har bir ob'ekt `self` orqali faqat o'z xotira maydoniga murojaat qiladi.
- **Metodlar zanjiri:** Klass ichidagi bir metod ichida o'zidan boshqa bir metodni chaqirishi uchun `self` dan foydalanadi (masalan: `self.hisobla()`).

3. Texnik tahlil (Kod misolida)

```
class Shaxs:
    def __init__(self, ism, yosh):
        self.ism = ism # self yordamida atribut yaratish
        self.yosh = yosh
```

```

def salomlash(self):
    # self yordamida atributni o'qish
    return f"Salom, mening ismim {self.ism}."

# Ob'ekt yaratish
shaxs1 = Shaxs("Anvar", 25)

# Chaqirish usuli:
print(shaxs1.salomlash())
# Orqa fonda Python buni shunday bajaradi: Shaxs.salomlash(shaxs1)

```

4. Muhim qaydlar

- **Standart (PEP 8):** `self` — bu majburiy nom emas (uni `me` yoki `obj` deb atash ham mumkin), lekin Python hamjamiyatining **PEP 8** standartiga ko'ra, har doim `self` ishlatish tavsiya etiladi.
- **Statik metodlar istisnosi:** Faqat `@staticmethod` va `@classmethod` dekoratorlari ishlatilganda `self` parametri odatiy ko'rinishda ishlatilmaydi.

Odatda metodlar ob'ekt bilan ishlashi uchun `self` shart, lekin `@classmethod` va `@staticmethod` bu qoidadan mustasno hisoblanadi.

Keling, har birini alohida tahlil qilaylik:

`@classmethod` (Klass metodi)

Bu metod ob'ektga emas, balki **klassning o'ziga** tegishli bo'ladi.

- **Farqi:** U birinchi parametr sifatida `self` (ob'ekt) emas, balki `cls` (klassning o'zi) parametrini qabul qiladi.
- **Ahamiyati:** Klass darajasidagi o'zgaruvchilar bilan ishlash yoki klassdan ob'ekt yaratishning muqobil usullarini (factory methods) yaratishda qo'llaniladi.

`@staticmethod` (Statik metod)

Bu metod na ob'ektga, na klassga bog'liq. U shunchaki klass ichiga mantiqan joylashtirilgan oddiy funksiyadir.

- **Farqi:** U birinchi parametr sifatida na `self`, na `cls` ni qabul qiladi. Uni chaqirish uchun ob'ekt yaratish shart emas.
- **Ahamiyati:** Agar metod klassning atributlariga (ma'lumotlariga) murojaat qilmasa, lekin mantiqan shu klassga tegishli bo'lsa (masalan, yordamchi hisob-kitoblar), statik metod ishlatiladi.

Amaliy misol tahlili:

```

class Avto:
    konstanta = "Transport vositasi"

```

```

def __init__(self, model):
    self.model = model

# 1. Odatiy metod (self kerak)
def modelni_ayt(self):
    return f"Model: {self.model}"

# 2. Klass metodi (self o'rniga cls)
@classmethod
def klass_haqida(cls):
    return f"Bu klass turi: {cls.konstanta}"

# 3. Statik metod (hech qanday maxsus parametr yo'q)
@staticmethod
def km_to_mille(km):
    return km * 0.621

```

Nima uchun bu istisno muhim?

1. **Xotirani tejash:** Agar metod ob'ekt ma'lumotlarini ishlatmasa, uni statik qilish xotira va resurslarni tejaydi.
2. **Mantiqiy guruhlash:** Funksiyani global maydonda yoyib yubormasdan, uni tegishli klass ichida saqlash kodning tartibli bo'lishini ta'minlaydi.
3. **To'g'ridan-to'g'ri chaqirish:** Statik va klass metodlarini `Avto.km_to_mille(100)` ko'rinishida, ya'ni klassdan nusxa (ob'ekt) olmasdan turib ishlatish mumkin.

Ushbu tayyorlangan ma'lumotlarni shakllantirishda Python dasturlash tilining xalqaro standartlari, rasmiy dokumentatsiyasi va nufuzli ta'limiy platformalariga tayanildi.

Foydalanilgan manbalar:

1. **Python Software Foundation.** ["Classes - Instance Objects"](#).
2. **GeeksforGeeks.** ["self in Python class"](#).
3. **Real Python.** ["Object-Oriented Programming \(OOP\) in Python 3"](#).
4. **W3Schools.** ["Python self Parameter"](#).

36-savol: Vorislik (Inheritance) tushunchasi: ota klass (Parent) va bola klass (Child) o'rtasidagi munosabat

Vorislik — bu obyektga yo'naltirilgan dasturlashning (OOP) asosiy tushunchasi bo'lib, u bir klassga (bola yoki hosila klass deb ataladi) boshqa klassdan (ota yoki asosiy klass deb ataladi) atributlar va metodlarni meros qilib olish imkonini beradi.

Misol: Bu yerda biz `info()` metodiga ega bo'lgan `Animal` (Hayvon) nomli ota klassni yaratamiz. So'ngra, `Animal` klassidan meros oluvchi va o'ziga xos xatti-harakatga ega bo'lgan `Dog` (Kuchuk) nomli bola klassni yaratamiz.

```

class Animal:

```

```

def __init__(self, name):
    self.name = name

def info(self):
    print("Hayvonning ismi:", self.name)

class Dog(Animal):
    def sound(self):
        print(self.name, "vovullaydi")

d = Dog("Buddy")
d.info()      # Meros qilib olingan metod
d.sound()    # Bola klassning o'z metodi

```

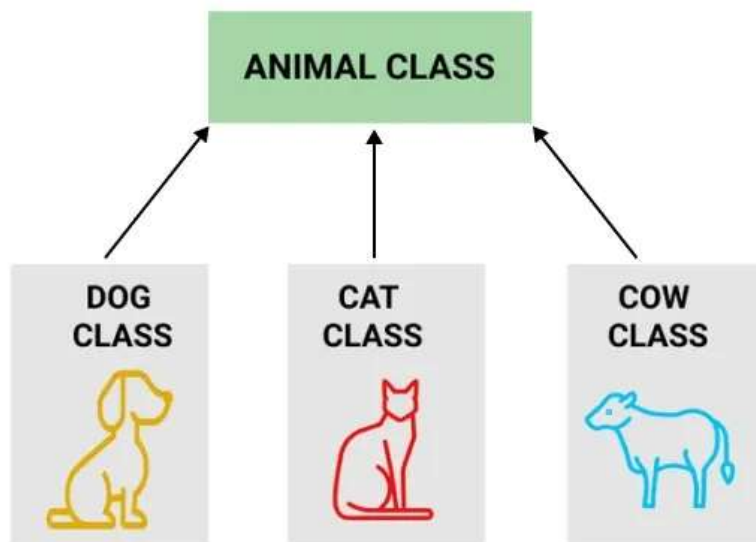
Natija:

Hayvonning ismi: Buddy

Buddy vovullaydi

Tushuntirish:

- **class Animal:** Ota klassni belgilaydi.
- **info():** Hayvonning ismini konsolga chiqaradi.
- **class Dog(Animal):** Dog klassini Animal klassining vorisi sifatida belgilaydi.
- **d.info():** Ota klassning info() metodini chaqiradi.
- **d.sound():** Bola klassning sound() metodini chaqiradi.



Nima uchun Vorislik (Inheritance) kerak?

- **Kodni qayta ishlatish:** Atribut va metodlarni klasslararo almashish orqali kod takrorlanishini kamaytiradi.
- **Real dunyo iyerarxiyasini modellashtirish:** Masalan, Hayvon -> Kuchuk yoki Shaxs -> Xodim kabi iyerarxik munosabatlarni yaratadi.

- **Texnik xizmat ko'rsatishni soddalashtirish:** Ota klassdagi bitta markazlashgan o'zgarish barcha voris klasslarda aks etadi.
- **Metodlarni qayta yozish (Overriding):** Subklasslar (bola klasslar) o'ziga xos xatti-harakatlarni belgilash imkonini beradi.
- **Kengaytiriluvchanlik:** Polimorfizm yordamida dastur dizaynini kengaytirishni qo'llab-quvvatlaydi.

super () funksiyasi

super() funksiyasi Pythonda **vorislik (inheritance)** mavzusi bilan bog'liq bo'lib, u bola klassdan (child class) turib ota klass (parent class) metodlariga murojaat qilish uchun ishlatiladi.

Misol: Dog klassi Animal klassi konstruktorini chaqirish uchun super() dan foydalanadi:

```
class Animal:
    def __init__(self, name):
        self.name = name

    def info(self):
        print("Animal name:", self.name)

# Bola klass: Dog
class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name) # Ota klass konstruktorini chaqirish
        self.breed = breed

    def details(self):
        print(self.name, "is a", self.breed)

d = Dog("Buddy", "Golden Retriever")
d.info() # Ota klass metodi
d.details() # Bola klass metodi
```

Natija:

```
Animal name: Buddy
Buddy is a Golden Retriever
```

Tushuntirish:

- super() funksiyasi Dog klassining __init__() metodi ichida Animal klassi konstruktorini chaqirish va meros qilib olingan atributni (name) ishga tushirish uchun ishlatiladi.
- Bu ota klass funksionalligini bola klassda qaytadan yozmasdan, undan qayta foydalanishni ta'minlaydi.

Python-da vorislik turlari

Vorislik ota va bola klasslar soniga qarab turli usullarda qo'llanilishi mumkin. Ular real dunyo munosabatlarini samarali modellashtirishga yordam beradi va kodni qayta ishlatishda moslashuvchanlikni ta'minlaydi.

Python vorislikning bir necha turini qo'llab-quvvatlaydi, ularni birma-bir ko'rib chiqamiz:

1. Oddiy vorislik (*Single Inheritance*)

Oddiy vorislikda bola klass faqat bitta ota klassdan meros oladi.

Misol: Ushbu misolda Employee (Xodim) bola klassi Person (Shaxs) ota klassidan xususiyatni meros qilib olishi ko'rsatilgan.

```
class Person:
    def __init__(self, name):
        self.name = name

class Employee(Person): # Employee Person-dan meros oladi
    def show_role(self):
        print(self.name, "is an employee")

emp = Employee("Sarah")
print("Name:", emp.name)
emp.show_role()
```

Natija:

```
Name: Sarah
Sarah is an employee
```

Tushuntirish: Bu yerda Employee klassi name atributini Person klassidan meros qilib oladi va o'zining show_role() metodini belgilaydi.

2. Ko'p otali vorislik (*Multiple Inheritance*)

Ko'p otali vorislikda bola klass bir vaqtning o'zida birdan ortiq ota klassdan meros olishi mumkin.

Misol: Ushbu misolda Employee (Xodim) klassi ikkita ota klassdan — Person (Shaxs) va Job (Ish) klasslaridan xususiyatlarni meros qilib olishi ko'rsatilgan.

```
class Person:
    def __init__(self, name):
        self.name = name
```



```

class Job:
    def __init__(self, salary):
        self.salary = salary

class Employee(Person, Job): # Ham Person, ham Job klasslaridan meros
    oladi
    def __init__(self, name, salary):
        Person.__init__(self, name)
        Job.__init__(self, salary)

    def details(self):
        print(self.name, "earns", self.salary)

emp = Employee("Jennifer", 50000)
emp.details()

```

Natija:

Jennifer earns 50000

Tushuntirish: Bu yerda `Employee` klassi atributlarni ham `Person`, ham `Job` klassidan oladi va u ham `name`, ham `salary` (ish haqi) ma'lumotlaridan foydalana oladi.

3. Ko'p bosqichli vorislik (Multilevel Inheritance)

Ko'p bosqichli vorislikda klass boshqa bir bola klassdan meros oladi (zanjir kabi).

Misol: Ushbu misolda `Manager` (Menejer) klassi `Employee` (Xodim) klassidan, o'z navbatida `Employee` esa `Person` (Shaxs) klassidan meros olishi ko'rsatilgan.

```

class Person:
    def __init__(self, name):
        self.name = name

class Employee(Person):
    def show_role(self):
        print(self.name, "is an employee")

class Manager(Employee): # Manager Employee-dan meros oladi
    def department(self, dept):
        print(self.name, "manages", dept, "department")

mgr = Manager("Joy")
mgr.show_role()
mgr.department("HR")

```

Natija:

Joy is an employee

Joy manages HR department

Tushuntirish: Bu yerda `Manager` klassi `Employee`dan, `Employee` esa `Person`dan meros oladi. Shuning uchun `Manager` ham ota, ham bobo klass metodlaridan foydalana oladi.

4. Iyerarxik vorislik (*Hierarchical Inheritance*)

Iyerarxik vorislikda bir nechta bola klasslar bitta ota klassdan meros oladi.

Misol: Ushbu misolda bitta `Person` (Shaxs) ota klassidan ikkita bola klass (`Employee` va `Intern`) meros olishi ko'rsatilgan.

```
class Person:
    def __init__(self, name):
        self.name = name

class Employee(Person):
    def role(self):
        print(self.name, "works as an employee")

class Intern(Person):
    def role(self):
        print(self.name, "is an intern")

emp = Employee("David")
emp.role()

intern = Intern("Eva")
intern.role()
```

Natija:

David works as an employee

Eva is an intern

Tushuntirish: Ham `Employee` (`Xodim`), ham `Intern` (`Stajyor`) klasslari `Person`dan meros oladi. Ular ota klassning xususiyatini (`name`) o'zaro bo'lishadilar, lekin o'zlarining shaxsiy metodlarini amalga oshiradilar.

5. Gibrid vorislik (*Hybrid Inheritance*)

Gibrid vorislik — bu vorislikning bir necha turining kombinatsiyasi (birlashmasi).

Misol: Ushbu misolda `TeamLead` klassi ham `Employee`dan (u o'z navbatida `Person`dan meros olgan), ham `Project`dan meros oladi. Bu bir nechta vorislik turlarining birlashmasidir.

```
class Person:
    def __init__(self, name):
        self.name = name
```

```

class Employee(Person):
    def role(self):
        print(self.name, "is an employee")

class Project:
    def __init__(self, project_name):
        self.project_name = project_name

class TeamLead(Employee, Project): # Gibrir vorislik
    def __init__(self, name, project_name):
        Employee.__init__(self, name)
        Project.__init__(self, project_name)

    def details(self):
        print(self.name, "leads project:", self.project_name)

lead = TeamLead("Sophia", "AI Development")
lead.role()
lead.details()

```

Natija:

Sophia is an employee

Sophia leads project: AI Development

Tushuntirish: Bu yerda `TeamLead` klassi `Employee`dan (u allaqachon `Person`dan meros olgan) va shuningdek `Project` klassidan meros oladi. Bu o'zida oddiy, ko'p bosqichli va ko'p ota-onali vorislik turlarini birlashtiradi -> gibrir vorislik.

Foydalanilgan manbalar:

1. **Python Software Foundation.** ["Inheritance"](https://docs.python.org/). docs.python.org.
2. **GeeksforGeeks.** ["Inheritance in Python"](https://www.geeksforgeeks.org/). geeksforgeeks.org.
3. **Real Python.** ["Inheritance and Composition: A Python OOP Guide"](https://realpython.com/). realpython.com.
4. **W3Schools.** ["Python Inheritance"](https://www.w3schools.com/). w3schools.com.

37-savol: Polimorfizm tushunchasi va uning dasturlashdagi amaliy ahamiyati

Polimorfizm (yunoncha *poly* — ko'p, *morph* — shakl) — bu turlicha ob'ektlarga bir xil nomli metodlar yoki funksiyalar orqali murojaat qilish imkoniyatidir. Soddaroq aytganda, bu "bitta interfeys, ko'p usullar" tamoyilidir.

Dasturlashda bu tushuncha quyidagicha ishlaydi: turli klasslar bir xil nomli metodga ega bo'lishi mumkin, lekin har bir klass bu metodni o'ziga xos tarzda (o'zining mantiqi bilan) amalga oshiradi.

Texnik jihatdan, Pythonda polimorfizm bir xil metod, funksiya yoki operatorga u ishlayotgan ob'ektga qarab turlicha yo'l tutish imkonini beradi. Bu kodni yanada moslashuvchan va qayta ishlatiladigan qiladi.

Nima uchun Polimorfizm kerak?

- Turli klasslar bo'ylab bir xil interfeyslarni ta'minlaydi.
- Ob'ektlarga bir xil metod chaqirig'iga turlicha javob berish imkonini beradi.
- Muayyan tiplarga emas, balki umumiy xatti-harakatlarga tayangan holda bog'liqlikni kamaytiradi (loose coupling).
- Turli tiplar bilan ishlaydigan moslashuvchan va qayta ishlatiladigan kod yozishga imkon beradi.
- Testlashni va kelajakda kodni kengaytirishni soddalashtiradi.

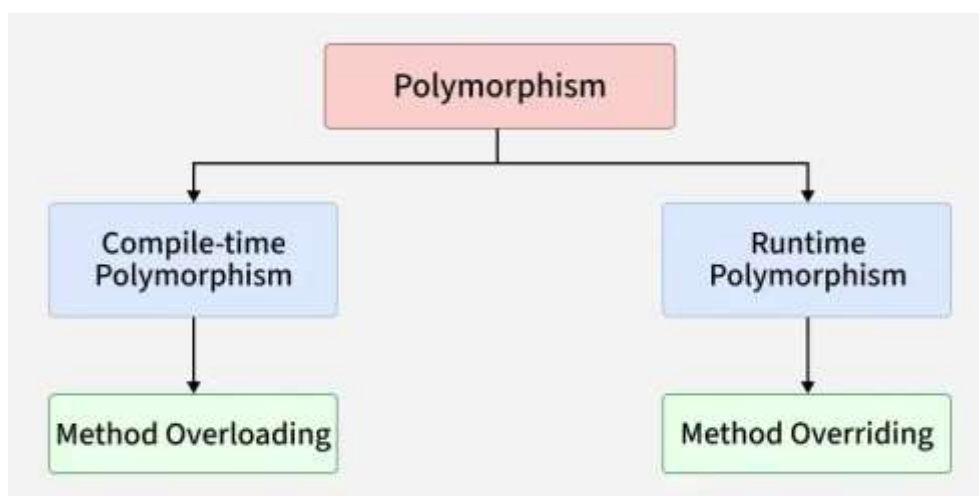
Polimorfizmga misol

Masofaviy boshqarish pulti (pult) TV, konditsioner yoki musiqa tizimi kabi bir nechta qurilmalarni boshqarishi mumkin. Siz quvvat tugmasini bosasiz va har bir qurilma turlicha javob beradi: TV yonadi, konditsioner sovutishni boshlaydi, musiqa tizimi musiqani ijro etadi.

Bu yerda polimorfizm bir xil interfeys (quvvat tugmasi), lekin qurilmaga (ob'ektga) qarab turlicha xatti-harakatni anglatadi.

Polimorfizm turlari

Pythonda polimorfizm bir xil metod yoki amalning ob'ekt yoki kontekstga qarab turlicha ishlash qobiliyatiga ishora qiladi. U asosan **kompilyatsiya vaqti (compile-time)** va **bajarilish vaqti (runtime) polimorfizmini** o'z ichiga oladi.



Har bir turni batafsil ko'rib chiqamiz.

1. Kompilyatsiya vaqti polimorfizmi (Compile-time Polymorphism)

Kompilyatsiya vaqti polimorfizmi qaysi metod yoki amalni bajarishni kompilyatsiya jarayonida, odatda metodlarni qayta yuklash (overloading) yoki operatorlarni qayta yuklash orqali hal qilishni anglatadi.

Java yoki C++ kabi tillar buni qo'llab-quvvatlaydi. Ammo Python buni qo'llab-quvvatlamaydi, chunki u dinamik tiplangan tildir — u metod chaqiriqlarini kompilyatsiya paytida emas, balki dastur bajarilish vaqtida (runtime) hal qiladi. Shuning uchun Pythonda haqiqiy metod overloading mavjud emas, lekin shunga o'xshash xatti-harakatga standart (default) yoki o'zgaruvchan argumentlar yordamida erishish mumkin.

Misol:

Ushbu kod standart va o'zgaruvchan uzunlikdagi argumentlardan foydalangan holda Pythonda metod overloading-ga misol bo'ladi. `multiply()` metodi turli miqdordagi kirish ma'lumotlari bilan ishlaydi va kompilyatsiya vaqti polimorfizmini taqlid qiladi.

```
class Calculator:
    def multiply(self, a=1, b=1, *args):
        result = a * b
        for num in args:
            result *= num
        return result

# Obyekt yaratish
calc = Calculator()

# Standart argumentlardan foydalanish
print(calc.multiply())
print(calc.multiply(4))

# Bir nechta argumentlardan foydalanish
print(calc.multiply(2, 3))
print(calc.multiply(2, 3, 4))
```

Natija:

```
1
4
6
24
```

2. Bajarilish vaqti polimorfizmi (Method Overriding)

Bajarilish vaqti (Runtime) polimorfizmi shuni anglatadiki, metodning xatti-harakati dastur ishlayotgan vaqtda, unga murojaat qilayotgan ob'ektga qarab aniqlanadi.

Pythonda bu **Metodni qayta belgilash (Method Overriding)** orqali yuz beradi — ya'ni bola klass ota klassda allaqachon belgilangan metodning o'z versiyasini taqdim etadi. Python dinamik til bo'lgani uchun u buni qo'llab-quvvatlaydi va bir xil metod chaqirig'ining turli ob'ekt tiplari uchun turlicha ishlashiga imkon beradi.

Misol:

Ushbu kod metodni qayta belgilash orqali bajarilish vaqti polimorfizmini ko'rsatadi. `sound()` metodi `Animal` asosiy klassida belgilangan va `Dog` hamda `Cat` klasslarida qayta belgilangan. Dastur bajarilayotgan vaqtda ob'ektning klassiga qarab to'g'ri metod chaqiriladi.

```
class Animal:
    def sound(self):
        return "Umumiy tovush"

class Dog(Animal):
    def sound(self):
        return "Vov-vov"

class Cat(Animal):
    def sound(self):
        return "Miyov"

# Polimorfik xatti-harakat
animals = [Dog(), Cat(), Animal()]
for animal in animals:
    print(animal.sound())
```

Natija:

```
Vov-vov
Miyov
Umumiy tovush
```

Tushuntirish: Bu yerda `sound` metodi ob'ekt `Dog`, `Cat` yoki `Animal` ekanligiga qarab turlicha ishlaydi va bu qaror dastur bajarilayotgan vaqtda qabul qilinadi. Ushbu dinamik tabiat Pythonni bajarilish vaqti polimorfizmi uchun ayniqsa kuchli qiladi.

O'rnatilgan (built-in) funksiyalarda polimorfizm

Pythonning `len()` va `max()` kabi o'rnatilgan funksiyalari polimorfikdir — ular turli ma'lumot turlari bilan ishlaydi va uzatilgan ob'ekt turiga qarab natija qaytaradi. Bu Pythonning dinamik tabiatini ko'rsatadi, ya'ni bir xil funksiya nomi kirish ma'lumotiga qarab o'z xatti-harakatini moslashtiradi.

Misol:

Ushbu kod bir xil funksiya nomidan foydalangan holda satrlar, ro'yxatlar, sonlar va belgilarni turlicha qayta ishlovchi Python o'rnatilgan funksiyalaridagi polimorfizmni namoyish etadi.

```
print(len("Salom"))      # Satr uzunligi
print(len([1, 2, 3]))    # Ro'yxat uzunligi

print(max(1, 3, 2))      # Butun sonlarning kattasi
print(max("a", "z", "m")) # Satrlardagi eng kattasi (alifbo bo'yicha)
```

Natija:

```
5
3
3
z
```

Funksiyalarda polimorfizm

Pythonda polimorfizm funksiyalarga turli xil ob'ekt turlarini qabul qilish imkonini beradi, bosharti ular kerakli xatti-harakatni qo'llab-quvvatlasa. **Duck typing (o'rdakcha turlash)** tamoyilidan foydalangan holda, Python ob'ektning turiga emas, balki unda kerakli metod mavjudligiga e'tibor qaratadi, bu esa moslashuvchan va qayta ishlatiladigan kod yozish imkonini beradi.

Misol:

Ushbu kod "duck typing" yordamida polimorfizmni namoyish etadi: `perform_task()` funksiyasi turli xil ob'ekt turlari (Pen va Eraser) bilan ishlaydi, chunki ularning har ikkalasida `.use()` metodi mavjud. Bu funksiya dizaynining moslashuvchanligini ko'rsatadi.

```
class Pen:
    def use(self):
        return "Yozmoqda"

class Eraser:
    def use(self):
        return "O'chirmoqda"

def perform_task(tool):
    print(tool.use())

perform_task(Pen())
perform_task(Eraser())
```

Natija:

```
Yozmoqda
```

Operatorlarda polimorfizm

Pythonda bir xil operator (+) operand turlariga qarab turli vazifalarni bajarishi mumkin. Bu operatorlarni qayta yuklash (operator overloading) deb ataladi. Ushbu moslashuvchanlik Pythondagi polimorfizmning asosiy jihatlaridan biridir.

Misol:

Ushbu kod operator polimorfizmini ko'rsatadi: + operatori ma'lumot turlariga qarab turlicha ishlaydi — butun sonlarni qo'shadi, satrlarni birlashtiradi va ro'yxatlarni birlashtiradi; bularning barchasi bir xil operatoridan foydalangan holda amalga oshiriladi.

```
print(5 + 10)           # Butun sonlarni qo'shish
print("Hello " + "World!") # Satrlarni birlashtirish (konkatenatsiya)
print([1, 2] + [3, 4])  # Ro'yxatlarni birlashtirish
```

Natija:

```
15
Hello World!
[1, 2, 3, 4]
```

Foydalanilgan manbalar:

1. **Python Software Foundation.** ["Polymorphism in Python"](#).
2. **Real Python.** ["The Power of Polymorphism"](#).
3. **GeeksforGeeks.** ["Polymorphism in Python"](#).

38-savol. Inkapsulyatsiya nima? Ma'lumotlarni himoyalash va yashirish (private, protected atributlar) usullari haqida gapiring.

Inkapsulyatsiya — bu klassning ichki tafsilotlarini yashirish va faqat zarur bo'lgan qismlarnigina tashqi muhitga ochib berishdir. U muhim ma'lumotlarni to'g'ridan-to'g'ri o'zgartirilishidan himoya qiladi hamda kodni xavfsiz va tartibli saqlashga yordam beradi.

Ushbu misolda inkapsulyatsiya Employee klassi ichida __salary o'zgaruvchisini private (yashirin) holatda saqlash orqali ko'rsatilgan. Unga klass tashqarisidan to'g'ridan-to'g'ri murojaat qilib bo'lmaydi.

```
class Employee:
    def __init__(self, name, salary):
        self.name = name           # ochiq atribut
        self.__salary = salary     # yashirin atribut
```



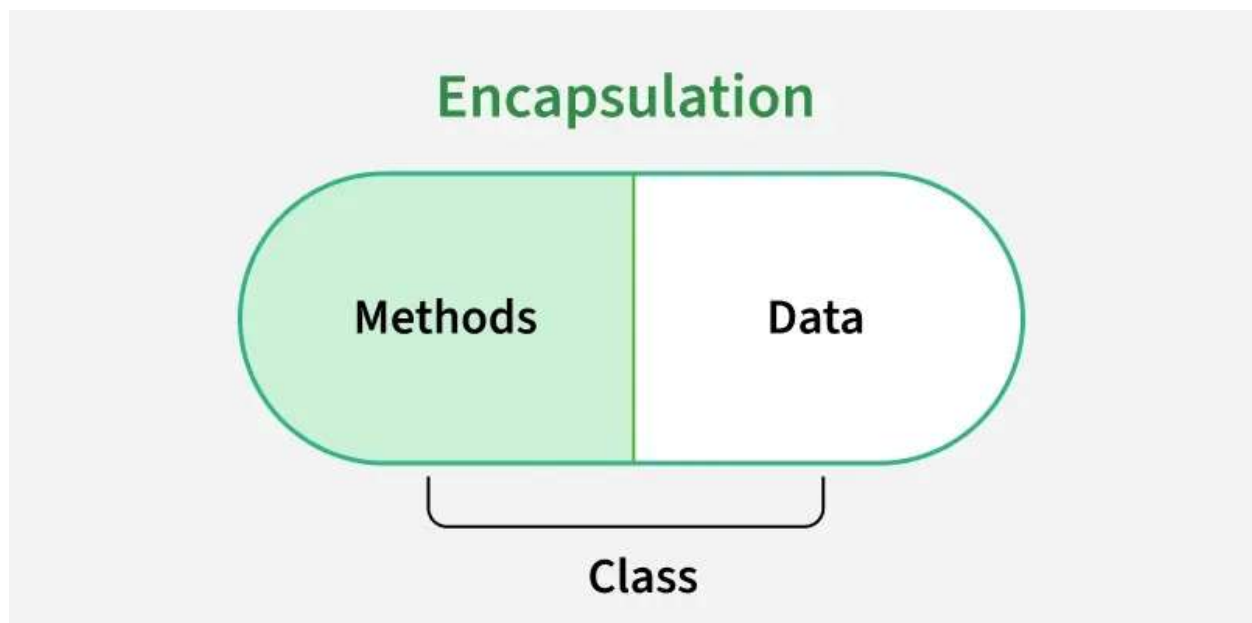
```
emp = Employee("Fedrick", 50000)
print(emp.name)
print(emp.__salary)
```

Natija (Output)

```
Fedrick
ERROR!
Traceback (most recent call last):
File "<main.py>", line 8, in
AttributeError: 'Employee' object has no attribute '__salary'
```

Tushuntirish:

`self.name = name`: Ochiq atribut, unga to'g'ridan-to'g'ri murojaat qilish mumkin.
`self.__salary = salary`: Yashirin atribut, unga to'g'ridan-to'g'ri murojaat qilib bo'lmaydi.
`print(emp.name)`: name ochiq bo'lgani uchun "Fedrick" ni chiqaradi.
`print(emp.__salary)`: __salary yashirin va berkitilgan bo'lgani sababli xatolik yuzaga keltiradi.

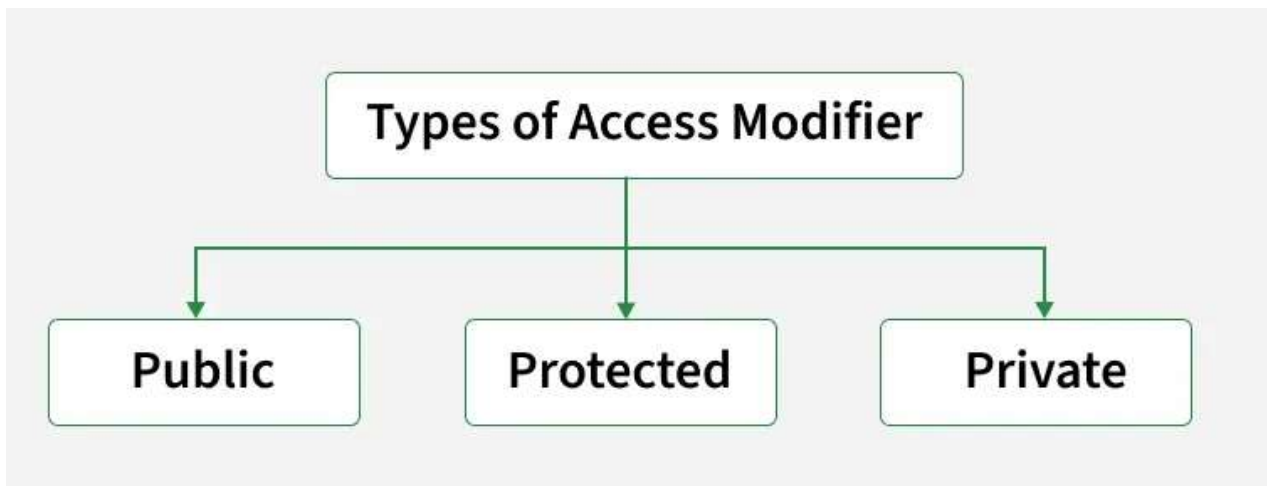


Nima uchun inkapsulyatsiya kerak? (soddaroq tilda)

- Ma'lumotlarni begona aralashuvdan va noto'g'ri o'zgartirib yuborishdan saqlaydi.
- Ma'lumotni faqat ruxsat berilgan usul bilan o'zgartirishga imkon beradi (getter va setter orqali).
- Dastur ichki tuzilishini yashirib, kodni tartibli qiladi.
- O'zgartirish va tuzatishlarni bitta joyda qilish imkonini beradi, shu sababli xizmat ko'rsatish osonlashadi.
- Haqiqiy hayotdagidek, masalan, bank hisobidagi pulga hamma ham to'g'ridan-to'g'ri kira olmasligini ifodalaydi.

Kirish darajalari (Access Specifiers)

Kirish darajalari klass a'zolari (o'zgaruvchilar va metodlar) ga klass tashqarisidan qanday murojaat qilish mumkinligini belgilaydi. Ular ma'lumotlarning ko'rinuvchanligini nazorat qilish orqali inkapsulyatsiyani amalga oshirishga yordam beradi. Kirish darajalarining uch turi mavjud:



Keling, ularni birma-bir ko‘rib chiqamiz.

1. Public a‘zolar (Public Members)

Public a‘zolar — bu klass ichidan, klass tashqarisidan yoki boshqa modullardan erkin murojaat qilish mumkin bo‘lgan o‘zgaruvchilar yoki metodlardir. Python‘da standart holatda barcha a‘zolar public hisoblanadi.

Ular hech qanday pastki chiziq (`_`) prefikssiz e‘lon qilinadi (masalan, `self.name`).

Misol: Ushbu misolda public atribut (`name`) va public metod (`display_name`) obyekt orqali klass tashqarisidan qanday chaqirilishi ko‘rsatilgan.

```
class Employee:
    def __init__(self, name):
        self.name = name # public atribut

    def display_name(self): # public metod
        print(self.name)

emp = Employee("John")
emp.display_name() # Murojaat qilish mumkin
print(emp.name) # Murojaat qilish mumkin
```

Natija (Output)

```
John
John
```

Tushuntirish:

- `self.name`: Pastki chiziqlarsiz e‘lon qilingan, shuning uchun u public atribut hisoblanadi.
- `display_name()`: Public metod bo‘lib, public atribut qiymatini ekranga chiqaradi.

- `emp.name`: Klass tashqarisidan to'g'ridan-to'g'ri murojaat qilinadi, bu public a'zolar to'liq ochiq ekanini ko'rsatadi.

Eslatma: `__init__` metodi konstruktor bo'lib, klassdan obyekt yaratilishi bilan avtomatik ishga tushadi.

2. Protected a'zolar (Protected Members)

Protected a'zolar — bu faqat klass ichida va undan meros olgan (subclass) klasslarda foydalanish uchun mo'ljallangan o'zgaruvchi yoki metodlardir. Ular to'liq yashirin (private) emas, ammo ichki foydalanish uchun deb qaraladi.

Python'da protected a'zolar bitta pastki chiziq (`_`) bilan belgilanadi (masalan, `self._name`).

Misol: Ushbu misolda protected atribut (`_age`) ning bola klass (subclass) ichida ishlatilishi ko'rsatilgan. Bu protected a'zolar asosan klass va uning vorislari uchun mo'ljallanganini anglatadi.

```
class Employee:
    def __init__(self, name, age):
        self.name = name      # ochiq (public)
        self._age = age       # himoyalangan (protected)

class SubEmployee(Employee):
    def show_age(self):
        print("Age:", self._age)  # subclass ichida murojaat qilish mumkin

emp = SubEmployee("Ross", 30)
print(emp.name)      # public – ochiq
emp.show_age()        # protected – subclass orqali foydalanildi
print(emp._age)
```

Natija (Output)

```
Ross
Age: 30
30
```

Tushuntirish:

- `self._age`: Bitta pastki chiziq bilan yozilganligi sababli protected atribut hisoblanadi.
- `SubEmployee`: `Employee` klassidan meros oladi va `_age` atributiga to'g'ridan-to'g'ri murojaat qila oladi.
- Protected a'zolar klass ierarxiyasidan tashqarida murojaat qilish tavsiya etilmaydi, biroq Python bu qoidani qat'iy majburlamaydi.

3. Private a'zolar (Private Members)

Private a'zolar — bu klass tashqarisidan to'g'ridan-to'g'ri murojaat qilib bo'lmaydigan o'zgaruvchi yoki metodlardir. Ular ichki ma'lumotlarni himoyalash va kirishni cheklash uchun ishlatiladi.

Python'da private a'zolar ikki pastki chiziq (__) bilan belgilanadi (masalan, `self.__salary`). Python bunday a'zolarga **name mangling** mexanizmini qo'llaydi, ya'ni ularni ichki tarzda qayta nomlaydi (masalan, `__salary` → `_ClassName__salary`). Bu to'g'ridan-to'g'ri murojaat qilishni oldini olish uchun qilinadi.

Misol: Ushbu misolda private atribut (`__salary`) ga faqat klass ichidagi public metod orqali murojaat qilinmoqda. Bu private a'zolar klass tashqarisidan bevosita ishlatib bo'lmashligini ko'rsatadi.

```
class Employee:
    def __init__(self, name, salary):
        self.name = name          # ochiq (public)
        self.__salary = salary    # yashirin (private)

    def show_salary(self):
        print("Salary:", self.__salary)

emp = Employee("Robert", 60000)
print(emp.name)                  # public – murojaat qilish mumkin
emp.show_salary()                # private atributga to'g'ri usulda murojaat
# print(emp.__salary)           # Xatolik: to'g'ridan-to'g'ri murojaat qilib bo'lmaydi
```

Natija (Output)

```
Robert
Salary: 60000
```

Tushuntirish:

- `self.__salary`: Ikki pastki chiziq bilan e'lon qilingan, shu sababli private atribut hisoblanadi.
- `show_salary()`: Private atributga xavfsiz tarzda murojaat qilishni ta'minlaydigan public metod.
- `emp.__salary` ga murojaat qilish `AttributeError` xatoligini keltirib chiqaradi, bu private a'zolar klass tashqarisidan to'g'ridan-to'g'ri ishlatilmasligini isbotlaydi.

Protected va Private metodlarni e'lon qilish

Python'da metodlarga kirish darajasini **nomlash kelishuvlari** orqali boshqarish mumkin:

- Metod nomi oldiga bitta pastki chiziq (`_`) qo'yilsa, u **protected metod** hisoblanadi va asosan klass ichida hamda undan meros olgan klasslarda foydalanish uchun mo'ljallanadi.
- Metod nomi oldiga ikki pastki chiziq (`__`) qo'yilsa, u **private metod** bo'ladi va **name mangling** tufayli faqat klass ichida foydalaniladi.

Eslatma: Boshqa dasturlash tillaridan farqli ravishda, Python'da public, protected yoki private kabi kirish cheklovlari til darajasida qat'iy majburlanmaydi. Biroq u nomlash kelishuvlariga amal qiladi va inkapsulyatsiyani ta'minlash uchun name mangling mexanizmidan foydalanadi.

Misol: Ushbu misolda protected metod (`_show_balance`) va private metod (`__update_balance`) orqali kirishni nazorat qilish ko'rsatilgan. Private metod balansni ichki tarzda yangilaydi, protected metod esa uni ko'rsatadi. Har ikkisi public metod (`deposit`) orqali chaqiriladi, bu Python'da inkapsulyatsiya qanday amalga oshirilishini namoyish etadi.

```
class BankAccount:
    def __init__(self):
        self.balance = 1000

    def _show_balance(self):
        print(f"Balance: ₹{self.balance}") # protected metod

    def __update_balance(self, amount):
        self.balance += amount # private metod

    def deposit(self, amount):
        if amount > 0:
            self.__update_balance(amount) # private metodga ichkaridan murojaat
            self._show_balance() # protected metodga murojaat
        else:
            print("Invalid deposit amount!")

account = BankAccount()
account._show_balance() # Ishlaydi, lekin ichki foydalanish deb qaraladi
# account.__update_balance(500) # Xatolik: private metod
account.deposit(500) # Har ikkala metoddan ichki foydalanadi
```

Natija (Output)

```
Balance: ₹1000
Balance: ₹1500
```

Tushuntirish:

- `_show_balance()`: Protected metod — tashqaridan chaqirilishi mumkin, ammo ichki yoki voris klasslarda foydalanish uchun mo'ljallangan.

- `__update_balance()`: Private metod — name mangling sababli faqat klass ichida mavjud.
- `deposit()`: Public metod bo‘lib, private va protected metodlardan xavfsiz tarzda foydalanadi.

Getter va Setter metodlari

Python’da **getter** va **setter** metodlari private atributlarga xavfsiz tarzda murojaat qilish va ularni o‘zgartirish uchun ishlatiladi. Private ma’lumotlarga to‘g‘ridan-to‘g‘ri murojaat qilish o‘rniga, ushbu metodlar nazorat ostidagi kirishni ta’minlaydi va quyidagi imkoniyatlarni beradi:

- Getter metodi yordamida ma’lumotni o‘qish
- Setter metodi yordamida tekshiruv (cheklovlar) bilan ma’lumotni yangilash

Misol: Ushbu misolda getter va setter metodlari yordamida private atribut (`__salary`) ga xavfsiz murojaat qilish va uni yangilash ko‘rsatilgan.

```
class Employee:
    def __init__(self):
        self.__salary = 50000 # yashirin (private) atribut

    def get_salary(self): # getter metodi
        return self.__salary

    def set_salary(self, amount): # setter metodi
        if amount > 0:
            self.__salary = amount
        else:
            print("Invalid salary amount!")

emp = Employee()
print(emp.get_salary()) # getter orqali maoshni olish

emp.set_salary(60000) # setter orqali maoshni yangilash
print(emp.get_salary())
```

Natija (Output)

```
50000
60000
```

Tushuntirish:

- `__salary` private atribut bo‘lgani uchun unga klass tashqarisidan to‘g‘ridan-to‘g‘ri murojaat qilib bo‘lmaydi.
- `get_salary()`: Getter metodi bo‘lib, joriy maosh qiymatini xavfsiz tarzda qaytaradi.

- `set_salary(amount)`: Setter metodi bo'lib, faqat qiymat musbat bo'lgandagina maoshni o'zgartiradi.
- `emp` obyekt ushbu metodlar orqali ma'lumotni himoyalangan holda o'qiydi va yangilaydi.

Name Mangling (ismlarni buzish yoki o'zgartirish) — bu Pythonda klass ichidagi **private** (maxfiy) atributlarni tasodifiy to'qnashuvlardan va tashqi tomondan to'g'ridan-to'g'ri kirishdan himoya qilish uchun ishlatiladigan mexanizmdir.

Siz klass ichida o'zgaruvchini ikkita pastki chiziq bilan (`__attribut`) boshlab e'lon qilganingizda, Python uning nomini avtomatik ravishda o'zgartirib qo'yadi.

1. U qanday ishlaydi?

Python `__attribut` ko'rinishidagi nomni quyidagi qolipga o'tkazadi:

`_KlassNomi__attribut`

2. Misol:

Keling, kodda buni ko'rib chiqamiz:

```
class Bank:
    def __init__(self, balans):
        self.__balans = balans # Private atribut
```

```
acc = Bank(1000)
```

```
# 1. To'g'ridan-to'g'ri murojaat qilish - XATOLIK beradi
# print(acc.__balans) # AttributeError
```

```
# 2. Obyektning barcha atributlarini ko'rish
print(dir(acc))
```

`dir(acc)` natijasini tekshirsangiz, u yerda `__balans` degan nomni topa olmaysiz. Buning o'rniga `_Bank__balans` degan nom paydo bo'lganini ko'rasiz.

3. Nima uchun Name Mangling kerak?

- **Tasodifiy o'zgartirishdan himoya:** Dasturchi adashib yoki bilmasdan private o'zgaruvchini o'zgartirib yubormasligi uchun.
- **Vorislikda nomlar to'qnashuvi oldini olish:** Agar ota klassda ham, bola klassda ham bir xil nomli private atribut bo'lsa, Name Mangling ularni klass nomi orqali farqlab beradi (masalan: `_Ota__kod` va `_Bola__kod`).

4. Unga baribir kirsam bo'ladimi?

Ha. Python "qattiq" cheklov qo'ymaydi. Agar siz juda xohlasangiz, o'sha o'zgargan nom orqali private ma'lumotni baribir o'qishingiz mumkin:

```
print(acc._Bank__balans) # 1000 natijasini beradi
```

Lekin bu **juda yomon amaliyot** hisoblanadi, chunki bu klassning ichki xavfsizlik qoidalarini buzishdir.

PHP va Python'da inkapsulyatsiya (qisqa solishtirish)

PHP:

- Inkapsulyatsiya **til darajasida qat'iy** qo'llanadi.
- `public`, `protected`, `private` kalit so'zlari bilan aniq belgilanadi.
- `private` a'zolarga klass tashqarisidan **umuman murojaat qilib bo'lmaydi**.
- Qoidalar buzilsa, **xatolik (error)** yuz beradi.

Python:

- Inkapsulyatsiya **nomlash kelishuvlari** orqali amalga oshiriladi.
- `public` — odatiy a'zolar,
`protected` — `__name`,
`private` — `__name` ko'rinishida belgilanadi.
- `protected` va `private` a'zolarga texnik jihatdan yetib borish mumkin, lekin **tavsiya etilmaydi**.
- Mas'uliyat **dasturchining o'ziga yuklanadi**.

Xulosa:

PHP inkapsulyatsiyani **qattiq qoidalar bilan majburlaydi**, Python esa **ishonch va dizayn madaniyatiga tayangan holda** inkapsulyatsiyani ta'minlaydi.

“Mas'uliyat dasturchining o'ziga yuklanadi” degani nima?

Bu shuni bildiradiki:

- **Python seni majburlamaydi,**
- **lekin nima to'g'ri, nima noto'g'ri ekanini ko'rsatib beradi.**

Ya'ni:

- Sen qoidani buzishing **mumkin,**
- lekin agar buzsang — **javobgarlik o'zingda bo'ladi.**

Aniq misol bilan

Private atribut


```
class A:
    def __init__(self):
        self.__x = 10
```

Python aytyapti:

“__x — ichki narsa, tegma”

Lekin sen baribir qilsang:

```
obj._A__x = 100
```

Python:

“Mayli, o‘zing bilasan”

Agar dastur buzilsa:

- ✗ Python aybdor emas
- ✗ Klass muallifi aybdor emas
- ✓ **Dasturchi aybdor**

Mana shu — **mas’uliyat**.

PHP bilan farq (1 jumlada)

- **PHP:** “Bo‘lishi mumkin emas ✗”
- **Python:** “Bo‘lishi mumkin, lekin to‘g‘ri emas ⚠”

Nega Python shunday qilingan?

Chunki Python:

- tez yozish
- moslashuvchanlik
- katta jamoalarda kelishuv

uchun yaratilgan.

Python falsafasi:

“We are all consenting adults here” (“Biz hammamiz ongli dasturchilarmiz”)

Mas’uliyat dasturchining o‘ziga yuklanadi — bu Python’da inkapsulyatsiya qoidalari qat’iy majburlanmasligi, ammo ularni buzish dastur dizayni va barqarorligiga zarar yetkazishi mumkinligini anglatadi.

Quyida **inkapsulyatsiya uchun hayotiy, real tushunarli misol** keltirilgan.

Bank hisob raqami (BankAccount)

Hayotda:

- Siz bank balansini to‘g‘ridan-to‘g‘ri o‘zgartira olmaysiz
- Faqat ruxsat berilgan amallar orqali ishlaysiz:
 - pul qo‘shish
 - pul yechish
 - balansni ko‘rish

Bu aynan **inkapsulyatsiya**.

```
class BankAccount:
    def __init__(self, owner):
        self.owner = owner          # public
        self.__balance = 0         # private

    def get_balance(self):          # getter
        return self.__balance

    def deposit(self, amount):      # pul qo‘shish
        if amount > 0:
            self.__balance += amount
        else:
            print("Noto‘g‘ri summa!")

    def withdraw(self, amount):     # pul yechish
        if 0 < amount <= self.__balance:
            self.__balance -= amount
        else:
            print("Balans yetarli emas!")
```

Foydalanish

```
account = BankAccount("Ali")

account.deposit(1000)
print(account.get_balance())    # 1000

account.withdraw(400)
print(account.get_balance())    # 600

# account.__balance = 999999    ✗ mumkin emas
```

Bu yerda inkapsulyatsiya qayerda?

Private ma’lumot

self.__balance

- Tashqaridan to‘g‘ridan-to‘g‘ri o‘zgartirib bo‘lmaydi

- Ichki himoyalangan ma'lumot

Ruxsat etilgan interfeys

```
deposit()  
withdraw()  
get_balance()
```

- Faqat shu metodlar orqali balansga ta'sir qilish mumkin
- Nazorat va tekshiruv mavjud

Agar inkapsulyatsiya bo'lmaganida nima bo'lardi?

```
account.balance = -5000    # ✗ mantiqsiz holat
```

Bank tizimi buziladi

Dastur ishonchsiz bo'ladi

1) Hayotiy misol: Telefon paroli (PIN)

```
class Phone:  
    def __init__(self, pin):  
        self.__pin = pin          # private  
        self.__locked = True     # private  
  
    def unlock(self, pin_input): # public method  
        if pin_input == self.__pin:  
            self.__locked = False  
            return "Telefon ochildi."  
        return "Noto'g'ri PIN! Telefon yopiq."  
  
    def lock(self):               # public method  
        self.__locked = True  
        return "Telefon qulflandi."  
  
    def status(self):            # public method  
        return "Yopiq" if self.__locked else "Ochiq"
```

Foydalanish:

```
p = Phone("1234")  
print(p.status())      # Yopiq  
print(p.unlock("0000")) # Noto'g'ri PIN! Telefon yopiq.  
print(p.unlock("1234")) # Telefon ochildi.  
print(p.status())      # Ochiq  
  
# p.__pin = "9999"     # ✗ bevosita o'zgartirib bo'lmaydi
```

2) Hayotiy misol: Harbiy obyektga kirish (AccessControl)

```
class MilitaryFacility:
```

```

def __init__(self, access_code):
    self.__access_code = access_code    # private
    self.__is_open = False              # private

def enter(self, code_input):            # public method
    if code_input == self.__access_code:
        self.__is_open = True
        return "Kirish ruxsat etildi."
    return "Kirish taqiqlanadi!"

def exit(self):                         # public method
    self.__is_open = False
    return "Chiqish amalga oshirildi."

def gate_status(self):                  # public method
    return "Darvoza ochiq" if self.__is_open else "Darvoza yopiq"

```

Foydalanish:

```

obj = MilitaryFacility("A-77")

print(obj.gate_status())    # Darvoza yopiq
print(obj.enter("X-00"))    # Kirish taqiqlanadi!
print(obj.enter("A-77"))    # Kirish ruxsat etildi.
print(obj.gate_status())    # Darvoza ochiq

# obj.__is_open = True      # ✗ bevosita o'zgartirib bo'lmaydi

```

39-savol: Obyektga yo'naltirilgan dasturlashda Abstraksiya tamoyili nima? abc moduli, ABC klassi va @abstractmethod dekoratorining vazifasi

Python'da **abstrakt klass** — bu mustaqil holda obyekt yaratib bo'lmaydigan va boshqa klasslar uchun andoza (shablon) vazifasini bajaradigan klassdir. Abstrakt klasslar voris (subclass) klasslarda majburiy tarzda amalga oshirilishi kerak bo'lgan metodlarni belgilash imkonini beradi. Bu esa bir xil interfeysni ta'minlagan holda, har bir voris klassga o'ziga xos amalga oshirishni berishga imkon yaratadi.

Abstrakt klasslardan qachon foydalaniladi?

Abstrakt klasslar quyidagi holatlarda foydalidir:

- Barcha voris klasslar uchun umumiy interfeysni belgilash zarur bo'lganda (masalan, barcha hayvonlarda `sound()` metodi bo'lishi shart).
- Bola klasslarda ayrim metodlarning majburiy amalga oshirilishini ta'minlash kerak bo'lganda.

- Umumiy funktsionallikni (aniq metodlarni) taqdim etgan holda, ayrim xatti-harakatlarni voris klasslar zimmasiga yuklash zarur bo‘lganda.

Abstrakt bazaviy klasslar (Abstract Base Classes)

Abstrakt bazaviy klass (ABC) — bu voris klasslar tomonidan majburiy amalga oshirilishi kerak bo‘lgan metodlarni belgilovchi klassdir. ABC lar barcha voris klasslar bir xil tuzilma va interfeysga rioya qilishini ta’minlaydi hamda ma’lum darajadagi abstraksiyani joriy etadi.

Python’da abstrakt bazaviy klasslarni yaratish va abstrakt metodlarni majburiy bajarilishini ta’minlash uchun **abc** moduli taqdim etilgan.

Misol

Ushbu misolda `Animal` abstrakt klassi va undagi `sound()` abstrakt metodi, shuningdek uni amalga oshiruvchi `Dog` konkret klassi ko‘rsatilgan.

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def sound(self):
        pass

class Dog(Animal):
    def sound(self):
        return "Bark"

dog = Dog()
print(dog.sound())
```

Tushuntirish:

- `Animal` — abstrakt klass bo‘lib, u `ABC` dan meros oladi va `sound()` nomli abstrakt metodni belgilaydi.
- `Animal` dan meros olgan har qanday klass `sound()` metodini albatta amalga oshirishi shart.
- `Dog` klassi `sound()` metodining o‘ziga xos amalga oshirilishini taqdim etadi, shuning uchun undan obyekt yaratish mumkin.
- `dog.sound()` chaqirilganda `"Bark"` qiymati ekranga chiqariladi.

Abstrakt metodlar (Abstract Methods)

Abstrakt metodlar — bu abstrakt klass ichida e’lon qilinadigan, ammo aniq amalga oshirilishi (implementation) bo‘lmagan metodlardir. Ular voris (subclass) klasslar uchun andoza (shablon) vazifasini bajaradi va har bir voris klass ushbu metodni o‘ziga xos tarzda amalga oshirishini majburiy qilib qo‘yadi.

Misol: Ushbu misolda abstrakt klassdan to‘g‘ridan-to‘g‘ri obyekt yaratishga urinish xatolik keltirib chiqarishi ko‘rsatilgan.

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        pass

# animal = Animal() # TypeError yuzaga keladi
```

Tushuntirish:

`make_sound()` — `Animal` klassidagi abstrakt metod bo‘lib, unda hech qanday kod yozilmagan. Shu sababli `Animal` klassidan to‘g‘ridan-to‘g‘ri obyekt yaratishga urunilganda Python `TypeError` xatoligini chiqaradi, chunki abstrakt metod hali amalga oshirilmagan.

Aniq (Concrete) metodlar

Concrete metodlar — bu to‘liq amalga oshirilgan (implementation’ga ega) metodlardir. Abstrakt klass ichida ham concrete metodlar bo‘lishi mumkin. Bunday metodlar voris klasslar tomonidan qayta yozilmasdan, to‘g‘ridan-to‘g‘ri meros qilib olinib ishlatilishi mumkin.

Misol: Ushbu misolda abstrakt metod bilan bir qatorda concrete metod ham mavjud.

```
from abc import ABC, abstractmethod

class Animal(ABC):
    def move(self):          # concrete method
        return "Hayvon harakatlanmoqda"

    @abstractmethod
    def make_sound(self):    # abstract method
        pass

class Dog(Animal):
    def make_sound(self):
        return "Bark"

dog = Dog()
print(dog.move())          # Animal'dan meros olingan concrete metod
print(dog.make_sound())    # Dog'da amalga oshirilgan abstract metod
```

Tushuntirish:

Bu yerda `Dog` klassi `Animal` klassidagi concrete metodni qayta yozmasdan

meros qilib oladi va shu bilan birga majburiy bo‘lgan `make_sound()` abstrakt metodini amalga oshiradi.

Abstrakt xususiyatlar (Abstract Properties)

Abstrakt xususiyatlar abstrakt metodlarga o‘xshash bo‘lib, lekin ular atribut (property) sifatida ishlatiladi. Ular `@property` dekoratori bilan e‘lon qilinadi va `@abstractmethod` bilan belgilanadi.

Abstrakt xususiyatlar voris klasslarda muayyan atribut (masalan, turi, kategoriyasi yoki nomi) bo‘lishini majburiy tarzda ta‘minlash uchun ishlatiladi.

Misol: Ushbu misolda `species` abstrakt xususiyat sifatida belgilangan va voris klass tomonidan amalga oshirilgan.

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @property
    @abstractmethod
    def species(self):
        pass

class Dog(Animal):
    @property
    def species(self):
        return "Canine"

dog = Dog()
print(dog.species)
```

Tushuntirish:

- `species` — `Animal` klassidagi abstrakt xususiyat bo‘lib, `@abstractmethod` bilan belgilangan.
- `Dog` klassi `species` xususiyatini amalga oshirgani sababli u aniq (concrete) klass hisoblanadi va undan obyekt yaratish mumkin.
- Abstrakt xususiyatlar voris klasslarda muayyan atribut mavjud bo‘lishini majburiy qiladi.

Abstrakt klassdan obyekt yaratish (Instantiation)

Abstrakt klasslardan to‘g‘ridan-to‘g‘ri obyekt yaratib bo‘lmaydi. Buning sababi shundaki, ular amalga oshirilmagan abstrakt metodlar yoki xususiyatlarga ega bo‘ladi. Shunday klassdan obyekt yaratishga urinish `TypeError` xatoligiga olib keladi.

Misol:

```
from abc import ABC, abstractmethod
```

```
class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        pass

# animal = Animal()
# TypeError: Can't instantiate abstract class Animal with abstract methods
# make_sound
```

Tushuntirish:

- `Animal` klassi abstrakt hisoblanadi, chunki u `make_sound()` abstrakt metodiga ega.
- `animal = Animal()` ko‘rinishida obyekt yaratishga urinish `TypeError` xatoligini keltirib chiqaradi.
- Faqat barcha abstrakt metod va xususiyatlarni to‘liq amalga oshirgan voris klasslarga (masalan, `Dog`) obyekt yaratishga yaroqli bo‘ladi.

Eslatma: Agar voris klass ota klassdagi barcha abstrakt metodlar yoki xususiyatlarni amalga oshirmasa, u ham abstrakt bo‘lib qoladi va undan obyekt yaratib bo‘lmaydi.

40-savol: Klasslarda destruktör (del metodi) nima va u qachon ishga tushadi?

Python’da obyektga yo‘naltirilgan dasturlashda nomi ikki pastki chiziq bilan boshlanib va tugaydigan maxsus metodlar mavjud bo‘lib, ular “**magic methods**” yoki “**dunder methods**” deb ataladi. Ushbu metodlar klasslarning xatti-harakatlarini ma’lum holatlarda moslashtirish imkonini beradi. Shunday metodlardan biri — `__del__` bo‘lib, u **destruktör metodi** deb ham ataladi. Ushbu bo‘limda `__del__` metodining mazmuni, undan qanday foydalanish va amaliy misollar ko‘rib chiqiladi.

`__del__` metodi nima?

`__del__` — bu Python’da obyekt yo‘q qilinish arafasida chaqiriladigan maxsus metoddır. U obyekt garbage collector tomonidan tozalanayotganda bajarilishi kerak bo‘lgan yakuniy amallarni aniqlash imkonini beradi. Bu metod ayniqsa obyekt bilan bog‘liq tashqi resurslarni (fayl deskriptori, tarmoq ulanishlari, ma’lumotlar bazasi ulanishlari va boshqalar) bo‘shatishda foydalidir.

- **Maqsad:** Obyekt yo‘q qilinishidan oldin bajarilishi lozim bo‘lgan amallarni belgilash. Bunga fayllarni yopish yoki ma’lumotlar bazasi ulanishlarini uzish kiradi.
- **Ishlatilishi:** Python’ning garbage collector’i obyektga boshqa hech qanday havola qolmaganini aniqlaganda, xotirani qayta egallashdan oldin ushbu obyektning `__del__` metodini chaqiradi.

- **Sintaksis:** `__del__` metodi quyidagi ko‘rinishda e‘lon qilinadi:

```
class ClassName:
    def __del__(self):
        # tozalash kodi shu yerda
        pass
```

`__del__` metodining chaqirilishiga misollar

`__del__` metodi obyektga bo‘lgan havolalar soni nolga tushganda Python tomonidan avtomatik chaqiriladi. Quyida `__del__` metodi qachon va qanday ishga tushishini ko‘rsatadigan uchta oddiy misol keltirilgan.

1-misol: Oddiy tozalash

Ushbu misolda obyekt yo‘q qilinayotganda xabar chiqariladi.

```
class SimpleObject:
    def __init__(self, name):
        self.name = name

    def __del__(self):
        print(f"SimpleObject '{self.name}' yo'q qilinmoqda.")

obj = SimpleObject('A')
del obj
```

Natija

SimpleObject 'A' yo‘q qilinmoqda.

2-misol: Vaqtinchalik fayllarni avtomatik tozalash

Bu misolda `TempFile` klassi obyekt yo‘q qilinganda vaqtinchalik faylni avtomatik o‘chiradi.

```
import os

class TempFile:
    def __init__(self, filename):
        self.filename = filename
        with open(self.filename, 'w') as f:
            f.write('Temporary data')

    def __del__(self):
        print(f"Vaqtinchalik fayl o'chirilmoqda: {self.filename}")
        os.remove(self.filename)

temp_file = TempFile('temp.txt')
print("Vaqtinchalik fayl yaratildi")
```

```
del temp_file
```

Natija

Vaqtinchalik fayl yaratildi
Vaqtinchalik fayl o'chirilmoqda: temp.txt

3-misol: Fayl deskriptorlarini yopish

Bu misolda `FileHandler` klassi obyekt yo'q qilinganda faylni yopishni ta'minlaydi.

```
class FileHandler:
    def __init__(self, filename):
        self.file = open(filename, 'w')
        self.file.write("Some initial data")
        print("Fayl ochildi va yozildi")

    def __del__(self):
        print("Fayl yopilmoqda")
        self.file.close()

file_handler = FileHandler('example.txt')
del file_handler
```

Natija

Fayl ochildi va yozildi
Fayl yopilmoqda

`__del__` metodidan foydalanishning afzalliklari

- **Resurslarni avtomatik bo'shatish:** Fayl, tarmoq va ma'lumotlar bazasi ulanishlarini obyekt yo'q qilinganda avtomatik yopadi. Bu resurslar sizib chiqishining oldini oladi.
- **Kod soddaligi:** Tozalash mantiqini obyekt ichida jamlash kodni tushunarli va parvarish qilishni oson qiladi.
- **Xatolarga bardoshlilik:** Obyekt hayoti davomida xatolik yuz bersa ham, resurslarni bo'shatish uchun zaxira mexanizm bo'lib xizmat qiladi.
- **Qulaylik:** Qisqa umrli yoki vaqtinchalik obyektlar uchun resurslarni alohida boshqarishdan ko'ra qulayroq bo'lishi mumkin.

Xulosa

Python'dagi `__del__` **metodi** obyekt yo'q qilinayotganda resurslarni tozalashni boshqarish uchun kuchli vositadir. U fayllarni yopish, blokirovkalarni bo'shatish yoki tarmoq ulanishlarini uzish kabi amallarni bajarish imkonini beradi. To'g'ri qo'llanganda, `__del__` metodi ilovalarning barqarorligi va ishonchliligini oshirib, resurslar sizib chiqishining oldini oladi.

41-savol: Mutable (o'zgaruvchan) va Immutable (o'zgarmas) tiplar

Mutable (o'zgaruvchan) tiplar — bu yaratilgandan so'ng ichki holatini (qiymatini) o'zgartirish mumkin bo'lgan ma'lumot turlaridir. Ya'ni obyekt xotirada o'sha joyda qoladi, faqat uning mazmuni yangilanadi.

Immutable (o'zgarmas) tiplar — bu yaratilgandan keyin o'zgartirib bo'lmaydigan ma'lumot turlaridir. Agar qiymatni “o'zgartirishga” urinsak, aslida yangi obyekt yaratiladi va eski obyekt o'zgarmay qoladi.

Mutable (o'zgaruvchan) tiplarga misollar

- `list` (ro'yxat)
- `dict` (lug'at)
- `set` (to'plam)
- `bytearray`

Misol:

```
lst = [1, 2, 3]
lst[0] = 10
print(lst)
```

Natija:

```
[10, 2, 3]
```

Bu yerda ro'yxatning o'zi o'zgardi, yangi obyekt yaratilgani yo'q.

Immutable (o'zgarmas) tiplarga misollar

- `int`, `float`
- `str` (satr)
- `tuple` (kortej)
- `frozenset`
- `bool`

Misol:

```
x = 10
x = x + 5
print(x)
```

Bu holatda 10 o'zgarmaydi, balki 15 qiymatli **yangi obyekt** yaratiladi.

Asosiy farqlar (qisqa jadval ko'rinishida)

Xususiyat	Mutable	Immutable
-----------	---------	-----------

Qiymatni o'zgartirish	Mumkin	Mumkin emas
Xotirada obyekt	O'sha obyekt saqlanadi	Yangi obyekt yaratiladi
Misollar	list, dict, set	int, str, tuple
Xavfsizlik	Pastroq	Yuqoriroq
Hash sifatida ishlatish	Yo'q	Ha (masalan, dict kaliti)

Nima uchun bu farq muhim?

- **Xavfsizlik:** Immutable obyektlar tasodifiy o'zgarishlardan himoyalangan bo'ladi.
- **Ishonchlilik:** Funksiya va metodlarda kutilmagan yon ta'sirlar (side effects) kamayadi.
- **Tezlik va optimizatsiya:** Python immutable obyektlarni samaraliroq boshqaradi.
- **Dictionary va set:** Faqat immutable obyektlargina kalit (key) sifatida ishlatiladi.

Foydalanilgan adabiyotlar va havolalar

1. Python Software Foundation — *Data model: Objects, values and types*
<https://docs.python.org/3/reference/datamodel.html>
2. Python Software Foundation — *Built-in Types*
<https://docs.python.org/3/library/stdtypes.html>
3. Real Python — *Mutable vs Immutable Objects in Python*
<https://realpython.com/python-mutable-vs-immutable-types/>
4. GeeksforGeeks — *Mutable vs Immutable Objects in Python*
<https://www.geeksforgeeks.org/mutable-vs-immutable-objects-in-python/>

42-savol: Pythonda Modul va Paketlar (Modules and Packages) nima?

Modul (Module) — bu Python'da kodni mantiqiy qismlarga ajratish uchun yaratilgan **.py** kengaytmali fayl bo'lib, unda funksiyalar, klasslar va o'zgaruvchilar saqlanadi. Moduldan foydalanish kodni tartibli, qayta ishlatiladigan va oson boshqariladigan qiladi.

Misol (modul):

```
# math_utils.py
def add(a, b):
    return a + b
import math_utils
print(math_utils.add(3, 5))
```

Paket (Package)

Paket — bu bir nechta modullarni bir joyga jamlagan **papka (katalog)** bo'lib, u katta loyihalarda kodni ierarxik tartibda tashkil etish uchun ishlatiladi. Paket odatda `__init__.py` fayliga ega bo'ladi (Python 3.3+ da majburiy emas, lekin tavsiya etiladi).

Tuzilma misoli (paket):

```
my_package/  
├── __init__.py  
├── module1.py  
└── module2.py
```

Paketdan foydalanish:

```
from my_package import module1
```

Modul va Paketning asosiy farqlari

Belgisi	Modul	Paket
Turi	Bitta .py fayl	Modullar jamlangan papka
Vazifasi	Kichik funksional kod	Katta loyihani tartiblash
Import	import module	import package.module

Import qilish usullari

- `import module` — modulni to‘liq import qiladi
- `from module import func` — faqat kerakli funktsiyani oladi
- `import module as m` — modulga qisqa nom beradi

Misol:

```
import math  
from math import sqrt  
import numpy as np
```

Nima uchun modul va paketlar kerak?

- Kodni **tartibli** va **o‘qilishi oson** qiladi
- **Qayta foydalanish** imkonini beradi
- Katta loyihalarda **texnik xizmat** va **kengaytirishni** osonlashtiradi
- Standart va tashqi kutubxonalar bilan ishlashni ta’minlaydi

Pythondagi mashhur modul va paketlarga misollar

Mashhur modullar (Modules)

Quyidagi modullar Python’ning **standart kutubxonasi** tarkibiga kiradi va ko‘p ishlatiladi:

- **math** — matematik funksiyalar (ildiz, trigonometrik funksiyalar va h.k.)
- **os** — operatsion tizim bilan ishlash (fayl, papka, muhit o‘zgaruvchilari)
- **sys** — Python interpreter va tizim parametrlari bilan ishlash
- **random** — tasodifiy sonlar generatsiyasi

- **datetime** — sana va vaqt bilan ishlash
- **json** — JSON formatdagi ma'lumotlarni o'qish va yozish
- **re** — muntazam ifodalar (regular expressions)

Misol:

```
import math
print(math.sqrt(16))
```

Mashhur paketlar (Packages)

Quyidagi paketlar odatda **tashqi kutubxonalar** bo'lib, `pip` orqali o'rnatiladi:

- **NumPy** — sonli hisob-kitoblar va massivlar bilan ishlash
- **Pandas** — jadval ko'rinishidagi ma'lumotlar (DataFrame) bilan ishlash
- **Matplotlib** — grafik va diagrammalar chizish
- **Seaborn** — statistik vizualizatsiya
- **Requests** — HTTP so'rovlar bilan ishlash
- **Flask** — yengil web-ilovalar yaratish
- **Django** — yirik web-ilovalar va backend tizimlar
- **PyQt5** — grafik interfeysli (GUI) dasturlar yaratish
- **Scikit-learn** — mashinali o'qitish algoritmlari
- **TensorFlow / PyTorch** — chuqur o'qitish (Deep Learning)

Misol:

```
import numpy as np
arr = np.array([1, 2, 3])
print(arr.mean())
```

Modul va Paketga birgalikda misol

```
import pandas as pd
from matplotlib import pyplot as plt
```

Bu yerda:

- `pandas` — paket
- `pyplot` — `matplotlib` paketining ichidagi modul

Qaysi sohalarda qaysi paketlar ko'p ishlatiladi?

- **Ma'lumotlar tahlili:** NumPy, Pandas
- **Sun'iy intellekt:** Scikit-learn, TensorFlow, PyTorch
- **Web dasturlash:** Flask, Django
- **Grafika va GUI:** PyQt5, Tkinter
- **Tarmoq va API:** Requests, Socket

Xulosa

Python'da **modullar** — aniq vazifani bajaruvchi `.py` fayllar, **paketlar** esa bir nechta modullarni birlashtiruvchi kutubxonalar bo'lib, ular dasturlashning deyarli barcha sohalarida keng qo'llaniladi.

Foydalanilgan adabiyotlar va havolalar

1. Python Software Foundation — *The Python Standard Library*
<https://docs.python.org/3/library/index.html>
2. Python Software Foundation — *Modules*
<https://docs.python.org/3/tutorial/modules.html>
3. Python Software Foundation — *Packages*
<https://docs.python.org/3/tutorial/packages.html>
4. Real Python — *Python Modules and Packages – An Overview*
<https://realpython.com/python-modules-packages/>
5. GeeksforGeeks — *Python Modules and Packages*
<https://www.geeksforgeeks.org/python-modules/>

43-savol: Xatoliklarni boshqarishda **try–except–else–finally** bloklarining vazifalari

Python'da xatoliklarni (exceptions) boshqarish uchun **try–except–else–finally** bloklaridan foydalaniladi. Bu mexanizm dastur to'xtab qolmasdan, xatolarni nazorat ostida ushlash va to'g'ri qayta ishlash imkonini beradi.

try–except–else–finally tuzilmasi

```
try:
    # Xatolik chiqishi mumkin bo'lgan kod
except SomeError:
    # Xatolik yuz berganda bajariladigan kod
else:
    # Xatolik bo'lmasa bajariladigan kod
finally:
    # Har doim bajariladigan kod
```

try bloki

- Xatolik yuz berishi **mumkin bo'lgan** kod yoziladi.
- Agar bu yerda xatolik chiqsa, dastur mos `except` blokka o'tadi.

Misol:

```
try:
    x = int("abc")
```

except bloki

- Xatolik yuz berganda **ushlab olinadi**.
- Dastur to‘xtab qolmaydi, muqobil amallar bajariladi.

Misol:

```
try:
    x = int("abc")
except ValueError:
    print("Son kiritilmadi")
```

else bloki

- **Faqat xatolik bo‘lmaganda** ishlaydi.
- Odatda muvaffaqiyatli bajarilgan natijalar shu yerda qayta ishlanadi.

Misol:

```
try:
    x = int("10")
except ValueError:
    print("Xato")
else:
    print("Kiritilgan son:", x)
```

finally bloki

- **Har qanday holatda** (xato bo‘lsa ham, bo‘lmasa ham) bajariladi.
- Resurslarni yopish (fayl, ulanish va h.k.) uchun juda muhim.

Misol:

```
try:
    f = open("data.txt", "r")
    print(f.read())
except FileNotFoundError:
    print("Fayl topilmadi")
finally:
    print("Dastur yakunlandi")
```

To‘liq misol (hammasi birga)

```
try:
    a = int(input("a ni kiriting: "))
    b = int(input("b ni kiriting: "))
    result = a / b
except ValueError:
    print("Faqat son kiriting")
```



```
except ZeroDivisionError:
    print("0 ga bo'lish mumkin emas")
else:
    print("Natija:", result)
finally:
    print("Hisoblash tugadi")
```

Nima uchun try-except kerak?

- Dastur to'satdan to'xtab qolmasligi uchun
- Foydalanuvchiga tushunarli xabar berish uchun
- Resurslarni to'g'ri yopish va tizimni barqaror saqlash uchun

Bloklarning birgalikda ishlash tartibi:

Holat	try	except	else	finally
Xato yuz bermasa	✓ Ishlaydi	✗ Ishlamaydi	✓ Ishlaydi	✓ Ishlaydi
Xato yuz bersa	⚠ To'xtaydi	✓ Ishlaydi	✗ Ishlamaydi	✓ Ishlaydi

Nima uchun bu muhim?

1. **Dastur barqarorligi:** Dastur qizil xatolar bilan "portlab" ketmaydi.
2. **Resurslar xavfsizligi:** `finally` bloki yordamida ochiq qolgan fayllar yoki tarmoq ulanishlarini har doim yopish kafolatlanadi.
3. **Mantiqiy ajratish:** Asosiy kod, xatolikni ushlash va yakuniy amallar bir-biridan aniq ajraladi.

Fayllar bilan ishlashda `try-except-finally` bloklarining qo'llanilishi eng muhim amaliy misollardan biri hisoblanadi. Chunki fayl ochilganda kutilmagan xato (masalan, fayl ichida xato ma'lumot bo'lishi) yuz bersa, dastur to'xtab qolsa ham **fayl ochiq qolmasligi** kerak.

Odatda biz `with open()` (kontekst menejeri) ishlatamiz, lekin uning ichki ishlash mantiqi aynan quyidagi `try-finally` zanjiriga asoslanadi:

```
fayl = None
try:
    # 1. Faylni o'qish uchun ochamiz
    fayl = open("ma'lumot.txt", "r", encoding="utf-8")

    # 2. Ma'lumotni o'qiymiz va sonli amal bajaramiz
    tartib_raqam = int(fayl.read().strip())
    natija = 100 / tartib_raqam

except FileNotFoundError:
    print("Xato: Fayl topilmadi!")
except ValueError:
    print("Xato: Fayl ichidagi ma'lumot son emas!")
except ZeroDivisionError:
    print("Xato: Fayldagi son nolga teng!")
```

```

else:
    # Xato bo'lmasa ishlaydi
    print(f"Hisoblash yakunlandi: {natija}")
finally:
    # Eng muhim qismi: xato bo'lsa ham, bo'lmasa ham faylni yopamiz
    if fayl:
        fayl.close()
        print("Fayl muvaffaqiyatli yopildi.")

```

Bu usulning afzalliklari:

1. **Fayl xavfsizligi:** Agar `int(fayl.read())` qismida xato yuz bersa, dastur darhol `except` ga sakraydi. Agar bizda `finally` bo'lmasa, `fayl.close()` qatori o'qilmay qolib ketishi va fayl xotirada ochiq qolishi mumkin edi.
2. **Toza xotira:** `finally` bloki har qanday holatda (hatto `except` ichida ham boshqa kutilmagan xato chiqsa) faylni yopishni kafolatlaydi.
3. **Aniq nazorat:** Dasturchi xatoning turiga qarab (fayl yo'qligi yoki ma'lumot xatoligi) alohida mantiq yozish imkoniga ega bo'ladi.

Bilish foydali: Zamonaviy Python'da `with open(...)` ishlatish tavsiya etiladi, chunki u `finally` blokini o'zi avtomatik boshqaradi (faylni yopishni unutib qo'yish xavfi yo'q).

Python'da `except` blokini ishlatishda biz xatoliklarni turli darajalarda ushlashimiz mumkin. Ularni asosan **3 ta asosiy yondashuvga** bo'lish mumkin:

1. Maxsus xatoliklarni ushlash (Specific Exceptions)

Bu eng to'g'ri va xavfsiz usul. Bunda biz aynan qaysi xato turini kutayotganimizni ko'rsatamiz. Pythonning juda ko'p o'rnatilgan xato turlari bor.

Eng ko'p uchraydigan turlari:

- **ValueError:** Ma'lumot turi to'g'ri, lekin qiymati noto'g'ri bo'lganda (masalan, "abc" ni `int()` ga o'tkazishda).
- **TypeError:** Amal noto'g'ri ma'lumot turi ustida bajarilganda (masalan, sonni matnga qo'shishda).
- **IndexError:** Ro'yxatda mavjud bo'lmagan indeksga murojaat qilinganda.
- **KeyError:** Lug'atda (dict) mavjud bo'lmagan kalit so'ralganda.
- **ZeroDivisionError:** Sonni nolga bo'lganda.
- **FileNotFoundError:** Ochilayotgan fayl topilmaganda.

```

try:
    son = int(input("Son kiriting: "))
    print(10 / son)
except ValueError:
    print("Siz son kiritmadingiz!")
except ZeroDivisionError:
    print("Nolga bo'lish mumkin emas!")

```

2. Bir nechta xatoni bitta blokda ushlash

Agar bir nechta xato turi uchun bir xil javob qaytarmoqchi bo'lsangiz, ularni qavs ichida guruhlashingiz mumkin.

```
try:
    # kod...
except (ValueError, TypeError, ZeroDivisionError):
    print("Ma'lumot kiritishda yoki hisoblashda xatolik yuz berdi!")
```

3. Umumiy (Generic) xatolarni ushlash

Barcha turdagi xatolarni bittada ushlash uchun ishlatiladi. Bu usuldan ehtiyot bo'lib foydalanish kerak, chunki u kutilmagan mantiqiy xatolarni ham yashirib yuborishi mumkin.

- `except Exception:` Barcha standart xatolarni ushlaydi (tavsiya etiladi).
- `except:` Mutloq barcha xatolarni, hatto dasturni to'xtatish buyrug'ini ham ushlaydi (tavsiya etilmaydi).

```
try:
    # murakkab kod
except Exception as e:
    print(f"Kutilmagan xato yuz berdi: {e}")
```

Python Xatolar Iyerarxiyasi (Hierarchy)

Pythonda xatolar daraxtsimon tuzilishga ega. Masalan, `ZeroDivisionError` bu `ArithmeticError`ning bir turi, u esa o'z navbatida `Exception` klassiga tegishli.

as kalit so'zi bilan ishlash

Xato haqidagi tizimli xabarni o'zgaruvchiga yuklab, uni chop etish uchun `as` so'zi ishlatiladi:

```
try:
    result = 10 / 0
except ZeroDivisionError as xato_xabari:
    print(f"Tizim xabari: {xato_xabari}")
    # Natija: Tizim xabari: division by zero
```

Qisqa tavsiya: Har doim imkon qadar aniq xato turini (`ValueError`, `KeyError` va h.k.) ko'rsatishga harakat qiling. Bu kodingizni tahlil qilishni (debugging) osonlashtiradi.

O'z shaxsiy xato turingizni yaratish (User-defined Exceptions) — bu dasturingizda mantiqiy xatolik yuz berganda (masalan, foydalanuvchi manfiy yosh kiritisa yoki paroli juda qisqa bo'lsa) Pythonning standart xatolaridan tashqari maxsus xabar chiqarish uchun kerak bo'ladi.

Pythonda barcha xatolar `Exception` klassidan meros oladi. Shuning uchun biz ham yangi klass yaratib, uni `Exception`ga bog'lab qo'yamiz.

1. Oddiy Custom Exception yaratish

```
# 1. Yangi xato klassini yaratamiz
class YoshXatosi(Exception):
    """Foydalanuvchi noto'g'ri yosh kiritganda chiqadigan xato"""
    pass

# 2. Uni ishlatib ko'ramiz
yosh = -5

try:
    if yosh < 0:
        # 'raise' yordamida xatoni majburan chaqiramiz
        raise YoshXatosi("Yosh manfiy bo'lishi mumkin emas!")
except YoshXatosi as e:
    print(f"Maxsus xato ushlandi: {e}")
```

2. Mukammalroq Custom Exception (Atributlar bilan)

Ba'zida xato haqida ko'proq ma'lumot saqlash kerak bo'ladi. Masalan, parolda necha harf kamligini ko'rsatish:

```
class ParolJudaQisqa(Exception):
    def __init__(self, uzunlik, kamida):
        self.uzunlik = uzunlik
        self.kamida = kamida
        self.message = f"Parol {uzunlik} belgidan iborat. Kamida {kamida} ta bo'lishi shart!"
        super().__init__(self.message)

def parol_tekshir(parol):
    if len(parol) < 8:
        raise ParolJudaQisqa(len(parol), 8)
    print("Parol qabul qilindi!")

try:
    p = input("Yangi parol yarating: ")
    parol_tekshir(p)
except ParolJudaQisqa as e:
    print(f"Xatolik: {e.message}")
```

Nima uchun bu kerak?

1. **Mantiqiy xatolar:** Python `yosh = -5` degan kodda xato ko'rmaydi (chunki bu matematik butun son). Lekin inson hayotida bu mantiqsiz. Custom exception buni boshqarishga yordam beradi.
2. **Katta loyihalar:** Agar siz kutubxona yoki paket yaratayotgan bo'lsangiz, boshqa dasturchilar sizning xatolaringizni `except MeningKutubxonamXatosi: deb` osonroq ushlab olishlari mumkin.

3. **Kodni o'qilishi:** `raise ValueError` degandan ko'ra `raise InsufficientFundsError` (Mablag' yetarli emas xatosi) deyish kodning ma'nosini darhol tushuntiradi.

Foydalanilgan adabiyotlar va havolalar

1. Python Software Foundation — *Errors and Exceptions*
<https://docs.python.org/3/tutorial/errors.html>
2. Python Software Foundation — *Built-in Exceptions*
<https://docs.python.org/3/library/exceptions.html>
3. Real Python — *Python Exceptions: An Introduction*
<https://realpython.com/python-exceptions/>
4. GeeksforGeeks — *Try, Except, Else and Finally in Python*
<https://www.geeksforgeeks.org/try-except-else-and-finally-in-python/>

44. Dastur kodini modullarga ajratish va kutubxonalarni import qilish (import, from ... import) qoidalari.

Katta hajmdagi dasturlarni yozishda barcha kodni bitta faylga joylashtirish uni o'qish, testlash va tahrirlashni qiyinlashtiradi. Shu sababli, dastur kodi mantiqiy qismlarga — **modullarga** ajratiladi.

1. Modul va Paket tushunchasi

- **Modul:** Bu `.py` kengaytmali oddiy fayl bo'lib, unda funksiyalar, klasslar va o'zgaruvchilar saqlanadi.
- **Paket:** Bu bir nechta modullarni o'z ichiga olgan jild (papka). Paket ichida odatda `__init__.py` fayli bo'ladi (Python 3.3 dan keyin bu majburiy emas, lekin tavsiya etiladi).

2. Kutubxonalarni import qilish usullari

Pythonda kodni boshqa fayldan chaqirishning bir necha usuli mavjud:

A. *To'liq import qilish (import)*

Ushbu usul butun modulni yuklaydi. Modul ichidagi funksiyalarga murojaat qilish uchun modul nomi ko'rsatilishi shart.

```
import math
```

```
print(math.sqrt(16)) # 4.0
print(math.pi)      # 3.1415...
```

B. Tanlab import qilish (*from ... import*)

Agar sizga modulning faqat ma'lum bir qismi (funksiya yoki klass) kerak bo'lsa, ushbu usuldan foydalaniladi. Bunda modul nomini yozish shart emas.

```
from math import sqrt, pi
```

```
print(sqrt(25)) # 5.0  
print(pi)
```

C. Taxallus (*Alias*) bilan import qilish (*as*)

Agar modul nomi uzun bo'lsa yoki mavjud o'zgaruvchi bilan to'qnashib qolsa, unga qisqa nom berish mumkin.

```
import pandas as pd  
import numpy as np  
from math import factorial as fact
```

```
print(fact(5)) # 120
```

D. Barchasini import qilish (*from ... import **) — Tavsiya etilmaydi

Ushbu usul modul ichidagi barcha narsani joriy kodga olib kiradi. Bu kodda nomlar to'qnashuviga (name clashing) olib kelishi mumkinligi sababli PEP 8 tomonidan tavsiya etilmaydi.

3. Shaxsiy modullarni yaratish va ulash

Tasavvur qiling, sizda `hisoblagich.py` fayli bor:

```
# hisoblagich.py fayli  
def qoshish(a, b):  
    return a + b
```

```
PI = 3.14
```

Uni asosiy dasturga (`main.py`) quyidagicha ulaymiz:

```
# main.py fayli  
import hisoblagich  
  
natija = hisoblagich.qoshish(5, 10)  
print(natija) # 15
```

4. Import qilish tartibi (PEP 8 qoidalari)

Kutubxonalarni import qilishda quyidagi ketma-ketlikka amal qilish tavsiya etiladi:

1. **Standart kutubxonalar** (Python bilan birga keladigan: `os`, `sys`, `math`).
2. **Tashqi kutubxonalar** (`pip` orqali o'rnatilgan: `requests`, `pandas`, `django`).
3. **Shaxsiy modullar** (O'zingiz yozgan fayllar).

5. `if __name__ == "__main__":` bloki

Modulni import qilganda, uning ichidagi kodlar avtomatik bajarilib ketmasligi uchun ushbu blokdan foydalaniladi. Bu blok faqat fayl to'g'ridan-to'g'ri ishga tushirilgandagina ishlaydi.

```
def main():  
    print("Dastur ishga tushdi")  
  
if __name__ == "__main__":  
    main()
```

Paket ichidagi moduldan aniq bir obyekt (klass, funksiya yoki o'zgaruvchi) import qilish :

```
from [paket].[modul] import [obyekt]
```

Foydali adabiyotlar:

1. **Python Docs:** [Modules](#) — Rasmiy qo'llanma.
2. **Real Python:** [Python Modules and Packages: An Introduction](#).
3. **W3Schools:** [Python Modules](#).